

ai-plan-service — Giải thích chi tiết toàn bộ code, workflow & data flow

ai-plan-service là service Python (FastAPI, cổng 8092) đứng sau **MAI** — trợ lý du lịch AI của Mravel. MAI **không phải model tự train**: nó là một lớp điều phối (orchestration) quanh một LLM OpenAI-compatible có sẵn, dựa trên bốn trụ cột tool-calling · retrieval · structured output · approval. Nguyên tắc cốt lõi là **DRAFT-FIRST**: AI chỉ *đề xuất* (draft / proposal); không có thao tác ghi nào lên plan thật xảy ra cho đến khi người dùng bấm **Duyệt** (/approve) hoặc **Áp dụng** (/apply-edits).

Tài liệu này viết bằng **tiếng Việt**; mọi code, identifier, tên field, tên tool và comment trong repo vẫn giữ **tiếng Anh** theo quy ước dự án. Đây là tài liệu **code-level**, đi sâu hơn so với **AI_OVERVIEW.md** (mức flow) và **README.md** (mức tổng quan); hai tài liệu kia nên đọc trước để nắm bức tranh lớn.

Mục lục

- Kiến trúc tổng quan
- Bốn hành vi của MAI (workflow)
- Luồng A — Tạo kế hoạch mới + Hỏi đáp/khám phá (plan mode) và Duyệt nháp thành plan thật
- Luồng B — Chỉnh sửa plan (edit mode): dò plan id → gate quyền → đề xuất ops → Áp dụng
- Luồng C — Vòng đời phiên, giao thức SSE, endpoint phụ (non-stream, regenerate, history) & hợp đồng frontend
- Khởi động, Dependency Injection & Cấu hình (main, config, dependencies, factory, core/*)
- Lớp API — toàn bộ endpoints, dò plan id, gate quyền, đóng gói SSE & schemas
- Agent Orchestrator — vòng lặp agent, hai SYSTEM PROMPT (plan/edit), primer
- Tool definitions & ToolDispatcher (catalog/web/route + mravel_url + fallback)
- Edit operations (EditOperation, parse/validate, board_summary) & EditService
- Lớp LLM (interface base, OpenAI-compatible client + fallback, stub offline)
- Constraint Extractor (regex + LLM) & Draft Composer (dựng nháp tắt định)
- Approval Service & các tiện ích (geo/haversine, pricing, catalog_location)
- Models phiên & Session Store (InMemory/File/Mongo + factory)
- Upstream Clients REST (catalog-service, plan-service, Tavily web search)
- Cấu hình (.env)
- Bảng tra cứu file
- Thuật ngữ & Gotchas

Kiến trúc tổng quan



Các thành phần & trách nhiệm

Thành phần	Vai trò
Frontend (React)	<code>useAIPlanSession</code> mở SSE, giữ state draft/edit/history; <code>AIPlanPanel</code> / <code>FloatingChatWidget</code> / <code>ChatPage</code> render chat MAI, link Mravel bấm được.
Gateway (:8080)	Reverse-proxy mọi <code>/api/ai-plan/**</code> về ai-plan-service; forward bearer JWT.
ai-plan-service (:8092)	Sở hữu phiên chat, trích constraints, vòng lặp agent LLM, luồng draft→approve, in-chat editor.
catalog-service	Nguồn gợi ý (read-only, public, không auth): hotels / restaurants / places.
plan-service	Nguồn chân lý ghi (authenticated, idempotent, RBAC): tạo plan, board, lists/cards.
Tavily (web search)	Fallback giá / thông tin nhạy cảm theo thời gian mà catalog thiếu.
MongoDB Atlas	Lưu phiên chat (<code>mravel_catalog.ai_plan_sessions</code>), 1 document/phiên.
LLM provider	Bất kỳ endpoint OpenAI-compatible (OpenRouter / 9Router / OpenAI / Groq / Ollama).

Sơ đồ thư mục `app/`

<code>app/</code>	
<code>main.py</code>	FastAPI app, CORS, routers, health
<code>config.py</code>	toàn bộ env settings (LLM, Mongo, web search, CORS...)
<code>api/</code>	
<code>ai_plan.py</code>	TẤT CẢ endpoint + dò plan id + gate quyền (<code>_assert_can_edit</code>) + SSE
<code>schemas.py</code>	request/response DTO (<code>SessionView</code> , <code>SessionSummary...</code>)

dependencies.py	DI wiring (singletons: llm, catalog, plan, store, orchestrator)
agent/	
orchestrator.py	vòng lặp agent + 2 SYSTEM PROMPT (plan & edit) + primer
tools.py	tool definitions + ToolDispatcher (catalog/web/route) + summaries
edits.py	EditOperation model, validate, board_summary, op summaries
llm/	
base.py	interface LLMClient (chat_with_tools, extract_constraints)
openai_client.py	impl OpenAI-compatible (+ fallback models)
stub.py	impl offline regex/tất định (không key)
factory.py	chọn stub vs openai client
services/	
session_store.py	InMemory / FileBacked / MongoSessionStore + factory
constraint_extractor.py	trích constraint lai regex + LLM
draft_composer.py	dựng nháp tất định (đường stub / fallback)
approval_service.py	draft → plan-service (create_plan + cards vào ngày seed)
edit_service.py	op đề xuất → plan-service board mutations
geo.py / pricing.py	haversine routing / heuristic giá
catalog_location.py	map catalog item → shape modal frontend
clients/	
catalog_client.py	catalog-service REST (search hotels/restaurants/places + faceted)
plan_client.py	plan-service REST (đọc board + writes idempotent)
web_search_client.py	Tavily client
core/	
response.py	envelope ApiResponse[T]
errors.py	exception miền + handler toàn cục
security.py	giải mã JWT, CurrentUser
models/session.py	PlanSession, Constraints, PlanDraft, ChatMessage, enums

Bốn hành vi của MAI (workflow)

Cùng một agent phục vụ bốn việc, quyết định theo từng tin nhắn:

- New-plan drafting** (chưa gắn plan) — gom destination/dates/people → tìm catalog → dựng **draft** nhiều ngày dày đặc (**finalize_draft**) → user review → **Duyệt** → tạo plan thật trên plan-service.
- Discovery / Q&A** ("khách sạn nào trên web?", "cho link địa điểm Đà Nẵng") — search catalog **TRƯỚC**, trả về **link mravel_url1 bấm được**. Chỉ khi catalog rỗng mới đề xuất **available_on_mravel** (thành phố lân cận) hoặc, cuối cùng, link **web_search** có gắn nhãn. Không bao giờ bịa venue hay dán link ngoài từ trí nhớ.
- Plan editing** — khi user nhắc tới một plan (/plans/43, "kế hoạch số 43"), phiên **bind** vào plan đó (sau khi kiểm quyền) và agent **đề xuất** thay đổi (**propose_plan_edits**); user bấm **Áp dụng** để ghi.
- Freshness-aware** — mọi fact nhạy cảm theo thời gian (giá vé, giờ mở cửa, thuê xe) **BẮT BUỘC** tra **web_search** với **năm hiện tại** (tiêm vào primer) và cite URL nguồn; không trích từ trí nhớ huấn luyện.

Stub mode vs agent mode. Khi **LLM_API_KEY** rỗng, service vẫn chạy ở chế độ **stub** tất định: trích constraint bằng regex + **DraftComposer** dựng draft thẳng từ catalog, **KHÔNG** có vòng lặp agent (**StubLLMClient.supports_tool_use()** trả **False**). Khi có key, **agent loop** ($\leq \text{MAX_ITERATIONS}=10$) điều khiển mọi thứ; mỗi khi lỗi/cạn vòng lặp, nó fallback về stub composer để người dùng luôn có phản hồi.

Luồng A — Tạo kế hoạch mới + Hỏi đáp/khám phá (plan mode) và Duyệt nháp thành plan thật

Luồng này mô tả vòng đời của một phiên chat **plan mode** (**plan_id == None**): người dùng tạo session, gửi tin nhắn để hỏi đáp/khám phá điểm đến hoặc dựng lịch trình, agent vừa trò chuyện vừa gọi tool tìm kiếm catalog/web rồi **finalize_draft** thành bản nháp (**PlanDraft**), và cuối cùng người dùng bấm **Duyệt** để biến nháp thành plan thật trên **plan-service**.

Phạm vi: chỉ phần **plan_id == None**. Nhánh **edit mode** (**plan_id != None** → **propose_plan_edits** → **apply-edits**) là một luồng riêng, chỉ được nhắc tới ở điểm rẽ nhánh.

Trình tự các bước

1. Tạo session — POST /api/ai-plan/sessions create_session (app/api/ai_plan.py:171).

- Auth qua **get_current_user** → **CurrentUser{ id, raw_token }**.
- store.create(user.id)** tạo **PlanSession** mới (in-memory) với **status=DRAFTING**, **constraints=Constraints()** rỗng, **messages=[]**, **draft=None**, **plan_id=None**, **pending_edits=[]** (**app/models/session.py:168**).
- Nếu body có **plan_id** → rẽ sang edit mode: gọi **_assert_can_edit** (gate phân quyền) rồi gán **session.plan_id**. **Trong Luồng A, plan_id để trống.**
- Nếu có **initial_message** → append một **ChatMessage{role=USER, content}**.
- Trả **ApiResponse[SessionView]** (**_to_view**, **app/api/ai_plan.py:113**) với hình dạng:

```
SessionView{ session_id, status, user_id, constraints, messages[], draft, approved_plan_id, plan_id, pending_edits[] }
```

2. Gửi tin nhắn (streaming) — POST /api/ai-plan/sessions/{id}/messages/stream stream_message (app/api/ai_plan.py:428). Đây là endpoint chính của Luồng A (frontend dùng streaming; /messages non-stream tại **app/api/ai_plan.py:214** là biến thể đồng bộ tương đương).

- Append **ChatMessage{role=USER, content=body.content}** vào **session.messages**, **store.save(session)**.

- **Điểm rẽ nhánh — phát hiện plan_id trong tin nhắn:** `_extract_plan_id(body.content)` ([app/api/ai_plan.py:85](#)) bắt URL `/plans/43` hoặc cụm "kế hoạch 43" (negative-lookahead loại "3 ngày", "2 người"). Nếu phát hiện id mới:
 - Gọi `_assert_can_edit` ([app/api/ai_plan.py:48](#)). Nếu `ForbiddenError` → trả `_one_shot_assistant_stream` (chỉ phát `session + assistant_message` từ chối + `done`), KHÔNG bind. Nếu OK → `session.plan_id = detected` và **chuyển sang edit mode** (`_consume_edit_stream`).
 - Trong Luồng A thuần (không nhắc plan id) → bỏ qua, đi tiếp.
- Gate trạng thái: nếu `session.status != DRAFTING` → `DomainError` ([app/api/ai_plan.py:480](#)).
- **Trích xuất constraint sơ bộ:** `extractor.update(...)` (bước 3) rồi `store.save`.
- Trả `StreamingResponse(_consume_agent_stream(...), media_type="text/event-stream")` với các tham số:

```
constraints      = session.constraints
chat_history     = session.messages[:-1] # loại tin vừa append
latest_user_message = body.content
prior_draft      = session.draft
bearer_token     = user.raw_token
```

3. Trích xuất constraint deterministic + LLM — `ConstraintExtractor.update` [app/services/constraint_extractor.py:27](#).

- **Step 1 (regex stub):** `StubLLMClient.extract_constraints` bắt destination/num_days/travelers/budget từ tiếng Việt/Anh → `_normalize` → `Constraints` (giữ giá trị prior).
- **Step 1b (recover destination):** nếu chưa có destination, `_parse_destination(conversation_text)` quét toàn bộ các turn USER (`_user_conversation_text`, [app/api/ai_plan.py:127](#)) để cứu điểm đến nói ở turn trước.
- **Step 2 (LLM refine):** chỉ chạy khi LLM thật được cấu hình; lỗi/garbage → fallback về baseline regex (không bao giờ làm hỏng request).
- Output: `Constraints{ destination, start_date, end_date, num_days, travelers, budget_total_vnd, interests[], pace }`.
- Lưu ý phân vai: extractor chỉ là *seed* sơ bộ; **agent mới là nguồn chân lý** của constraints (qua `set_constraints`, bước 5).

4. Phát SSE đầu luồng + gọi agent — `_consume_agent_stream` [app/api/ai_plan.py:279](#).

- Phát ngay event `session: { session_id, constraints }`.
- Khởi tạo `final_draft=prior_draft, final_text="", final_constraints=constraints`.
- Gọi `agent.plan_stream(...)` và chuyển tiếp từng event nội bộ thành SSE (bước 6).

5. Vòng lặp agent — `AgentOrchestrator.plan_stream / _run_agent_loop_stream` [app/agent/orchestrator.py:424](#) và [app/agent/orchestrator.py:491](#).

- **Fallback không LLM:** nếu `not llm.supports_tool_use()` → `_stub_path` ([app/agent/orchestrator.py:485](#)): nếu constraints `is_minimally_complete()` thì `DraftComposer.compose` ra nháp, ngược lại trả câu hỏi làm rõ. Phát `draft_ready?` + `assistant_message`, kết thúc.
- **Có LLM:** dựng messages qua `_build_messages` ([app/agent/orchestrator.py:801](#)): một `context primer` (`_format_primer`, [app/agent/orchestrator.py:840](#), chứa "Hôm nay là ...", "Known constraints so far", quy tắc TIMING) + replay tối đa 12 turn gần nhất (lọc bỏ câu fallback canned của assistant để model không lặp lại). `tools = tool_definitions()` ([app/agent/tools.py:31](#)).
- Lặp tối đa `MAX_ITERATIONS=10`. Mỗi vòng:
 - Phát `thinking { iteration }`.
 - `llm.chat_with_tools(_SYSTEM_PROMPT, messages, tools)`.
 - Nếu `stop_reason == "error"` → raise → bị bắt ở `plan_stream` ([app/agent/orchestrator.py:459](#)): phát `error` rồi chạy `_deterministic_fallback` (composer nếu đủ constraints, hoặc câu hỏi làm rõ).
 - Nếu `stop_reason != "tool_use"` (model trả lời thẳng) → đó là **hỏi đáp/khám phá**: lấy `narrative`, phát `assistant_message`, return. (Nếu narrative rỗng → `_deterministic_fallback`.)
 - Nếu `stop_reason == "tool_use"` → duyệt `turn.tool_uses` (bước 5a–5c).

5a. Tool `set_constraints` ([app/agent/orchestrator.py:542](#)) `apply_set_constraints(args, working)` ([app/agent/tools.py:769](#)) merge: field có trong args thẳng, field vắng giữ nguyên prior; **lọc garbage** (destination phải ≥ 2 ký tự alnum, ngày parse được, số > 0). Phát `constraints_updated { constraints }` + `tool_result { name, summary }`. Đây là cách agent ghi nhớ — `working` (bản Constraints sống) được trả ngược về caller.

5b. Tool tìm kiếm read-only ([app/agent/orchestrator.py:588](#) → `ToolDispatcher.run`, [app/agent/tools.py:631](#)) Mỗi tool phát `tool_call { name, arguments }` (trước khi chạy) rồi `tool_result { name, summary }` (sau khi chạy). Hình dạng kết quả luôn `{ items: [...], count }` (hoặc đặc thù):

- `search_hotels / search_restaurants / search_places` → `CatalogClient` POST `/api/catalog/{...}/search` ([app/clients/catalog_client.py](#)). Mỗi item trimmed gồm id, slug, name, mrvael_url, address, latitude, longitude, price..., avg_rating, cover_image_url (`_summarise_*`). Cap 8 item (`_MAX_ITEMS_PER_TOOL`).
 - **Fallback địa điểm rỗng:** `_empty_location_fallback` ([app/agent/tools.py:605](#)) trả `{ items:[], no_results_for_location, available_on_mrvael:[...], note }` để agent đề xuất thành phố lân cận thay vì link ngoài.
 - `search_places` ưu tiên `search_places_faceted(parentSlug)` (accent-proof) rồi mới `search_places(q)` (giữ dấu).
- `route_legs` → [app/services/geo.py:128](#): tính haversine + chọn mode (đi bộ/xe máy/ô tô/intercity) + thời gian; trả `{ legs: [{from,to,distance_km,distance_text,minutes,transport_mode,route_hint}], total_km, total_minutes }`.
- `web_search` → `WebSearchClient.search` ([app/clients/web_search_client.py](#), Tavily). **Fallback:** nếu không có API key → `{ enabled:false, results:[], note }` (agent được dẫn nói rõ "ước tính"). Never raises.
- `view_my_plans / search_plans` → `PlanClient` (cần `bearer_token`); thiếu token → `{ items:[], note }`.

5c. Tool terminal `finalize_draft` ([app/agent/orchestrator.py:558](#)) `parse_finalize_draft(args, working)` ([app/agent/tools.py:843](#)) chuyển payload có cấu trúc thành `PlanDraft`:

- destination lấy từ args hoặc constraints (bắt buộc, nếu thiếu → `DraftValidationError`).
- ngày: nếu thiếu → `constraints.resolved_date_range()` neo về **hôm nay + num_days-1** ([app/models/session.py:70](#)). `end_date < start_date` → lỗi.
- mỗi activity → `DraftActivity{ title, description, day_index, start_time, end_time, duration_minutes, activity_type, estimated_cost_vnd, unit_price_vnd, quantity, reason, recommendation: RecommendationRef?, location_name, address, note,`

- Khi finalize thành công: đồng bộ ngược **working** (destination/start/end/travelers) để regenerate không bị chặn "incomplete"; phát **constraints_updated**, **tool_result** ("draft saved"), rồi **draft_ready** { **draft**: **PlanDraft** } + **assistant_message** (nếu model im lặng, gọi LLM 1 lượt nữa để sinh câu tóm tắt mời Duyệt), return.

- `session_to_save = store.get(session_id, user_id)` (đọc lại bản mới nhất), gán `draft` (nếu có), `constraints`, `append ChatMessage{ASSISTANT, final_text}`, `store.save`.
- Phát `done { draft: PlanDraft|null, assistant_message, constraints }` — đóng stream.

Duyệt nhập thành plan thật — **POST** .../approve approve_session (app/api/ai_plan.py:618).

- Các SSE event phát ra và thời điểm (Luồng A)

Event	Payload	Thời điểm phát
session	{ session_id, constraints }	Ngay khi vào <code>_consume_agent_stream</code> (đầu stream)
thinking	{ iteration }	Đầu mỗi vòng lặp agent (trước khi gọi LLM)
tool_call	{ name, arguments }	Trước khi chạy mỗi tool (kể cả <code>set_constraints/finalize_draft</code>)
tool_result	{ name, summary }	Sau khi chạy mỗi tool xong
constraints_updated	{ constraints }	Sau <code>set_constraints</code> và sau <code>finalize_draft</code> (đồng bộ ngược)
draft_ready	{ draft }	Khi <code>finalize_draft</code> parse thành công (hoặc composer fallback)
assistant_message	{ text }	Khi model trả lời thẳng, sau draft, hoặc trong fallback
error	{ message }	LLM lỗi, exception trong stream, hoặc trong <code>plan_stream</code> catch
done	{ draft, assistant_message, constraints }	Cuối cùng, sau khi persist session — đóng stream
edit_proposal	{ operations }	Chỉ ở edit mode (<code>_consume_edit_stream</code>) — không xuất hiện trong Luồng A thuần

Sơ đồ ASCII (sequence)

Frontend	ai_plan.py (API)	ConstraintExtractor	AgentOrchestrator	ToolDispatcher / clients
ApprovalService ->	plan-service			
	-- POST /sessions ---->	create PlanSession		
	<-- SessionView -----	(status=DRAFTING)		
	-- POST /messages/stream (SSE) -->			

```

|
|
| append USER msg
|
|
| _extract_plan_id? --[có id + EDITOR]--> edit mode (propose_plan_edits) ... (luồng khác)
|
| extractor.update() --->| regex+recover+LLM
|
|<-- Constraints -----|
|
|<== event: session ====|
|
| -- plan_stream(...) ----->| build primer+messages
|
|<== thinking =====<-----| loop (<=10):
|
|<== tool_call =====<-----| set_constraints -----> apply_set_constraints
|
|<== constraints_updated<-----| search_* / route / web -> catalog/geo/Tavily
|
|<== tool_result =====<-----|<-----
{items,count}/{legs}/{results}|
|<== draft_ready =====<-----| finalize_draft -> PlanDraft
|
|<== assistant_message =<-----|
|
| persist draft+constraints+ASSISTANT msg
|
|<== done =====|
|
| -- POST /approve ----->| status==APPROVED? -> DomainError
|
| draft is None? -> reject
|
| -- approve(draft) -----> create_plan -> POST
/api/plans
|
| get_board (seeded DAY
lists)
|
| rename_list +
create_cardxN
|
| (Idempotency-Key uuid5)
|<-- ApprovalResult ----| status=APPROVED, approved_plan_id=plan_id |<-- {plan_id, operations} --
--|

```

Điểm rẽ nhánh, gate phân quyền, fallback, idempotency

- **Rẽ nhánh plan vs edit mode:** quyết định bởi `session.plan_id`. Plan mode (`None`) → dựng `PlanDraft` + Duyệt. Edit mode (`!= None`, set lúc create hoặc khi `_extract_plan_id` bắt được id trong tin nhắn) → `edit_stream` → `edit_proposal` → `apply-edits. /messages` non-stream chặn thẳng edit mode ([app/api/ai_plan.py:223](#)).
- **Rẽ nhánh trong agent:** `stop_reason` quyết định hỏi đáp (`!= tool_use` → `assistant_message`) vs tiếp tục gọi tool (`tool_use`) vs kết thúc bằng `finalize_draft` (terminal → `draft_ready`).
- **Gate phân quyền** (`_EDIT_ROLES = {OWNER, EDITOR}`):
 - `_assert_can_edit` ([app/api/ai_plan.py:48](#)) tại: `create_session` (edit mode), khi bind plan id qua tin nhắn, và lại tại `apply-edits` (role có thể bị thu hồi giữa chừng).
 - `edit_stream` còn một gate defense-in-depth đọc `board.myRole` ([app/agent/orchestrator.py:649](#)).
 - `plan-service` là enforcer cuối cùng (per-op). Gate ở ai-plan-service chỉ để fail-fast với thông điệp thân thiện. VIEWER bị từ chối dưới dạng `assistant_message` (không phải hard 403) khi bind qua tin nhắn.
- **Fallback nhiều tầng:**
 - Không có LLM (`supports_tool_use()==False`) → `DraftComposer.compose` (deterministic, geo-aware) nếu đủ constraints, ngược lại câu hỏi làm rõ (`_clarification_text`).
 - LLM lỗi / model im lặng / hết 10 vòng → `_deterministic_fallback` (composer hoặc câu hỏi cụ thể về field còn thiếu, tránh hỏi lại fact đã biết).
 - `web_search` không cấu hình/lỗi → trả `enabled:false`, agent dùng ước tính có nhãn.
 - Catalog rỗng theo location → `available_on_mravel` (thành phố lân cận) thay vì dead-end.
 - Catalog/web client lỗi 5xx/unreachable → `UpstreamError`, được nuốt thành kết quả rỗng trong dispatcher (không abort cả turn).
- **Idempotency / chống trùng:**
 - Approve: chặn lặp ở mức session (`status == APPROVED`); mỗi card mang `Idempotency-Key` UUID5 deterministic (`plan:{plan_id}:card:{day}:{position}`) gửi qua endpoint `/board/cmd/.../cards` (path cũ bỏ qua key) → retry an toàn.
 - Regenerate: chặn double-click bằng cách phát hiện turn [`regenerate`] chưa trả lời ([app/api/ai_plan.py:531](#)).
 - Persist trong stream luôn `store.get` lại bản mới nhất trước khi append, tránh ghi đè turn xen kẽ.

Luồng B — Chinh sửa plan (edit mode): dò plan id → gate quyền → đề xuất ops → Áp dụng

Luồng B cho phép người dùng chat với MAI để chỉnh sửa một plan đã tồn tại. Khác với Luồng A (tạo bản nháp mới), edit mode **chỉ chạy streaming** (`/messages/stream`) và **không tự ghi** thay đổi — agent chỉ ĐỀ XUẤT operations; chỉ khi người dùng bấm **Áp dụng** (`/apply-edits`) các thay đổi mới được ghi vào

plan-service.

Có hai cách session vào edit mode:

- **Tạo session ở edit mode:** client gọi `POST /sessions` kèm `plan_id` → gate quyền ngay tại `create_session` ([app/api/ai_plan.py:178](#)).
- **Global MAI tự dò plan id từ tin nhắn:** session ban đầu không gắn plan, người dùng dán `/plans/43` hoặc gõ "kế hoạch số 43" → `stream_message` dò id, gate quyền, rồi bind ([app/api/ai_plan.py:450](#)). Mục này tập trung vào con đường thứ hai (đầy đủ nhất), nhưng bước áp dụng dùng chung.

Các bước

1. **Client gửi tin nhắn (HTTP endpoint)** — `POST /api/ai-plan/sessions/{session_id}/messages/stream`, handler `stream_message` ([app/api/ai_plan.py:428](#)).

- Input: path `session_id`, body `SendMessageRequest { content: str }`, header `Authorization: Bearer <token>` (giải mã thành `CurrentUser { id, raw_token }`).
- Lấy session từ `store.get(session_id, user.id)` (ràng buộc theo `user.id` → user khác không đọc được session), append `ChatMessage(role=USER, content=body.content)` rồi `store.save` ([app/api/ai_plan.py:440](#)).

2. **Dò plan id trong nội dung tin nhắn** — `_extract_plan_id(body.content)` ([app/api/ai_plan.py:85](#), gọi tại [app/api/ai_plan.py:450](#)).

- Regex `_PLAN_REF_PATTERNS` ([app/api/ai_plan.py:76](#)): bắt URL `/plans/<id>` và cụm "plan/kế hoạch [số]#[id] <số>".
- **Điểm rẽ nhánh / chống false-positive:** negative lookahead `_PLAN_UNIT` ([app/api/ai_plan.py:75](#)) loại các đơn vị đếm ("kế hoạch 3 ngày", "plan 2 người") để không nhầm thành plan id.
- Output: `detected_plan_id: int | None`.
- **Điều kiện bind:** chỉ xử lý khi `detected_plan_id is not None and detected_plan_id != session.plan_id` ([app/api/ai_plan.py:451](#)).

3. **Gate phân quyền (sớm, thân thiện)** — `_assert_can_edit(plan_client, user.raw_token, detected_plan_id)` ([app/api/ai_plan.py:48](#), gọi tại [app/api/ai_plan.py:453](#)).

- Gọi `plan_client.get_board(bearer_token, plan_id)` → `GET /api/plans/{plan_id}/board` ([app/clients/plan_client.py:141](#)). Board trả về (`BoardResponse`) mang field `myRole`.
- **Gate:** `role = board["myRole"].upper()` phải thuộc `_EDIT_ROLES = {"OWNER", "EDITOR"}` ([app/api/ai_plan.py:45](#)).
- **Fallback từ chối (KHÔNG hard 403):** nếu `_assert_can_edit` ném `ForbiddenError`, `stream_message` KHÔNG raise 403 mà trả về stream một-lần `_one_shot_assistant_stream` ([app/api/ai_plan.py:454](#), [app/api/ai_plan.py:96](#)) — phát `session` → `assistant_message` (lời từ chối) → `done`, đồng thời lưu `assistant message` vào session. (Lưu ý: nhánh `create_session` ở [app/api/ai_plan.py:181](#) thì raise 403 thật vì chưa có chat để từ chối mềm.)
- Nếu OK: `session.plan_id = detected_plan_id; store.save(session)` ([app/api/ai_plan.py:460](#)) → session chuyển sang edit mode.

4. **Phân nhánh edit mode vs draft mode** — `if session.plan_id is not None` ([app/api/ai_plan.py:464](#)).

- Edit mode: trả `StreamingResponse(_consume_edit_stream(...))` ([app/api/ai_plan.py:465](#)), `media_type="text/event-stream"`, headers `_SSE_HEADERS`.
- Dữ liệu truyền vào `_consume_edit_stream`: `agent, store, session_id, user_id=user.id, plan_id=session.plan_id, chat_history=list(session.messages[:-1])` (lịch sử KHÔNG gồm tin vừa append), `latest_user_message=body.content`, `bearer_token=user.raw_token`.

5. **SSE consumer cho edit mode** — `_consume_edit_stream(...)` ([app/api/ai_plan.py:368](#)).

- Phát ngay `session { session_id, plan_id }` ([app/api/ai_plan.py:384](#)).
- Khởi tạo `final_text="", operations=[]`; lặp `async for ev in agent.edit_stream(plan_id, chat_history, latest_user_message, bearer_token)` ([app/api/ai_plan.py:387](#)) và map mỗi event nội bộ → SSE event (xem bảng dưới).
- **Persist sau khi stream kết thúc** ([app/api/ai_plan.py:412](#)): re-read session, `session.pending_edits = operations` (đề xuất mới nhất, chờ "Áp dụng"), append `ChatMessage(ASSISTANT, final_text)`, `store.save`.
- Phát `done { operations, assistant_message, plan_id }` ([app/api/ai_plan.py:418](#)).
- **Fallback lỗi:** mọi exception trong vòng lặp được bắt và phát `error { message }` ([app/api/ai_plan.py:408](#)), nhưng vẫn chạy tiếp phần `persist + done`.

6. **Agent loop edit mode** — `AgentOrchestrator.edit_stream(...)` ([app/agent/orchestrator.py:623](#)).

- **6a. Đọc board lần nữa:** `board = await self._plan_client.get_board(bearer_token, plan_id)` ([app/agent/orchestrator.py:636](#)). Nếu lỗi → phát `error` + một `assistant_message` xin lỗi rồi `return` ([app/agent/orchestrator.py:637](#)).
- **6b. Gate quyền lần 2 (defense-in-depth):** `role = board["myRole"].upper()`; nếu không thuộc ("OWNER", "EDITOR") → chỉ phát một `assistant_message` từ chối (phân biệt VIEWER vs role khác) rồi `return`, KHÔNG build proposal ([app/agent/orchestrator.py:649](#)).
- **6c. Fallback không có LLM:** nếu `not llm.supports_tool_use()` → phát `assistant_message` chứa `board_summary(board)` + lời nhắc cấu hình `LLM_API_KEY`, rồi `return` ([app/agent/orchestrator.py:664](#)).
- **6d. Vòng lặp tool-use** (tối đa `MAX_ITERATIONS = 10`): `messages = _build_edit_messages(summary, chat_history, latest_user_message)` ([app/agent/orchestrator.py:757](#)) — primer là `board_summary` (mô tả board kèm `list_id/card_id`, ngày hôm nay) + tối đa 12 lượt chat gần nhất (lọc các câu fallback canned). `Tools = edit_tool_definitions()`.
- Mỗi iteration: phát `thinking { iteration }`, gọi `llm.chat_with_tools(_EDIT_SYSTEM_PROMPT, messages, tools)`.
 - `stop_reason == "error"` → phát `error` + `assistant_message` (kèm `board_summary`) rồi `return` ([app/agent/orchestrator.py:685](#)).
 - `stop_reason != "tool_use"` (model chat/hỏi lại) → phát `assistant_message` (narrative hoặc câu mặc định) rồi `return` ([app/agent/orchestrator.py:698](#)). Đây là nhánh "không có đề xuất" — `operations` rỗng.
 - Với mỗi `tool_use`: phát `tool_call { name, arguments }`.
 - `get_current_plan` → đọc lại board, trả `board_summary` vào tool result, phát `tool_result` "đã đọc plan" ([app/agent/orchestrator.py:711](#)).
 - `propose_plan_edits` (terminal) → `parse_operations(arguments["operations"], board=board)` ([app/agent/orchestrator.py:721](#)). Nếu `EditValidationError` → tool result là error, phát `tool_result` "error: ...", `proposed=None`, lặp tiếp. Nếu OK → phát `tool_result` "đề xuất N thay đổi".

- Tool khác (search_hotels/restaurants/places, route_legs, web_search) → `dispatcher.run(...)`, phát `tool_result` với summary (`app/agent/orchestrator.py:737`).
 - **6e. Kết thúc khi có proposal:** nếu `proposed` is not None → phát `edit_proposal` { `operations`: [`op.model_dump()` for `op` in `proposed`] } rồi `assistant_message` (narrative hoặc `_default_edit_narrative`), return (`app/agent/orchestrator.py:742`).
 - **6f. Fallback cận iteration:** hết 10 vòng không proposal → phát một `assistant_message` xin mô tả rõ hơn (`app/agent/orchestrator.py:751`).
- 7. Validate + làm sạch đề xuất** — `parse_operations(raw, board)` (`app/agent/edits.py:75`).
- Mỗi entry: `op` phải thuộc `OP_TYPES` (8 loại), validate thành `EditOperation`, qua `_has_required_targets` (đu id/field bắt buộc theo từng op — `app/agent/edits.py:138`), và **khi có board** qua `_ids_exist` (loại op tham chiếu `list_id/card_id` LLM bịa ra — `app/agent/edits.py:125`).
 - Entry không hợp lệ bị **drop** (không abort cả proposal); nếu rỗng → `EditValidationError("no valid operations")`.
 - Gán `summary` id-free cho UI nếu thiếu (`operation_summary`, `app/agent/edits.py:158`).
 - **Hình dạng dữ liệu:** `List[EditOperation]` với `op` ∈ {`update_card`, `create_card`, `delete_card`, `move_card`, `rename_list`, `add_day`, `delete_list`, `update_plan`}, kèm targets (`list_id`, `card_id`, `target_list_id`, `target_position`), các field card (`text/start_time/.../recommendation`), field list/plan (`title/start_date/end_date/budget_total_vnd`), và `summary`. Khi serialize qua SSE là list dict (`op.model_dump()`).
- 8. Lưu đề xuất vào session** — `session.pending_edits = operations` trong `_consume_edit_stream` (`app/api/ai_plan.py:414`). Đây là nguồn dữ liệu duy nhất cho bước Áp dụng. Frontend nhận `edit_proposal/done` → render approval cards + nút **Áp dụng**.
- 9. Áp dụng (write thật)** — `POST /api/ai-plan/sessions/{session_id}/apply-edits`, handler `apply_edits` (`app/api/ai_plan.py:655`). Đây là **nơi DUY NHẤT** ghi edit.
- Tiền điều kiện: `session.plan_id` phải có (`app/api/ai_plan.py:669`) và `session.pending_edits` không rỗng (`app/api/ai_plan.py:671`).
 - **Snapshot idempotency-cấp-session:** `applied_edits = list(session.pending_edits)` (`app/api/ai_plan.py:675`) — chụp đúng tập đang áp dụng để sau này chỉ clear ĐÚNG tập này.
 - **Gate quyền lần 3 (tại thời điểm write):** `await _assert_can_edit(plan_client, user.raw_token, session.plan_id)` (`app/api/ai_plan.py:680`) — role có thể đã bị thu hồi sau khi mở session; fail nhanh thay vì 403 từng-op.
 - `operations = parse_operations(session.pending_edits)` (`app/api/ai_plan.py:683`; lần này **không truyền board** → chỉ validate cấu trúc/targets). Lỗi → `DomainError("Đề xuất chỉnh sửa không hợp lệ: ...")`.
 - Gọi `edit_service.apply(session.plan_id, operations, user.raw_token)` (`app/api/ai_plan.py:688`). `UpstreamError` → bọc lại "Không áp dụng được thay đổi: ..." (`app/api/ai_plan.py:689`).
- 10. Thực thi từng op chống nhau** — `EditService.apply(...)` (`app/services/edit_service.py:149`).
- **Batch idempotency seed:** `batch_seed = _batch_seed(plan_id, operations)` (`app/services/edit_service.py:157`, `app/services/edit_service.py:29`) = hash nội dung của ĐÚNG tập ops trên plan này. Cùng proposal (double-click) → cùng seed → dedup; proposal khác → seed khác → áp dụng.
 - Lập từng `op`, gọi `_apply_one(plan_id, op, bearer, idx, batch_seed)` (`app/services/edit_service.py:168`). **Per-op error collection:** một op lỗi không abort batch — bắt exception, push {`op`, `ok`: False, `error`} (`app/services/edit_service.py:163`).
 - Mỗi `op` → một endpoint `PlanClient`: `update_card/create_card/delete_card/move_card/rename_list/delete_list/update_plan` (`title/dates/budget` tách lệnh con). **Idempotency-Key** (`_idem(f"{seed}:{idx}:...")`) chỉ gán cho `create_card` / `move_card` / `add_day` (các write có nguy cơ nhân đôi khi retry — `app/services/edit_service.py:177`).
 - **Validate trước khi ghi:** `move_card` raise nếu response thiếu `entityId` (chống báo no-op là success — `app/services/edit_service.py:189`); `update_plan` reject range đảo ngược `start>end` (`app/services/edit_service.py:218`).
 - Output: { `applied`: int, `total`: int, `results`: [{`op`, `ok`, `summary`, `detail|error`}] }.
- 11. Tổng hợp + dọn pending** — quay lại `apply_edits` (`app/api/ai_plan.py:693`).
- Dựng summary "Đã áp dụng {`applied`}/{`total`} thay đổi..." (+ số op lỗi nếu có), append `ChatMessage(ASSISTANT, summary)`.
 - **Re-read session** (`session = store.get(...)` — `app/api/ai_plan.py:700`) vì một edit-stream đồng thời có thể đã append proposal mới khi đang await apply.
 - **Idempotency / chống mất proposal mới:** chỉ `session.pending_edits = []` nếu `session.pending_edits == applied_edits` (`app/api/ai_plan.py:705`) — nếu proposal mới đã đến giữa chừng thì giữ nguyên, không xóa nhầm.
 - Response: `ApplyEditsResult` { `applied`, `total`, `results`, `assistant_message` } (`app/api/ai_plan.py:709`).

Các SSE event và thời điểm phát (edit mode)

Event	Thời điểm phát	Nguồn
session	Ngay khi mở stream, mang { <code>session_id</code> , <code>plan_id</code> }	<code>_consume_edit_stream</code> <code>app/api/ai_plan.py:384</code>
thinking	Đầu mỗi iteration của agent loop, { <code>iteration</code> }	<code>app/agent/orchestrator.py:680</code>
tool_call	Mỗi khi agent gọi một tool, { <code>name</code> , <code>arguments</code> }	<code>app/agent/orchestrator.py:709</code>
tool_result	Sau mỗi tool (gồm <code>get_current_plan</code> , <code>propose_plan_edits</code> , <code>catalog/web/route</code>), { <code>name</code> , <code>summary</code> }	<code>app/agent/orchestrator.py:718</code>
assistant_message	Khi model chat/hỏi/từ chối, hoặc kèm theo proposal, { <code>text</code> }	<code>app/agent/orchestrator.py:701</code>
edit_proposal	Khi <code>propose_plan_edits</code> cho ra ops hợp lệ (terminal), { <code>operations</code> }	<code>app/agent/orchestrator.py:744</code>
error	Khi LLM lỗi hoặc exception trong consumer, { <code>message</code> }	<code>app/agent/orchestrator.py:687</code> , <code>app/api/ai_plan.py:409</code>
done	Cuối stream (luôn phát), { <code>operations</code> , <code>assistant_message</code> , <code>plan_id</code> }	<code>_consume_edit_stream</code> <code>app/api/ai_plan.py:418</code>

Lưu ý: `draft_ready` và `constraints_updated` thuộc Luồng A (draft mode, `_consume_agent_stream`) và **không** xuất hiện trong edit mode. Bước Áp dụng (`/apply-edits`) là **JSON thường, không phải SSE** — trả `ApiResponse[ApplyEditsResult]`.

Sơ đồ luồng (edit)



Tóm tắt điểm rẽ nhánh, gate, fallback, idempotency (edit)

- **3 lớp gate quyền** (defense-in-depth, plan-service là enforcer cuối cùng per-op):
 1. Lúc dò/bind plan id trong `stream_message` (hoặc lúc `create_session`) — `_assert_can_edit` ([app/api/ai_plan.py:453](#) / [app/api/ai_plan.py:181](#)).
 2. Trong `edit_stream` trước khi build proposal — đọc lại `myRole` ([app/agent/orchestrator.py:649](#)).
 3. Lúc Áp dụng, ngay trước write — `_assert_can_edit` lần nữa ([app/api/ai_plan.py:680](#)).
- **Gate trả khác nhau tùy endpoint:** `create_session` raise 403 cứng; `stream_message` từ chối mềm bằng `_one_shot_assistant_stream`; `edit_stream` từ chối bằng một `assistant_message`.
- **Điểm rẽ nhánh chính:** (a) `_extract_plan_id` có/không bắt được id + lookahead chống đếm ngày/người; (b) edit mode vs draft mode tại `session.plan_id is not None` ([app/api/ai_plan.py:464](#)); (c) trong agent loop: `tool_use` vs chat-thường, `propose_plan_edits` thành công vs `EditValidationError`.
- **Fallback:** lỗi đọc board → xin lỗi; không có LLM → trả `board_summary`; LLM error → `error` + board hiện tại; cạn 10 iteration không proposal → xin mô tả rõ hơn; op lỗi khi apply → thu thập per-op, không abort batch.
- **Idempotency:**
 - Cấp write: `Idempotency-Key` từ `_batch_seed` (hash nội dung tập ops trên plan) cho `create_card` / `move_card` / `add_day` → double-click cùng proposal được dedup ở plan-service; proposal khác → key khác → vẫn áp dụng ([app/services/edit_service.py:29](#), [app/services/edit_service.py:177](#); `headers_extra= {"Idempotency-Key": ...}` được build ở [app/clients/plan_client.py:172](#) rồi merge vào request trong `_raw_request` tại [app/clients/plan_client.py:315](#) — lưu ý `_request:30` chỉ phục vụ `create_plan`).
 - Cấp session: snapshot `applied_edits` + clear `pending_edits` chỉ khi vẫn khớp snapshot → không xóa nhầm proposal mới đến giữa chừng ([app/api/ai_plan.py:675](#), [app/api/ai_plan.py:700](#)).
- **Hallucinated id:** `parse_operations(..., board=board)` trong agent loop loại op tham chiếu `list_id/card_id` không có trên board ([app/agent/edits.py:125](#)) — đảm bảo proposal không bao giờ chứa id bừa khi tới bước apply.

Tham chiếu file chính: [app/api/ai_plan.py](#), [app/agent/orchestrator.py](#) (`edit_stream`), [app/agent/edits.py](#) (`EditOperation`, `parse_operations`), [app/services/edit_service.py](#) (`EditService.apply`), [app/clients/plan_client.py](#) (các endpoint board/card/list/plan).

Luồng C — Vòng đời phiên, giao thức SSE, endpoint phụ (non-stream, regenerate, history) & hợp đồng frontend

Luồng này mô tả end-to-end vòng đời của một `PlanSession`: từ lúc tạo phiên, gửi tin nhắn (cả streaming SSE lẫn non-stream), regenerate, đến danh sách lịch sử (history) — kèm hợp đồng dữ liệu (data contract) mà frontend dựa vào để render. Tất cả endpoint nằm dưới prefix `/api/ai-plan` ([app/api/ai_plan.py:39](#)).

1. Mô hình phiên — đơn vị dữ liệu nền tảng

`PlanSession` ([app/models/session.py:168](#)) là object trạng thái duy nhất trôi qua mọi bước. Hình dạng (các field):

- `session_id`: `str` (UUID, sinh tự động) — khóa định danh.
- `user_id`: `int` — chủ phiên; mọi `store.get()` đối chiếu field này để gate phân quyền.
- `status`: `SessionStatus` — `DRAFTING` (mặc định) hoặc `APPROVED` ([app/models/session.py:163](#)).

- `constraints`: `Constraints` ([app/models/session.py:29](#)) — `destination`, `start_date`, `end_date`, `num_days`, `travelers=2`, `budget_total_vnd`, `interests`, `pace`.
- `messages`: `List[ChatMessage]` — mỗi phần tử `{role: USER|ASSISTANT, content: str, created_at: datetime}` ([app/models/session.py:157](#)).
- `draft`: `Optional[PlanDraft]` ([app/models/session.py:139](#)) — bản nháp đang dựng (chế độ tạo mới).
- `approved_plan_id`: `Optional[int]` — id plan đã duyệt (sau approve).
- `plan_id`: `Optional[int]` — nếu set ⇒ **chế độ chỉnh sửa** (edit mode); chat sửa plan có sẵn thay vì dựng draft.
- `pending_edits`: `List[Dict]` — đề xuất chỉnh sửa mới nhất của agent, chờ user bấm "Áp dụng".
- `created_at`, `updated_at`: `datetime`.

`SessionStatus` chỉ có 2 trạng thái và là máy trạng thái một chiều: `DRAFTING` → `APPROVED` (không quay lại). `plan_id` quyết định nhánh tạo-mới hay chỉnh-sửa.

2. Persistence — `session_store.py`

`store.save(session)` luôn cập nhật `updated_at = now(UTC)` ([app/services/session_store.py:50](#)) rồi lưu. Có 3 backend cài cùng interface (`create`, `get`, `save`, `list_by_user`):

- `MongoSessionStore` (mặc định, [app/services/session_store.py:141](#)) — mỗi phiên là 1 document `_id = session_id`; `save = 1` upsert; index (`user_id`, `updated_at` DESC) cho history.
- `FileBackedSessionStore` ([app/services/session_store.py:64](#)) — JSON ghi atomic qua tmp file + `os.replace`; nạp 1 lần lúc start.
- `InMemorySessionStore` ([app/services/session_store.py:29](#)) — dict + `RLock`.

Factory `get_session_store()` ([app/services/session_store.py:235](#)) chọn theo `session_store_backend`; nếu Mongo không kết nối được thì **fallback** về file/memory ([app/services/session_store.py:267](#)). Gate phân quyền cốt lõi nằm ở `get()`: ném `DomainError` nếu không tìm thấy phiên, hoặc nếu `session.user_id != user_id` ([app/services/session_store.py:43](#)) — không ai đọc được phiên của người khác.

3. Tạo phiên — `POST /sessions` → `create_session` ([app/api/ai_plan.py:171](#))

- Body `CreateSessionRequest` ([app/api/schemas.py:9](#)): `{ initial_message?: str, plan_id?: int }`.
- `store.create(user.id)` sinh phiên mới `DRAFTING`.
- **Rẽ nhánh edit mode**: nếu `body.plan_id` được set ⇒ gọi `_assert_can_edit(...)` TRƯỚC khi mở phiên ([app/api/ai_plan.py:181](#)); chỉ gán `session.plan_id` nếu vai trò là OWNER/EDITOR, nếu không ném `ForbiddenError` (403).
- `initial_message` (nếu có) được append làm `ChatMessage(role=USER)`.
- Trả `ApiResponse[SessionView]` (status 201). `SessionView` ([app/api/schemas.py:17](#)) = toàn bộ snapshot phiên: `session_id`, `status`, `user_id`, `constraints`, `messages`, `draft`, `approved_plan_id`, `plan_id`, `pending_edits`.

4. Gate phân quyền chỉnh sửa — `_assert_can_edit` ([app/api/ai_plan.py:48](#))

Đọc board plan qua `plan_client.get_board` (mang theo `myRole`). Quy tắc (`_EDIT_ROLES = {OWNER, EDITOR}`, [app/api/ai_plan.py:45](#)):

- `UpstreamError` (plan-service trả 401/403) ⇒ `ForbiddenError` "không có quyền truy cập".
- `myRole == VIEWER` ⇒ `ForbiddenError` với message gợi ý xin quyền EDITOR.
- Vai trò khác ⇒ `ForbiddenError` chung.
- OWNER/EDITOR ⇒ trả về `board` để tải dùng (tránh round-trip thứ 2).

Đây chỉ là **early gate thân thiện**; plan-service vẫn là enforcer cuối (kiểm tra EDITOR trên mỗi mutation).

5. Gửi tin nhắn non-stream — `POST /sessions/{id}/messages` → `send_message` ([app/api/ai_plan.py:214](#))

Chỉ dùng cho **chế độ tạo mới**. Trình tự:

1. `store.get(id, user.id)` (gate user).
2. **Rẽ nhánh**: nếu `session.plan_id` is not None ⇒ ném `DomainError` ("chế độ chỉnh sửa dùng streaming") — edit mode KHÔNG có endpoint non-stream ([app/api/ai_plan.py:223](#)).
3. Nếu `status != DRAFTING` ⇒ `DomainError` ([app/api/ai_plan.py:226](#)).
4. Append `ChatMessage(USER, body.content)`.
5. `extractor.update(...)` cập nhật `constraints`, truyền `conversation_text` = toàn bộ lượt USER (`_user_conversation_text`, [app/api/ai_plan.py:127](#)) để khôi phục dữ kiện (đặc biệt `destination`) đã nêu ở lượt trước.
6. `agent.plan(...)` (await trọn, không stream) trả (`draft`, `narrative`, `updated_constraints`). Agent là nguồn chân lý của `constraints` — có thể tinh chỉnh giữa lượt qua `set_constraints`.
7. Ghi `constraints`, `draft` (nếu có), append `ChatMessage(ASSISTANT, narrative)`, `store.save`.
8. Trả `ApiResponse[SendMessageResponse]` ([app/api/schemas.py:52](#)): `{ session_id, constraints, draft?, assistant_message, needs_more_info: bool, missing_fields: [str] }`. `missing_fields` tính bởi `_missing_constraint_fields` ([app/api/ai_plan.py:154](#)): thiếu `destination`, hoặc thiếu cả (`num_days`) lẫn (`range start_date+end_date`). `needs_more_info = bool(missing) and draft is None`.

6. Giao thức SSE — `_sse_event` & headers

Mỗi sự kiện là một frame SSE: `event: <tên>\ndata: <json>\n\n` (`_sse_event`, [app/api/ai_plan.py:275](#)), JSON dùng `ensure_ascii=False` (giữ tiếng Việt) và `default=str` (date/datetime). Headers cố định `_SSE_HEADERS` ([app/api/ai_plan.py:268](#)): `Cache-Control: no-cache`, `X-Accel-Buffering: no` (tắt buffer nginx), `Connection: keep-alive`, `media_type = "text/event-stream"`.

7. Streaming tin nhắn — `POST /sessions/{id}/messages/stream` → `stream_message` ([app/api/ai_plan.py:428](#))

Đây là điểm rẽ nhánh trung tâm. Trình tự:

1. `store.get`, append `ChatMessage(USER, body.content)`, `store.save` ngay (lượt user được lưu trước khi stream).

2. **Phát hiện plan id trong tin nhắn:** `_extract_plan_id` (`app/api/ai_plan.py:85`) bắt `/plans/43`, "kế hoạch số 43", "plan #43"... với negative lookahead loại "kế hoạch 3 ngày"/"plan 2 người" (`_PLAN_UNIT`, `app/api/ai_plan.py:75`). Nếu phát hiện id khác `session.plan_id`:
 - Gọi `_assert_can_edit`. Nếu `ForbiddenError` ⇒ **fallback mềm**: trả `_one_shot_assistant_stream` (`app/api/ai_plan.py:96`) — phát đúng 3 event `session` → `assistant_message` (message từ chối) → `done`, KHÔNG ném 403 cứng (chat không bị treo).
 - Nếu OK ⇒ gán `session.plan_id`, `store.save` ⇒ chuyển phiên sang edit mode.
3. **Rẽ nhánh chính:**
 - `plan_id is not None` ⇒ `_consume_edit_stream` (luồng chỉnh sửa, Luồng B).
 - ngược lại: kiểm `status == DRAFTING` (nếu không ⇒ `DomainError`), `extractor.update constraints`, `store.save`, rồi `_consume_agent_stream` (luồng dựng draft, Luồng A).
4. Trả `StreamingResponse(..., media_type="text/event-stream", headers=_SSE_HEADERS)`.

8. Regenerate — streaming & non-stream

`POST /sessions/{id}/regenerate/stream` → `stream_regenerate` (`app/api/ai_plan.py:504`). Các **gate** (đều ném `DomainError`):

- `status == APPROVED` ⇒ "Cannot regenerate after approval" (`app/api/ai_plan.py:515`).
- `constraints` chưa minimally-complete (`is_minimally_complete`, `app/models/session.py:57`) ⇒ từ chối (`app/api/ai_plan.py:517`).
- `draft is None` ⇒ "Chưa có bản nháp để tạo lại" — regenerate là "thử phương án khác", không phải "khởi tạo" (`app/api/ai_plan.py:523`).
- **Idempotency / chống double-click**: nếu tin nhắn cuối là một USER bắt đầu bằng `[regenerate]` (chưa được trả lời) ⇒ coi như đang có một lần regenerate đang chạy, ném `DomainError` từ chối lần thứ 2 (`app/api/ai_plan.py:531`) để tránh interleave/drop turn trên phiên in-memory.

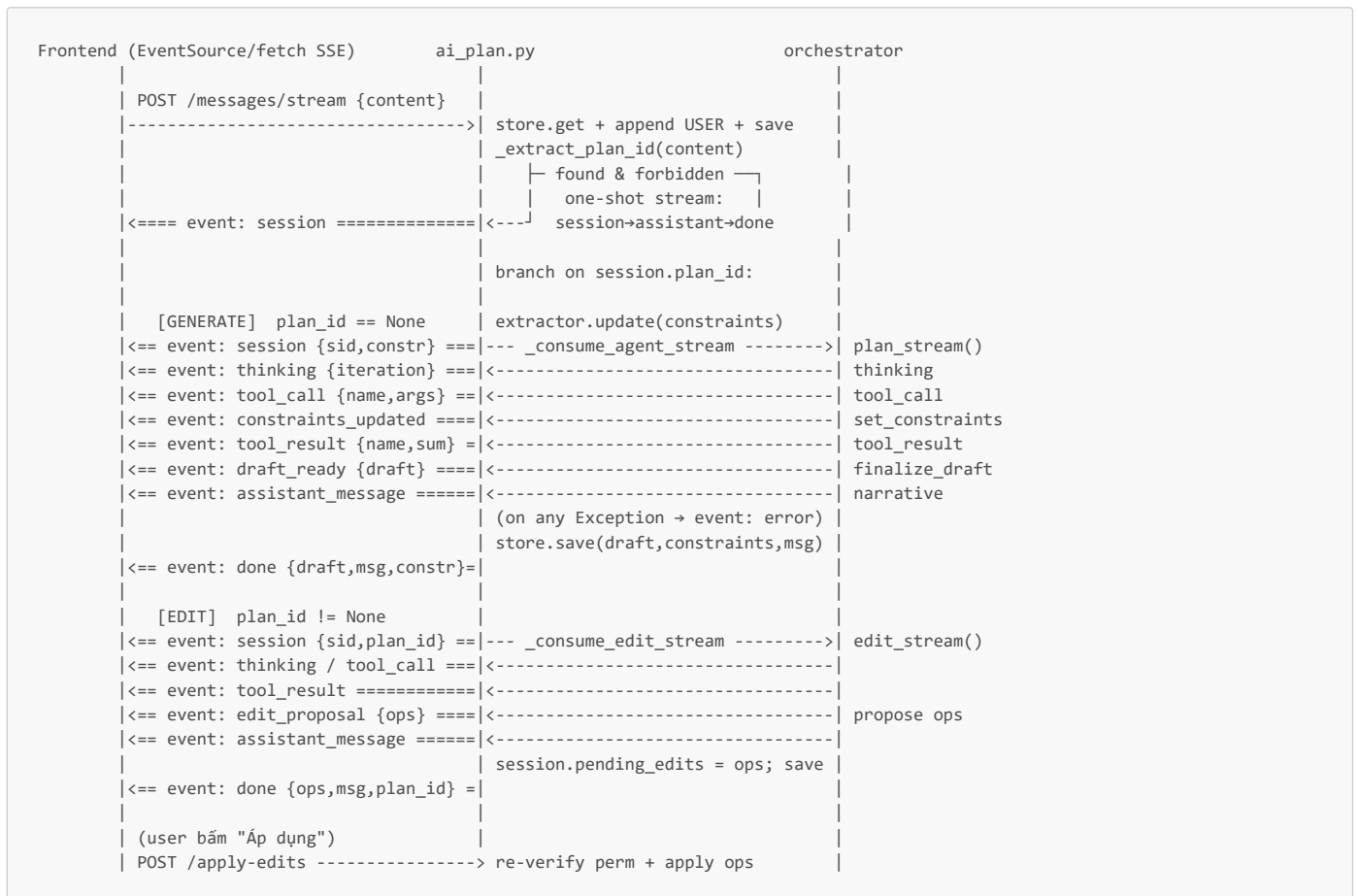
Sau đó append `ChatMessage(USER, "[regenerate] {instruction}")`, `store.save`, rồi gọi chính `_consume_agent_stream` nhưng có thêm `revision_hint=instruction` (`app/api/ai_plan.py:554`). Body `RegenerateRequest` (`app/api/schemas.py:61`): `{ instructions?: str }`; rỗng ⇒ mặc định "Hãy thử một phương án khác." Tập SSE event giống hết Luồng A.

`POST /sessions/{id}/regenerate` → `regenerate` (`app/api/ai_plan.py:562`): bản non-stream tương đương, KHÔNG có gate idempotency `[regenerate]`, gọi `agent.plan(...)` await trọn, trả `ApiResponse[SendMessageResponse]` (cùng shape mục 5).

9. Lịch sử & xem lại phiên — history endpoints

- `GET /sessions` → `list_sessions` (`app/api/ai_plan.py:189`): trả `ApiResponse[List[SessionSummary]]`. `store.list_by_user` đã sort `updated_at` giảm dần; **lọc bỏ phiên rỗng** (if `s.messages`) để danh sách có nghĩa (`app/api/ai_plan.py:196`). Mỗi `SessionSummary` (`app/api/schemas.py:29`) build bởi `_summarize_session` (`app/api/ai_plan.py:138`): `{ session_id, title (≤50 ký tự, từ tin USER đầu, mặc định "Hội thoại mới"), preview (≤70 ký tự, từ tin cuối), status, plan_id, message_count, updated_at }`. `_truncate` (`app/api/ai_plan.py:133`) gộp whitespace + thêm "...".
- `GET /sessions/{id}` → `get_session` (`app/api/ai_plan.py:200`): trả `ApiResponse[SessionView]` đầy đủ (qua `_to_view`, `app/api/ai_plan.py:113`) để khôi phục toàn bộ hội thoại (messages, draft, pending_edits, constraints) khi user mở lại.

10. Sơ đồ trình tự — streaming send (generate vs edit)



11. Tóm tắt hợp đồng SSE cho frontend

Event	Khi nào	Payload	Luồng
session	đầu tiên, ngay khi mở stream	{session_id, constraints} (generate) / {session_id, plan_id} (edit)	cả hai
thinking	đầu mỗi vòng LLM	{iteration}	cả hai
tool_call	LLM gọi tool	{name, arguments}	cả hai
tool_result	tool chạy xong	{name, summary}	cả hai
constraints_updated	sau set_constraints/finalize_draft	{constraints}	generate
draft_ready	có PlanDraft	{draft}	generate
edit_proposal	agent đề xuất sửa	{operations}	edit
assistant_message	câu trả lời/làm rõ	{text}	cả hai
error	exception bất kỳ	{message}	cả hai
done	c cuối cùng, luôn phát	{draft?, assistant_message, constraints} (generate) / {operations, assistant_message, plan_id} (edit)	cả hai

Điểm cốt lõi: `session` luôn mở đầu, `done` luôn kết thúc (kể cả khi có `error` ở giữa — stream không treo); frontend chỉ commit trạng thái cuối (`draft/constraints/pending_edits`) tại `done`, và phân biệt `generate/edit` qua `plan_id`.

12. Endpoint kết thúc vòng đời (tham chiếu chéo)

Hai endpoint dưới đây kết thúc vòng đời và chuyển trạng thái (chi tiết ở Luồng A và Luồng B):

- `POST /sessions/{id}/approve` ([app/api/ai_plan.py:618](#)): `gate status != APPROVED + draft != None`; thành công $\Rightarrow$ `status = APPROVED`, `approved_plan_id = result.plan_id`. **Idempotency**: gọi lại khi đã `APPROVED` $\Rightarrow$ `DomainError` "Plan đã được tạo trước đó".
- `POST /sessions/{id}/apply-edits` ([app/api/ai_plan.py:655](#)): `gate plan_id != None + pending_edits` không rỗng; re-verify `_assert_can_edit` tại thời điểm ghi (vai trò có thể bị thu hồi sau khi mở phiên); **idempotency/đồng thời**: snapshot `applied_edits`, sau khi apply chỉ xóa `pending_edits` nếu nó vẫn đúng bằng tập đã apply ([app/api/ai_plan.py:705](#)).

Khởi động, Dependency Injection & Cấu hình (main, config, dependencies, factory, core/*)

Cụm module này là "khung xương" của `ai-plan-service`: nó dựng ứng dụng FastAPI, đọc cấu hình từ biến môi trường, lắp ráp các dependency (LLM client, các HTTP client xuống service khác, các service nghiệp vụ), định nghĩa envelope phản hồi chuẩn `ApiResponse`, đăng ký bộ xử lý lỗi toàn cục, và giải mã JWT để xác định người dùng hiện tại. Đây là điểm khởi đầu (`app.main:app`) mà `uvicorn` nạp khi chạy service.

app/main.py — Điểm khởi động & lắp ráp ứng dụng FastAPI

Trách nhiệm: tạo instance `FastAPI`, gán middleware CORS, đăng ký exception handlers, mount router chính của AI plan, expose một endpoint health-check, và (khi chạy trực tiếp) khởi động `uvicorn`.

`create_app()` -> `FastAPI`

App-factory. Không nhận tham số, trả về một instance `FastAPI` đã cấu hình đầy đủ.

```
def create_app() -> FastAPI:
    settings = get_settings()
    app = FastAPI(title="Mravel AI Plan Service", version="0.1.0")
    app.add_middleware(
        CORSMiddleware,
        allow_origins=settings.cors_origin_list(),
        allow_credentials=True,
        allow_methods=["*"],
        allow_headers=["*"],
    )
    register_exception_handlers(app)
    app.include_router(ai_plan_router)
```

- Gọi `get_settings()` ([app/config.py](#)) để lấy `Settings` đã cache ([app/main.py:15](#)).
- Tạo `FastAPI` với `title="Mravel AI Plan Service"` và `version="0.1.0"` ([app/main.py:16](#)).
- Gán `CORSMiddleware`: `allow_origins` lấy từ `settings.cors_origin_list()` (danh sách origin tách từ env `CORS_ORIGINS`), `allow_credentials=True`, cho phép mọi method và mọi header ([app/main.py:18](#)). Lưu ý "gotcha": dùng đồng thời `allow_credentials=True` với `allow_methods=["*"]/allow_headers=["*"]` là hợp lệ, nhưng `allow_origins` KHÔNG được là `"*"` khi bật credentials — ở đây origin là danh sách cụ thể nên an toàn.
- `register_exception_handlers(app)` đăng ký toàn bộ handler lỗi từ `app/core/errors.py` ([app/main.py:26](#)).
- `app.include_router(ai_plan_router)` mount router AI plan import từ `app/api/ai_plan.py` ([app/main.py:6](#), [app/main.py:27](#)).

Side effect: việc gọi `get_settings()` lần đầu sẽ trigger đọc file `.env` và biến môi trường (qua `pydantic-settings`).

`health()` — endpoint health-check

```
@app.get("/api/ai-plan/health")
async def health() -> ApiResponse[dict]:
    return ApiResponse.ok("OK", {"service": "ai-plan-service", "version": "0.1.0"})
```

- Route: GET /api/ai-plan/health (app/main.py:29).
- Trả về ApiResponse[dict] với success=true, message="OK", data={"service": "ai-plan-service", "version": "0.1.0"}, timestamp tự sinh. Không có side effect, không gọi ra ngoài.

Khởi module-level và __main__

```
app = create_app()

if __name__ == "__main__":
    import uvicorn
    settings = get_settings()
    uvicorn.run("app.main:app", host=settings.ai_plan_host, port=settings.ai_plan_port)
```

- app = create_app() ở mức module (app/main.py:36) là biến mà uvicorn app.main:app import. App được tạo ngay khi module nạp.
- logging.basicConfig(level=logging.INFO) đặt mức log toàn cục là INFO (app/main.py:11).
- Khởi __main__ (app/main.py:39) chỉ chạy khi gọi python -m app.main trực tiếp; nó dùng settings.ai_plan_host/settings.ai_plan_port. Comment nêu rõ chủ ý KHÔNG dùng reload=True; VSCode task chạy CLI uvicorn riêng, và tránh reload để không sinh tiến trình con multiprocessing mồ côi (orphan) sống sót sau khi kill tiến trình cha trên Windows — đây là một "gotcha" vận hành đặc thù Windows.

app/config.py — Cấu hình tập trung (pydantic-settings)

Trách nhiệm: định nghĩa toàn bộ biến môi trường/cấu hình của service trong một class Settings (đọc từ .env và env), kèm các helper chuyển đổi và một getter có cache.

class Settings(BaseSettings)

Mọi field đều có default, nên service vẫn chạy được khi thiếu .env (phù hợp dev/test). model_config cấu hình nguồn đọc:

```
class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", env_file_encoding="utf-8", extra="ignore")
```

- env_file=".env", encoding UTF-8, extra="ignore" (app/config.py:8) — biến lạ trong .env bị bỏ qua thay vì gây lỗi. Tên field map sang env theo quy ước viết hoa (vd field llm_api_key ↔ env LLM_API_KEY).

Danh sách field, kiểu, default và ý nghĩa (env tương ứng là tên field viết hoa):

Field	Kiểu	Default	Ý nghĩa
ai_plan_host	str	"0.0.0.0"	Host bind khi chạy __main__ (app/config.py:10)
ai_plan_port	int	8092	Port của service (app/config.py:11)
catalog_service_base_url	str	"http://localhost:8083"	Base URL catalog-service (app/config.py:13)
plan_service_base_url	str	"http://localhost:8086"	Base URL plan-service (app/config.py:14)
jwt_secret	str	""	Secret HMAC để verify JWT; rỗng → KHÔNG verify chữ ký (app/config.py:16)
jwt_algorithm	str	"HS256"	Thuật toán JWT (app/config.py:17)
llm_api_key	str	"9router-local"	API key LLM; rỗng → dùng stub offline (app/config.py:21)
llm_base_url	str	"http://localhost:20128/v1"	Endpoint OpenAI-compatible (app/config.py:22)
llm_model	str	"Mravel"	Model chính (app/config.py:23)
llm_extractor_model	str	""	Model riêng cho trích xuất; rỗng → fallback về llm_model (app/config.py:24)
llm_fallback_models	str	""	Chuỗi model phân tách bằng dấu phẩy, thử khi model chính trả 429/402/5xx (app/config.py:28)
llm_app_name	str	"Mravel AI Plan Agent"	Tên app gửi kèm (header phục vụ OpenRouter) (app/config.py:29)
llm_app_url	str	""	URL app gửi kèm (app/config.py:30)
llm_max_tokens	int	8000	Max output tokens cho một lượt agent (app/config.py:34)
web_search_provider	str	"tavily"	Provider web search (app/config.py:38)

Field	Kiểu	Default	Ý nghĩa
web_search_api_key	str	""	API key web search; rỗng → tắt, agent dùng heuristic giá (app/config.py:39)
public_web_base_url	str	""	Origin web app để tạo link tuyệt đối; rỗng → dùng path tương đối (app/config.py:45)
cors_origins	str	"http://localhost:5173,http://localhost:8080"	Danh sách origin CORS, phân tách bằng dấu phẩy (app/config.py:47)
http_timeout_seconds	float	30.0	Timeout cho HTTP client gọi downstream (app/config.py:48)
session_store_backend	str	"mongo"	Backend lưu session: "mongo"/"file"/"memory" (app/config.py:53)
mongo_uri	str	"mongodb://localhost:27017"	Connection string MongoDB (Atlas thật để trong .env) (app/config.py:57)
mongo_db	str	"mraavel_catalog"	Tên DB (app/config.py:58)
session_collection	str	"ai_plan_sessions"	Collection lưu session (app/config.py:59)
session_migrate_from_file	bool	True	Lần đầu khởi động Mongo, import session từ file legacy nếu collection rỗng (app/config.py:62)
session_store_path	str	".data/sessions.json"	Path file backend legacy / nguồn migrate (app/config.py:65)

Lưu ý quan trọng: `mongo_db` mặc định là `"mraavel_catalog"` — tức session AI plan được lưu chung DB với dữ liệu catalog (tải dùng instance MongoDB của stack), chỉ khác collection. Các giá trị nhạy cảm (connection string Atlas chứa credential) phải đặt trong `.env`, không `hardcode`.

Các "gotcha" về fallback logic được encode trong default/comment:

- `llm_api_key=""` → service chuyển sang stub offline (xem `factory.py`).
- `web_search_api_key=""` → web search bị vô hiệu, agent dùng heuristic pricing.
- `public_web_base_url=""` → sinh path tương đối như `/hotels/<slug>` để browser tự resolve theo origin SPA.

Các method helper của `Settings`

```
def cors_origin_list(self) -> List[str]:
    return [o.strip() for o in self.cors_origins.split(",") if o.strip()]

def has_llm(self) -> bool:
    return bool(self.llm_api_key.strip())

def effective_extractor_model(self) -> str:
    return self.llm_extractor_model.strip() or self.llm_model

def fallback_model_list(self) -> List[str]:
    return [m.strip() for m in self.llm_fallback_models.split(",") if m.strip()]
```

- `cors_origin_list()` -> `List[str]` (app/config.py:67): tách `cors_origins` theo dấu phẩy, strip khoảng trắng, bỏ phần tử rỗng. Dùng ở `main.create_app()`.
- `has_llm()` -> `bool` (app/config.py:70): `True` nếu `llm_api_key` (đã strip) không rỗng. Là cờ quyết định stub vs LLM thật trong `factory.build_llm_client`.
- `effective_extractor_model()` -> `str` (app/config.py:73): trả `llm_extractor_model` đã strip, hoặc fallback về `llm_model` nếu rỗng. Cho phép dùng model rẻ/nhanh riêng cho việc trích constraint.
- `fallback_model_list()` -> `List[str]` (app/config.py:76): tách `llm_fallback_models` theo dấu phẩy thành list, dùng làm chuỗi model dự phòng.

`get_settings()` -> `Settings`

```
@lru_cache
def get_settings() -> Settings:
    return Settings()
```

- Có `@lru_cache` (app/config.py:80) nên `Settings()` chỉ được khởi tạo MỘT lần trong vòng đời tiến trình → singleton. Mọi nơi gọi `get_settings()` đều nhận cùng instance.
- Side effect (lần gọi đầu): `pydantic-settings` đọc `.env` + biến môi trường để dựng `Settings`.
- "Gotcha": vì cache theo tham số và hàm không nhận tham số, đổi env sau khi đã gọi sẽ không có hiệu lực cho đến khi restart tiến trình. Đây cũng là lý do `dependencies.py` đọc trực tiếp `get_settings()` thay vì truyền `Settings` làm tham số (xem dưới).

app/api/dependencies.py — Dependency Injection (DI)

Trách nhiệm: định nghĩa các "provider" cho FastAPI `Depends()` — vừa các singleton hạ tầng (HTTP client, LLM client) được cache, vừa các factory tạo service nghiệp vụ theo từng request. Đây là nơi lắp ráp toàn bộ object graph của service.

Hai tầng provider:

- Hàm cache nội bộ (`_catalog_client`, `_plan_client`, `_web_search_client`, `_llm_client`) — `@lru_cache`, tạo đúng một instance singleton.
- Hàm public không cache (`get_catalog_client`, ...) — chỉ ủy thác sang hàm cache; là thứ truyền vào `Depends(...)`.

```
@lru_cache
def _catalog_client() -> CatalogClient:
    return CatalogClient(get_settings())
```

"Gotcha" được ghi rõ trong comment ([app/api/dependencies.py:19](#)): `Settings` (pydantic model) KHÔNG hashable, nên không thể truyền vào hàm `@lru_cache`; do đó các hàm cache không nhận tham số mà tự gọi `get_settings()` bên trong.

Các singleton hạ tầng

- `_catalog_client()` ([app/api/dependencies.py:22](#)) / `get_catalog_client()` ([app/api/dependencies.py:42](#)): tạo `CatalogClient(get_settings())`. Client HTTP gọi catalog-service.
- `_plan_client()` ([app/api/dependencies.py:27](#)) / `get_plan_client()` ([app/api/dependencies.py:46](#)): tạo `PlanClient(get_settings())`. Client HTTP gọi plan-service.
- `_web_search_client()` ([app/api/dependencies.py:32](#)) / `get_web_search_client()` ([app/api/dependencies.py:50](#)): tạo `WebSearchClient(get_settings())`. Client web search (Tavily...).
- `_llm_client()` ([app/api/dependencies.py:37](#)) / `get_llm_client()` ([app/api/dependencies.py:54](#)): gọi `build_llm_client(get_settings())` (factory). Trả `LLMClient` (interface base ở [app/llm/base.py](#)), thực tế là `OpenAILLMClient` hoặc `StubLLMClient`.

Tất cả đều singleton suốt vòng đời tiến trình nhờ `@lru_cache` ở tầng nội bộ — tái dùng connection pool, tránh dựng lại client mỗi request.

Các factory service nghiệp vụ (không cache, theo request)

```
def get_constraint_extractor(llm: LLMClient = Depends(get_llm_client)) -> ConstraintExtractor:
    return ConstraintExtractor(llm)

def get_draft_composer(catalog: CatalogClient = Depends(get_catalog_client)) -> DraftComposer:
    return DraftComposer(catalog)

def get_agent_orchestrator(llm, catalog, composer, plan_client, web_search) -> AgentOrchestrator:
    settings = get_settings()
    return AgentOrchestrator(
        llm, catalog, composer,
        plan_client=plan_client, web_search=web_search,
        web_base_url=settings.public_web_base_url,
        max_tokens=settings.llm_max_tokens,
    )
```

- `get_constraint_extractor(llm)` ([app/api/dependencies.py:58](#)): bọc `ConstraintExtractor` quanh LLM client.
- `get_draft_composer(catalog)` ([app/api/dependencies.py:62](#)): bọc `DraftComposer` quanh catalog client.
- `get_agent_orchestrator(...)` ([app/api/dependencies.py:66](#)): factory phức tạp nhất — FastAPI tự resolve toàn bộ dependency con (`llm`, `catalog`, `composer`, `plan_client`, `web_search`), rồi orchestrator được dựng với hai tham số rút trực tiếp từ settings: `web_base_url=settings.public_web_base_url` và `max_tokens=settings.llm_max_tokens`. Đây là "trái tim" mà các endpoint chat/agent dùng (xem [app/agent/orchestrator.py](#)).
- `get_approval_service(plan_client)` ([app/api/dependencies.py:85](#)): bọc `ApprovalService` quanh plan client (duyet/áp dụng plan).
- `get_edit_service(plan_client)` ([app/api/dependencies.py:89](#)): bọc `EditService` quanh plan client (sửa plan).
- `get_store()` ([app/api/dependencies.py:93](#)): trả về `get_session_store()` từ [app/services/session_store.py](#). Lưu ý kiểu trả về annotation là `InMemorySessionStore` nhưng thực thể runtime có thể là backend Mongo/File/Memory tùy `session_store_backend` — annotation chỉ phản ánh interface chung, không phải implementation cụ thể.

"Gotcha" DI: các factory nghiệp vụ KHÔNG có `@lru_cache`, nên về danh nghĩa được gọi lại mỗi request — nhưng vì FastAPI tự cache kết quả `Depends` trong phạm vi một request, và các dependency hạ tầng bên dưới đều singleton, chỉ phí tạo `ConstraintExtractor/DraftComposer/AgentOrchestrator` chỉ là dựng wrapper nhẹ quanh các singleton sẵn có.

app/llm/factory.py — Factory chọn implementation LLM

Trách nhiệm: quyết định dùng LLM thật hay stub offline dựa trên cấu hình.

`build_llm_client(settings: Settings) -> LLMClient`

```
def build_llm_client(settings: Settings) -> LLMClient:
    if not settings.has_llm():
        return StubLLMClient()
    from app.llm.openai_client import OpenAILLMClient
    return OpenAILLMClient(
        api_key=settings.llm_api_key,
        base_url=settings.llm_base_url,
        model=settings.llm_model,
        extractor_model=settings.effective_extractor_model(),
        app_name=settings.llm_app_name,
```

```

app_url=settings.llm_app_url,
fallback_models=settings.fallback_model_list(),
)

```

- Tham số: `settings: Settings`. Trả về một `LLMClient` (interface).
- Nhánh fallback offline: nếu `settings.has_llm()` là `False` (tức `llm_api_key` rỗng) → trả `StubLLMClient()` ([app/llm/factory.py:15](#)). Stub dùng regex extractor, KHÔNG có vòng lặp agent — cho phép chạy/test service mà không cần API key.
- Nhánh online: tạo `OpenAILLMClient` với toàn bộ tham số map từ settings ([app/llm/factory.py:20](#)). Đáng chú ý: `extractor_model=settings.effective_extractor_model()` (đã áp dụng fallback về `llm_model`), và `fallback_models=settings.fallback_model_list()` (list đã parse).
- "Gotcha" import: `from app.llm.openai_client import OpenAILLMClient` được import LAZY bên trong nhánh online ([app/llm/factory.py:18](#)), không ở đầu file. Mục đích: khi chạy ở chế độ stub (không có LLM), không phải import module client OpenAI và các dependency nặng/optional của nó (vd thư viện `openai/httpx` cấu hình riêng), tránh lỗi import khi môi trường tối giản.
- Docstring ([app/llm/factory.py:7](#)) nêu rõ client là "OpenAI-compatible", chạy được với OpenRouter, OpenAI, Groq, Together, Ollama — đổi provider chỉ cần đổi `LLM_BASE_URL + LLM_MODEL`.
- Không có side effect mạng tại thời điểm build (client lười tạo kết nối khi gọi).

app/core/response.py — Envelope phản hồi chuẩn `ApiResponse[T]`

Trách nhiệm: định nghĩa cấu trúc JSON bao bọc mọi phản hồi API, khớp với `com.mravel.common.response.ApiResponse<T>` bên backend Java để frontend (`axiosInstance`) đọc cùng một envelope qua cả service Java lẫn Python.

```
class ApiResponse(BaseModel, Generic[T])
```

```

class ApiResponse(BaseModel, Generic[T]):
    success: bool
    message: str
    data: Optional[T] = None
    timestamp: datetime = Field(default_factory=lambda: datetime.now(timezone.utc))

```

Các field (thứ tự và tên phải giữ khớp với Java) ([app/core/response.py:16](#)):

- `success: bool` — cờ thành công/thất bại.
- `message: str` — thông điệp người-đọc-được.
- `data: Optional[T] = None` — payload generic; `None` khi lỗi.
- `timestamp: datetime` — mặc định `datetime.now(timezone.utc)` (UTC, có timezone) sinh tại thời điểm tạo object qua `default_factory`.

`Generic[T]` cho phép khai báo kiểu payload, vd `ApiResponse[dict]` ở health-check.

Constructor tiện ích

```

@classmethod
def ok(cls, message: str, data: T | None = None) -> "ApiResponse[T]":
    return cls(success=True, message=message, data=data)

@classmethod
def err(cls, message: str) -> "ApiResponse[Any]":
    return cls(success=False, message=message, data=None)

```

- `ApiResponse.ok(message, data=None)` ([app/core/response.py:21](#)): tạo response thành công, `success=True`, gán `data` (mặc định `None`).
- `ApiResponse.err(message)` ([app/core/response.py:25](#)): tạo response lỗi, `success=False`, `data=None`. Được các exception handler dùng để serialize lỗi.

"Gotcha" serialize: ở các handler lỗi, response được dump bằng `.model_dump(mode="json")` để `datetime` chuyển thành chuỗi ISO-8601 hợp lệ cho JSON (xem [errors.py](#)).

app/core/errors.py — Exception nghiệp vụ & handler toàn cục

Trách nhiệm: định nghĩa các exception miền (domain) ánh xạ sang HTTP status, và đăng ký handler để mọi lỗi đều trả về cùng envelope `ApiResponse.err(...)`.

Các lớp exception

Tất cả đều kế thừa `Exception` và lưu `self.message`:

- `DomainError(message)` — vi phạm business rule → HTTP **400** ([app/core/errors.py:9](#)).
- `UnauthorizedError(message="Unauthorized")` — chưa xác thực / token sai → HTTP **401** ([app/core/errors.py:17](#)). Default message "Unauthorized".
- `ForbiddenError(message="Forbidden")` — đã xác thực nhưng thiếu quyền (vd chỉ có role VIEWER trên plan muốn sửa) → HTTP **403** ([app/core/errors.py:23](#)). Default message "Forbidden".
- `UpstreamError(message)` — downstream service (plan/catalog) lỗi → HTTP **502** ([app/core/errors.py:32](#)).

```
register_exception_handlers(app: FastAPI) -> None
```

Đăng ký 6 handler. Mỗi handler trả `JSONResponse` với status code tương ứng và body là `ApiResponse.err(...).model_dump(mode="json")`.

```
@app.exception_handler(DomainError)
async def _domain(_: Request, exc: DomainError) -> JSONResponse:
    return JSONResponse(status_code=400, content=ApiResponse.err(exc.message).model_dump(mode="json"))
```

- `DomainError` → 400 ([app/core/errors.py:41](#)).
- `UnauthorizedError` → 401 ([app/core/errors.py:45](#)).
- `ForbiddenError` → 403 ([app/core/errors.py:49](#)).
- `UpstreamError` → 502 ([app/core/errors.py:53](#)).
- `RequestValidationError` (validation body/query của FastAPI) → 400 ([app/core/errors.py:57](#)): chỉ lấy thông điệp của lỗi ĐẦU TIÊN (`exc.errors()[0]`), fallback "Invalid request" nếu danh sách rỗng. "Gotcha": nếu request có nhiều lỗi validation, người dùng chỉ thấy lỗi đầu tiên — phần `loc`/chi tiết còn lại bị bỏ.
- `StarletteHTTPException` (mọi `HTTPException` chuẩn, vd 404/405) → giữ nguyên `exc.status_code`, message lấy từ `exc.detail` (fallback "Error") ([app/core/errors.py:65](#)).

Điểm thiết kế quan trọng: nhờ handler `StarletteHTTPException`, kể cả lỗi 404/405 mặc định của FastAPI cũng được bọc thành envelope `ApiResponse`, đảm bảo frontend luôn nhận cùng cấu trúc `{success, message, data, timestamp}`. Tham số `Request` được đặt tên `_` vì không dùng. Mọi handler đều dùng `.model_dump(mode="json")` để `timestamp` (datetime) được serialize an toàn.

"Gotcha": không có handler bắt `Exception` chung — lỗi không lường trước (vd `KeyError`, `RuntimeError`) sẽ rơi vào xử lý mặc định của Starlette và trả HTTP 500 dạng plain (không theo envelope `ApiResponse`).

`app/core/security.py` — Xác thực JWT & người dùng hiện tại

Trách nhiệm: giải mã bearer token để lấy `user_id`, expose dependency `get_current_user` cho các endpoint cần xác thực. Token gốc được giữ lại để forward xuống downstream service.

```
@dataclass(frozen=True) class CurrentUser
```

```
@dataclass(frozen=True)
class CurrentUser:
    id: int
    raw_token: str # full bearer token, forwarded to downstream services
```

- `id: int` — id số của user ([app/core/security.py:13](#)).
- `raw_token: str` — token bearer đầy đủ (không gồm tiền tố `Bearer`), được forward nguyên vẹn sang catalog/plan service để chúng tự xác thực lại ([app/core/security.py:14](#)).
- `frozen=True` → immutable, hashable.

```
_decode_subject(token: str) -> int
```

Helper nội bộ giải mã token và rút ra user id.

- Đọc `settings` qua `get_settings()` ([app/core/security.py:18](#)).
- Nhánh có verify: nếu `jwt_secret` không rỗng → `jwt.decode` verify chữ ký bằng `jwt_secret` và `algorithms=[jwt_algorithm]` ([app/core/security.py:20](#)).
- Nhánh KHÔNG verify (gotcha bảo mật quan trọng): nếu `jwt_secret` rỗng → `jwt.decode(token, options={"verify_signature": False})` — KHÔNG kiểm tra chữ ký ([app/core/security.py:23](#)). Lý do trong comment: auth-service/gateway/plan-service đã verify upstream; service này chỉ cần `user_id` cho traceability nên "tin" subject claim chưa verify. Hệ quả: ở môi trường không cấu hình `JWT_SECRET`, ai cũng có thể giả mạo id claim — chấp nhận được vì chỉ là service nội bộ phía sau gateway, nhưng KHÔNG được expose trực tiếp ra internet mà thiếu secret.
- Bắt mọi `jwt.PyJWTError` → ném `UnauthorizedError(f"Invalid token: {exc}")` ([app/core/security.py:26](#)) → handler trả HTTP 401.
- Thứ tự ưu tiên claim: `id` → `userId` → `user_id` → `uid` ([app/core/security.py:31](#)). Comment nêu rõ: Mravel auth-service lưu id số ở claim `id`, còn `sub` chứa `email` (nên KHÔNG dùng `sub` làm id). Nếu không claim nào có giá trị → `UnauthorizedError("Token missing user id claim")` ([app/core/security.py:37](#)). Ép `int(raw)`; nếu không phải số → `UnauthorizedError("Token user id claim is not numeric")` ([app/core/security.py:39](#)).
- "Gotcha" với `or`: vì dùng chuỗi `or`, một claim `id` có giá trị falsy hợp lệ (vd `0`) sẽ bị bỏ qua và rơi sang claim kế tiếp — trong thực tế user id bắt đầu từ ≥ 1 nên không phát sinh, nhưng đáng lưu ý.

```
get_current_user(authorization) -> CurrentUser
```

Dependency FastAPI, đọc header `Authorization`.

```
async def get_current_user(authorization: Optional[str] = Header(default=None)) -> CurrentUser:
    if not authorization or not authorization.lower().startswith("bearer "):
        raise UnauthorizedError("Missing Authorization: Bearer header")
    token = authorization.split(" ", 1)[1].strip()
    if not token:
        raise UnauthorizedError("Empty bearer token")
    return CurrentUser(id=_decode_subject(token), raw_token=token)
```

- Tham số `authorization: Optional[str]` lấy từ HTTP header `Authorization` (FastAPI `Header(default=None)`) ([app/core/security.py:45](#)).

- Validate: thiếu header hoặc không bắt đầu bằng "bearer " (so sánh case-insensitive qua `.lower()`) → `UnauthorizedError("Missing Authorization: Bearer header")` ([app/core/security.py:46](#)).
- Tách token sau khoảng trắng đầu tiên (`split(" ", 1)[1]`) rồi `strip()` ([app/core/security.py:48](#)); nếu rỗng → `UnauthorizedError("Empty bearer token")` ([app/core/security.py:49](#)).
- Trả `CurrentUser(id=_decode_subject(token), raw_token=token)` — `raw_token` là token đã bỏ tiền tố Bearer ([app/core/security.py:51](#)).
- Mọi `UnauthorizedError` phát sinh đều được handler ở `errors.py` ánh xạ về HTTP 401 với envelope `ApiResponse.err(...)`.

Tổng kết luồng phụ thuộc của cụm: `main.create_app()` → đăng ký handler (`errors.py`) + router → các endpoint trong router dùng `Depends(get_*)` từ `dependencies.py` → `dependencies` lắp ráp client/LLM (qua `factory.build_llm_client`) và service nghiệp vụ, tất cả lấy cấu hình từ `config.get_settings()` (singleton cache) → endpoint cần auth thì thêm `Depends(get_current_user)` (`security.py`) → mọi phản hồi (thành công lẫn lỗi) đều đóng gói trong `ApiResponse(response.py)`.

Lớp API — toàn bộ endpoints, dò plan id, gate quyền, đóng gói SSE & schemas

Cụm module này là tầng HTTP của `ai-plan-service`: nó định nghĩa toàn bộ REST/SSE endpoints dưới prefix `/api/ai-plan`, ánh xạ request → phiên hội thoại (`PlanSession`), gọi vào tầng dịch vụ/agent, và đóng gói kết quả thành `ApiResponse` hoặc `text/event-stream`. Có hai file: `app/api/ai_plan.py` chứa router + toàn bộ logic điều phối, và `app/api/schemas.py` định nghĩa các Pydantic model dùng làm request body và response payload.

app/api/schemas.py

Trách nhiệm: khai báo các DTO (request/response) cho lớp API. Tất cả đều là `pydantic.BaseModel`, và tái sử dụng các domain model `ChatMessage`, `Constraints`, `PlanDraft` từ `app/models/session` (import tại [app/api/schemas.py:6](#)). Không có logic, không có side effect — chỉ là kế ước dữ liệu giữa frontend và service.

Danh sách model với chữ ký field cụ thể:

- **CreateSessionRequest** ([app/api/schemas.py:9](#)) — body cho POST `/sessions`.
 - `initial_message: Optional[str] = None` — tin nhắn đầu tiên (tùy chọn) của người dùng.
 - `plan_id: Optional[int] = None` — nếu set, phiên ở **edit mode** (chỉnh sửa plan đã tồn tại) thay vì tạo plan mới.
- **SessionView** ([app/api/schemas.py:17](#)) — chế độ xem đầy đủ một phiên; là payload của `create_session`, `get_session`.
 - `session_id: str, status: str, user_id: int`.
 - `constraints: Constraints` (domain model).
 - `messages: List[ChatMessage]` — toàn bộ lịch sử chat.
 - `draft: Optional[PlanDraft] = None` — bản nháp plan hiện tại.
 - `approved_plan_id: Optional[int] = None` — id plan đã được tạo sau approve.
 - `plan_id: Optional[int] = None` — id plan đang được chỉnh (edit mode).
 - `pending_edits: List[Dict[str, Any]] = Field(default_factory=list)` — danh sách operations đề xuất chờ "Áp dụng".
- **SessionSummary** ([app/api/schemas.py:29](#)) — một dòng trong danh sách lịch sử chat AI.
 - `session_id: str, title: str, preview: str = "", status: str`.
 - `plan_id: Optional[int] = None, message_count: int = 0`.
 - `updated_at: datetime` — dùng để sắp xếp theo hoạt động mới nhất.
- **ApplyEditsResult** ([app/api/schemas.py:41](#)) — payload của `apply_edits`.
 - `applied: int, total: int` — số operations áp dụng thành công / tổng số.
 - `results: List[Dict[str, Any]]` — kết quả từng op (mỗi phần tử có ít nhất field `ok`).
 - `assistant_message: Optional[ChatMessage] = None`.
- **SendMessageRequest** ([app/api/schemas.py:48](#)) — body cho gửi tin nhắn.
 - `content: str = Field(min_length=1, max_length=4000)` — **ràng buộc độ dài**: rỗng hoặc > 4000 ký tự sẽ bị Pydantic từ chối (HTTP 422 — qua handler `RequestValidationError` thành 400) trước khi vào handler. Đây là gotcha quan trọng: validation xảy ra ở tầng schema, không phải trong handler.
- **SendMessageResponse** ([app/api/schemas.py:52](#)) — payload của `send_message` / `regenerate` (non-stream).
 - `session_id: str, constraints: Constraints, draft: Optional[PlanDraft] = None`.
 - `assistant_message: ChatMessage`.
 - `needs_more_info: bool` — true khi còn thiếu thông tin và chưa có draft.
 - `missing_fields: List[str] = Field(default_factory=list)` — tên field còn thiếu (vd "destination", "num_days").
- **RegenerateRequest** ([app/api/schemas.py:61](#)) — body cho regenerate.
 - `instructions: Optional[str] = Field(default=None, max_length=4000, ...)` — gợi ý ngôn ngữ tự nhiên để lái bản nháp mới (vd "rẻ hơn, đổi khách sạn khác").
- **ApprovalResult** ([app/api/schemas.py:70](#)) — payload của `approve_session`.
 - `plan_id: int` — id plan vừa được tạo trên plan-service.
 - `operations: List[Dict[str, Any]]` — danh sách operations đã ghi.

Gotcha về schema: `SessionView` và `SendMessageResponse` nhúng `Constraints/PlanDraft/ChatMessage` trực tiếp, nên hình dạng JSON trả về phụ thuộc vào định nghĩa của các domain model đó (xem [app/models/session.py](#)). Mọi response của các endpoint không-stream đều được bọc thêm một lớp `ApiResponse[...]` (từ [app/core/response.py](#)).

app/api/ai_plan.py

Trách nhiệm: định nghĩa `router = APIRouter(prefix="/api/ai-plan", tags=["ai-plan"])` ([app/api/ai_plan.py:39](#)) và toàn bộ 9 endpoint của service, cùng các hàm helper cho gate quyền, dò plan id, đóng gói SSE, và persist phiên. Module này điều phối giữa: `InMemorySessionStore` (lưu phiên), `ConstraintExtractor` (rút trích ràng buộc), `AgentOrchestrator` (LLM agent), `ApprovalService / EditService` (ghi xuống plan-service), và `PlanClient` (HTTP gọi plan-service).

Các dependency được inject qua `Depends(...)` từ [app/api/dependencies.py](#): `get_agent_orchestrator`, `get_approval_service`, `get_constraint_extractor`, `get_edit_service`, `get_plan_client`, `get_store`. `CurrentUser` lấy từ `get_current_user` ([app/core/security.py](#)), mang `id: int` và `raw_token: str` (bearer token để chuyển tiếp xuống plan-service).

Hằng số, biến môi trường, enum module dùng tới

- `_EDIT_ROLES = {"OWNER", "EDITOR"}` ([app/api/ai_plan.py:45](#)) — tập role (lấy từ `BoardResponse.myRole`) được phép edit plan. Mọi role khác (đặc biệt `VIEWER`) bị chặn.
- `_PLAN_UNIT` ([app/api/ai_plan.py:75](#)) và `_PLAN_REF_PATTERNS` ([app/api/ai_plan.py:76](#)) — regex để dò plan id từ text người dùng (xem `_extract_plan_id`).
- `logger = logging.getLogger("ai_plan.api")` ([app/api/ai_plan.py:266](#)).
- `_SSE_HEADERS` ([app/api/ai_plan.py:268](#)) — header cố định cho mọi `StreamingResponse`:

```
_SSE_HEADERS = {
    "Cache-Control": "no-cache",
    "X-Accel-Buffering": "no",          # tắt buffering của nginx để SSE đẩy ngay
    "Connection": "keep-alive",
}
```

X-Accel-Buffering: no là chi tiết then chốt: nếu thiếu, reverse-proxy (nginx) có thể gom buffer và làm chậm/treo SSE.

- Enum dùng tới (từ [app/models/session](#)): `ChatRole` (`USER/ASSISTANT`), `SessionStatus` (`DRAFTING/APPROVED`). Module không đọc env var trực tiếp; mọi cấu hình (LLM key, plan-service URL) nằm trong các dependency được inject.

Helper: gate quyền `_assert_can_edit`

```
async def _assert_can_edit(plan_client: PlanClient, bearer_token: str, plan_id: int) -> dict:
    try:
        board = await plan_client.get_board(bearer_token, plan_id)
    except UpstreamError as exc:
        raise ForbiddenError(f"Bạn không có quyền truy cập kế hoạch #{plan_id}.") from exc
    role = str(board.get("myRole") or "").upper()
    if role not in _EDIT_ROLES:
        if role == "VIEWER":
            raise ForbiddenError("Bạn chỉ có quyền XEM kế hoạch này nên không thể chỉnh sửa. ...")
        raise ForbiddenError("Bạn không có quyền chỉnh sửa kế hoạch này.")
    return board
```

([app/api/ai_plan.py:48](#))

- Tham số: `plan_client`, `bearer_token`, `plan_id`. Trả về: `dict` board (chính là `BoardResponse` từ plan-service) để caller tái sử dụng, tránh round-trip thứ hai.
- Lời gọi ra ngoài: `HTTP plan_client.get_board(...) → plan-service`.
- Xử lý lỗi: plan-service map 401/403 thành `UpstreamError`; helper chuyển thành `ForbiddenError` (HTTP 403) với thông điệp tiếng Việt. Phân biệt rõ trường hợp `VIEWER` (chỉ xem) để báo người dùng đi xin quyền `EDITOR`.
- Gotcha: đây chỉ là **early friendly gate** — theo comment, plan-service (`PlanPermissionService`) mới là authoritative enforcer, yêu cầu `EDITOR` trên mọi mutation. Gate này tránh việc một viewer dựn được proposal không thể apply.

Helper: dò plan id `_extract_plan_id`

```
_PLAN_UNIT = r"(?!s*(?:ngày|đêm|người|khách|tuần|tháng|năm|giờ|tiếng|phòng))"
_PLAN_REF_PATTERNS = [
    re.compile(r"/plans?/(\d+)", re.IGNORECASE),
    re.compile(r"\b(?:plan|kế\s*hoạch)\s*(?:số|#|id)?\s*[:#]?\s*(\d+)\b" + _PLAN_UNIT, re.IGNORECASE),
]

def _extract_plan_id(text: str) -> int | None:
    for pattern in _PLAN_REF_PATTERNS:
        match = pattern.search(text or "")
        if match:
            try:
                return int(match.group(1))
            except (TypeError, ValueError):
                continue
    return None
```

([app/api/ai_plan.py:75](#)–[app/api/ai_plan.py:93](#))

- Mục đích: cho phép chat MAI toàn cục tự "chuyển sang chế độ chỉnh sửa" khi người dùng nhắc tới plan trong tin nhắn — bắt cả URL `/plans/<id>` lẫn cách nói tự nhiên ("plan 43", "kế hoạch số 43", "plan #43").
- Tham số: `text: str` (chịu được `None` nhờ `text or ""`). Trả về: `int | None`.
- Gotcha then chốt:** negative lookahead `_PLAN_UNIT` loại bỏ các con số đi kèm đơn vị đếm/thời lượng, để "kế hoạch 3 ngày" / "plan 2 người" **không** bị hiểu nhầm là plan id 3 / 2. Pattern URL được thử trước; pattern ngôn ngữ tự nhiên thử sau.

Helper: SSE đóng gói `_sse_event`

```
def _sse_event(event: str, payload: dict) -> str:
    return f"event: {event}\ndata: {json.dumps(payload, ensure_ascii=False, default=str)}\n\n"
```

([app/api/ai_plan.py:275](#))

- Tạo một frame SSE đúng chuẩn: dòng `event:` + dòng `data:` (JSON) + dòng trống kết thúc.
- `ensure_ascii=False` giữ nguyên tiếng Việt có dấu; `default=str` để serialize được các kiểu không-JSON (vd `datetime`) mà không vỡ stream.

Helper: stream một-câu `_one_shot_assistant_stream`

([app/api/ai_plan.py:96](#))

- Phát đúng một assistant message dưới dạng SSE (và persist nó), dùng để **từ chối** một thao tác edit mà người dùng không có quyền, nhưng không để chat treo. Trình tự event: `session` → `assistant_message` → `done`. Side effect: append message + `store.save`.

Helper: chuyển đổi & tóm tắt

- `_to_view(session)` ([app/api/ai_plan.py:113](#)) → `SessionView`: ánh xạ `PlanSession` sang DTO. Lưu ý `status=session.status.value` (chuyển enum → str).
- `_user_conversation_text(session)` ([app/api/ai_plan.py:127](#)) → `str`: nối tất cả lượt của `ChatRole.USER` bằng `\n`, để extractor phục hồi được dữ kiện (đặc biệt điểm đến) đã nói ở tin nhắn trước mà pass đơn-tin-nhắn sẽ bỏ sót.
- `_truncate(text, limit)` ([app/api/ai_plan.py:133](#)) → `str`: chuẩn hóa khoảng trắng rồi cắt với hậu tố
- `_summarize_session(session)` ([app/api/ai_plan.py:138](#)) → `SessionSummary`: `title` = tin user đầu tiên (cắt 50 ký tự), fallback "Hội thoại mới"; `preview` = tin cuối (cắt 70 ký tự).
- `_missing_constraint_fields(constraints)` ([app/api/ai_plan.py:154](#)) → `list[str]`:

```
has_dates = constraints.start_date is not None and constraints.end_date is not None
if not constraints.num_days and not has_dates:
    missing.append("num_days")
```

Thiếu `destination` → thêm `"destination"`. Về timing: thỏa mãn bởi **HOẶC** `num_days` (neo theo hôm nay) **HOẶC** cặp `start_date+end_date`; không còn bắt buộc ngày lịch cụ thể.

Endpoint: POST `/sessions` — `create_session` ([app/api/ai_plan.py:166](#))

- Tạo phiên mới (HTTP 201). Nếu `plan_id` được cung cấp → vào edit mode, nhưng **gate quyền trước khi mở phiên** (raise 403 nếu không phải OWNER/EDITOR), để viewer không bao giờ chạm tới editor.
- Side effect: tạo + lưu session. Lỗi gọi ngoài: HTTP `get_board` (chỉ khi có `plan_id`).
- Trả về `ApiResponse[SessionView]`.

Endpoint: GET `/sessions` — `list_sessions` ([app/api/ai_plan.py:189](#))

Trả về `ApiResponse[List[SessionSummary]]`: lịch sử chat của user hiện tại, mới nhất trước. Gotcha: **lọc bỏ phiên rỗng** (if `s.messages`) — phiên tạo nhưng chưa dùng bị ẩn để danh sách có ý nghĩa. Không gọi ngoài (chỉ đọc store theo `user.id`).

Endpoint: GET `/sessions/{session_id}` — `get_session` ([app/api/ai_plan.py:200](#))

Trả về `ApiResponse[SessionView]`. `store.get(session_id, user.id)` ràng buộc quyền sở hữu: lấy phiên của *chính user đó* (nếu không thuộc user, store sẽ ném lỗi). Không gọi ngoài.

Endpoint: POST `/sessions/{session_id}/messages` — `send_message (non-stream)` ([app/api/ai_plan.py:210](#))

Biến thể đồng bộ của gửi tin nhắn (không SSE).

- Edge case 1: nếu `session.plan_id is not None` → raise `DomainError("Chế độ chỉnh sửa kế hoạch dùng luồng streaming (/messages/stream).")` — edit mode chỉ streaming.
- Edge case 2: nếu `session.status != SessionStatus.DRAFTING` → `DomainError("Session is no longer in DRAFTING state")`.
- Luồng: append tin user → `extractor.update(...)` cập nhật constraints (truyền `conversation_text=user_conversation_text(session)`) → gọi `agent.plan(...)` (LLM) với `chat_history=session.messages[:-1]` (loại tin vừa append), `prior_draft`, `bearer_token`. Agent là source of truth cho constraints và có thể tinh chỉnh giữa lượt qua `set_constraints` — kết quả ghi lại vào `session.constraints = updated_constraints`.
- Side effect: append assistant message, `store.save`. Lỗi gọi ngoài: **LLM** (`agent.plan`), và có thể **HTTP** từ trong agent (tool calls xuống plan-service).
- Trả về `ApiResponse[SendMessageResponse]`; `needs_more_info = bool(missing)` and `session.draft is None`.

Helper SSE chính: `_consume_agent_stream` (chế độ tạo mới/regenerate) ([app/api/ai_plan.py:279](#))

Là async generator yield các SSE frame, dùng cho `stream_message` (tạo draft) và `stream_regenerate`.

- Khởi đầu yield `session` (kèm `constraints` JSON). Sau đó lặp `async for ev in agent.plan_stream(...)`, dịch từng `ev["event"]` của agent thành SSE event tương ứng:

```
async for ev in agent.plan_stream(...):
    kind = ev.get("event")
    if kind == "draft_ready":
        final_draft = ev["draft"]
        yield _sse_event("draft_ready", {"draft": json.loads(final_draft.model_dump_json())})
    elif kind == "assistant_message":
        final_text = ev.get("text", "") or final_text
        yield _sse_event("assistant_message", {"text": final_text})
    elif kind == "constraints_updated":
        final_constraints = ev.get("constraints") or final_constraints
        yield _sse_event("constraints_updated", {"constraints": ...})
    elif kind == "tool_call":
        yield _sse_event("tool_call", {"name": ..., "arguments": ...})
    elif kind == "tool_result":
        yield _sse_event("tool_result", {"name": ..., "summary": ...})
    elif kind == "thinking":
        yield _sse_event("thinking", {"iteration": ev.get("iteration", 1)})
    elif kind == "error":
        yield _sse_event("error", {"message": ev.get("message", "")})
except Exception as exc: # noqa: BLE001
    logger.exception("stream failed: %s", exc)
    yield _sse_event("error", {"message": str(exc)})
```

- **Các loại SSE event downstream:** `session`, `draft_ready`, `assistant_message`, `constraints_updated`, `tool_call`, `tool_result`, `thinking`, `error`, `done`.
- Xử lý lỗi: bất kỳ exception nào trong stream được nuốt và phát thành event `error` (stream không vỡ), đồng thời log qua `logger.exception`.
- **Persist sau khi stream xong** (gotcha quan trọng):

```
session_to_save = store.get(session_id, user_id) # đọc LẠI bản mới nhất
if final_draft is not None:
    session_to_save.draft = final_draft
session_to_save.constraints = final_constraints
session_to_save.messages.append(assistant_msg)
store.save(session_to_save)
```

Đọc lại session từ store (không dùng tham chiếu cũ để tránh ghi đè thay đổi đồng thời. Event `done` cuối cùng mang `draft`, `assistant_message`, `constraints`.

- Lời gọi ngoài: **LLM** + (qua agent) **HTTP** plan-service.

Helper SSE chính: `_consume_edit_stream` (chế độ chỉnh sửa) ([app/api/ai_plan.py:368](#))

Async generator cho edit mode: stream reasoning của agent + một `edit_proposal` (danh sách operations chờ duyệt), rồi persist proposal lên `session.pending_edits`.

- Khởi đầu yield `session` kèm `plan_id`. Lặp `async for ev in agent.edit_stream(plan_id=..., chat_history=..., latest_user_message=..., bearer_token=...)`. Khi `kind == "edit_proposal"`: `operations = ev.get("operations")` or `[]` và yield `edit_proposal`. Các event khác (`assistant_message`, `tool_call`, `tool_result`, `thinking`, `error`) như stream thường.
- Persist: đọc lại session, gán `session_to_save.pending_edits = operations` (proposal mới nhất, chờ "Áp dụng"), append assistant message. Event `done` mang `operations`, `assistant_message`, `plan_id`.
- Xử lý lỗi: giống `_consume_agent_stream` (nuốt exception → event `error`, log "edit stream failed").
- Lời gọi ngoài: **LLM** (`agent.edit_stream`) + (qua agent) **HTTP** plan-service để đọc plan đang chỉnh.

Endpoint: `POST /sessions/{session_id}/messages/stream` — `stream_message` ([app/api/ai_plan.py:428](#))

Biến thể SSE của `/messages`; tự chuyển sang edit mode khi phát hiện/đã có `plan_id`.

- Trình tự quyết định: (1) append tin user và lưu ngay; (2) **dò plan id** trong nội dung — nếu thấy id khác id hiện tại thì gate quyền: nếu `ForbiddenError`, trả về stream một-câu từ chối nhẹ nhàng (`exc.message`) thay vì 403 cứng; nếu OK thì bind `session.plan_id`; (3) nếu phiên có `plan_id` → `_consume_edit_stream`; (4) ngược lại, chỉ tạo mới khi `status == DRAFTING`, cập nhật constraints rồi → `_consume_agent_stream`.
- Gotcha: gate quyền edit chỉ trả 403 cứng ở `create_session`; ở luồng stream thì "from-message binding" trả lời từ chối qua assistant message để không làm gây UX chat.
- Lời gọi ngoài: HTTP `get_board` (khi dò ra id), LLM + HTTP (qua agent/extractor).

Endpoint: `POST /sessions/{session_id}/regenerate/stream` — `stream_regenerate` ([app/api/ai_plan.py:504](#))

Tạo lại bản nhập (SSE). Các guard tuần tự:

- `status == APPROVED` → `DomainError("Cannot regenerate after approval")`.

- `not constraints.is_minimally_complete()` → `DomainError` yêu cầu gửi tin planning trước.
- `session.draft is None` → `DomainError("Chưa có bản nháp để tạo lại...")` — regenerate là "thử phương án khác cho plan hiện tại", không phải "bắt đầu một plan".
- **Chống double-click**: nếu tin cuối là của USER và bắt đầu bằng "[regenerate]" → `DomainError("Đang tạo lại bản nháp, vui lòng đợi...")`, tránh hai luồng regenerate chạy song song làm xen kẽ/rớt lượt trong in-memory session.
- Sau đó append tin f"[regenerate] {instruction}" (instruction mặc định "Hãy thử một phương án khác.") và gọi `_consume_agent_stream(..., revision_hint=instruction, ...)`.

Endpoint: `POST /sessions/{session_id}/regenerate` — `regenerate (non-stream)` ([app/api/ai_plan.py:562](#))

Bản đồng bộ của regenerate. Guard ít hơn bản stream: chỉ chặn `APPROVED` và `is_minimally_complete()` (KHÔNG có guard `draft is None` và KHÔNG có guard chống double-click — đây là điểm khác biệt đáng lưu ý so với `stream_regenerate`). Gọi `agent.plan(..., revision_hint=instruction, prior_draft=session.draft, ...)`. Trả về `ApiResponse[SendMessageResponse]` với message "Draft regenerated".

Endpoint: `POST /sessions/{session_id}/approve` — `approve_session` ([app/api/ai_plan.py:614](#))

Biến draft thành plan thật trên plan-service.

- Guard: `status == APPROVED` → `DomainError("Plan đã được tạo trước đó...")`; `draft is None` → `DomainError("Chưa có bản nháp để duyệt...")`.
- Gọi `approval.approve(session.draft, user.raw_token)`. Xử lý lỗi: `UpstreamError` của plan-service được bọc lại với prefix tiếng Việt (giữ nguyên message gốc, không che thành 500 chung chung); `RuntimeError` cũng chuyển thành `UpstreamError`.
- Thành công: set `status = APPROVED`, `approved_plan_id = result["plan_id"]`, append assistant message thông báo f"Đã tạo plan #{...} từ bản nháp...", `store.save.result` từ `approval.approve` có shape `{"plan_id": int, "operations": [...]}`.
- Lời gọi ngoài: **HTTP** ghi xuống plan-service (qua `ApprovalService`).

Endpoint: `POST /sessions/{session_id}/apply-edits` — `apply_edits` ([app/api/ai_plan.py:655](#))

Ghi các operations đã đề xuất xuống plan đang chỉnh. Đây là **nơi duy nhất** edits được ghi thật, sau khi người dùng bấm "Áp dụng".

- Guard: `session.plan_id is None` → `DomainError("Phiên này không gắn với kế hoạch nào...")`; `not session.pending_edits` → `DomainError("Chưa có thay đổi nào để áp dụng...")`.
- Logic chống race quan trọng:

```
applied_edits = list(session.pending_edits)          # snapshot ĐÚNG set sắp apply
await _assert_can_edit(plan_client, user.raw_token, session.plan_id) # re-verify quyền tại thời điểm ghi
try:
    operations = parse_operations(session.pending_edits)
except Exception as exc:
    raise DomainError(f"Đề xuất chỉnh sửa không hợp lệ: {exc}") from exc
result = await edit_service.apply(session.plan_id, operations, user.raw_token)
...
session = store.get(session_id, user.id)            # đọc lại bản mới nhất
session.messages.append(assistant_msg)
if session.pending_edits == applied_edits:          # chỉ xóa ĐÚNG set vừa apply
    session.pending_edits = []
store.save(session)
```

- Các gotcha then chốt:
 1. **Re-verify quyền tại write-time** qua `_assert_can_edit` — role có thể bị thu hồi sau khi phiên mở; thất bại sớm với message rõ ràng thay vì "403 storm" do partial-apply.
 2. **Snapshot + so khớp `applied_edits`**: nếu một proposal mới hơn xuất hiện giữa chừng (do edit-stream đồng thời), KHÔNG xóa `pending_edits` để không nuốt mất proposal mới.
 3. **Đọc lại session** sau `await edit_service.apply(...)` vì luồng đồng thời có thể đã append message/proposal.
 4. `parse_operations(...)` (từ [app/agent/edits.py](#)) validate proposal; lỗi → `DomainError` (HTTP 4xx) chứ không phải 500.
- Summary: f"Đã áp dụng {result['applied']}/{result['total']} thay đổi..."; nếu có op `not r.get("ok")` thì thêm cảnh báo số op lỗi. `result` shape: `{"applied": int, "total": int, "results": [{"ok": bool, ...}]}`.
- Trả về `ApiResponse[ApplyEditsResult]`. Lời gọi ngoài: **HTTP** `get_board` (gate) + ghi edits xuống plan-service (qua `EditService`).

Tổng hợp mô hình lỗi & các loại exception

- `DomainError` ([app/core/errors.py](#)) — lỗi nghiệp vụ phía client (sai trạng thái, thiếu draft, double-click...) → HTTP 4xx.
- `ForbiddenError` — gate quyền 403; ở luồng stream được hạ cấp thành assistant message từ chối.
- `UpstreamError` — lỗi từ plan-service; được bọc lại với thông điệp tiếng Việt và giữ message gốc.
- Trong các SSE consumer, mọi `Exception` đều bị nuốt và phát thành event `error` (stream vẫn kết thúc bằng event `done` ở phần persist sau khối try/except).

Bảng nhanh các SSE event theo luồng

- `_consume_agent_stream` (tạo/regenerate): `session` → (lập) `thinking` / `tool_call` / `tool_result` / `constraints_updated` / `assistant_message` / `draft_ready` / `error` → `done` (kèm `draft`, `assistant_message`, `constraints`).
- `_consume_edit_stream` (chỉnh sửa): `session` (kèm `plan_id`) → (lập) `thinking` / `tool_call` / `tool_result` / `assistant_message` / `edit_proposal` / `error` → `done` (kèm `operations`, `assistant_message`, `plan_id`).
- `_one_shot_assistant_stream` (từ chối quyền): `session` → `assistant_message` → `done` (kèm `assistant_message`, `plan_id`).

Agent Orchestrator — vòng lặp agent, hai SYSTEM PROMPT (plan/edit), primer

Cụm module này là "bộ não điều phối" của **ai-plan-service**: nó nhận lịch sử chat + trạng thái ràng buộc (constraints), xây dựng prompt hệ thống và context primer, rồi chạy một vòng lặp gọi LLM kèm tool-use cho đến khi agent hoặc tạo ra bản nháp lịch trình (`finalize_draft`), hoặc đề xuất chỉnh sửa kế hoạch hiện có (`propose_plan_edits`), hoặc kết thúc lượt để trò chuyện/hỏi lại. Toàn bộ cụm nằm trong `app/agent/orchestrator.py`.

`app/agent/orchestrator.py`

Trách nhiệm tổng quát: định nghĩa lớp `AgentOrchestrator` và toàn bộ logic xoay quanh nó — hai chế độ vòng lặp agent (chế độ **plan** dựng nháp mới và chế độ **edit** sửa kế hoạch sẵn có), hai SYSTEM PROMPT khổng lồ (`_SYSTEM_PROMPT`, `_EDIT_SYSTEM_PROMPT`), các hàm dựng "context primer" và message list theo định dạng OpenAI, các hàm fallback khi không có LLM hoặc khi LLM lỗi, và các tiện ích nhỏ (tóm tắt tool result, lọc câu fallback "đóng hộp" do chính agent từng phát ra, tránh việc replay chúng vào history và khiến model học cách lặp lại).

Hàng số & cấu hình module-level

- `logger = logging.getLogger("ai_plan.agent")` — `app/agent/orchestrator.py:38`. Logger dùng chung cho mọi log của vòng lặp.
- `MAX_ITERATIONS = 10` — `app/agent/orchestrator.py:40`. Số vòng lặp tối đa cho cả hai chế độ (plan và edit). Khi cạn vòng lặp mà agent chưa `finalize_draft/propose_plan_edits`, orchestrator rơi vào nhánh fallback.
- `_SYSTEM_PROMPT` — `app/agent/orchestrator.py:43`. SYSTEM PROMPT cho chế độ **plan** (dựng nháp mới).
- `_EDIT_SYSTEM_PROMPT` — `app/agent/orchestrator.py:254`. SYSTEM PROMPT cho chế độ **edit** (sửa kế hoạch hiện có).
- `_CANNED_FALLBACK_MARKERS` — `app/agent/orchestrator.py:926`. Tuple các chuỗi (lowercase) dùng để nhận diện và LOẠI BỎ các câu clarification "đóng hộp" do chính agent từng phát ra, tránh việc replay chúng vào history và khiến model học cách lặp lại.

Các import đáng chú ý (`app/agent/orchestrator.py:23`) cho thấy quan hệ phụ thuộc ra ngoài: từ `app.agent.edits` (lấy `EditValidationError`, `board_summary`, `parse_operations`), từ `app.agent.tools` (lấy `DraftValidationError`, `ToolDispatcher`, `apply_set_constraints`, `edit_tool_definitions`, `parse_finalize_draft`, `tool_definitions`), các client `CatalogClient` / `PlanClient`, `LLMClient`, các model `ChatMessage`, `ChatRole`, `Constraints`, `PlanDraft`, và `DraftComposer`.

Hai SYSTEM PROMPT (plan vs edit) — điểm khác biệt then chốt

Cả hai đều là chuỗi tiếng Việt + tiếng Anh dài, nhúng cứng (hard-coded) trong file, viết hoàn toàn bằng văn bản (không phải template f-string — chúng KHÔNG chèn ngày/biến; ngày tháng và constraints được đưa vào qua *primer* trong message list, không phải qua system prompt).

`_SYSTEM_PROMPT` (chế độ plan, :43--251) hướng dẫn agent:

- Giao tiếp bằng tiếng Việt, thân thiện, hỏi MỘT câu mỗi lần khi thiếu thông tin.
- Bộ tool được phép gọi: `set_constraints`, `search_hotels`, `search_restaurants`, `search_places`, `route_legs`, `web_search`, `view_my_plans`, `search_plans`, và `finalize_draft` (terminal).
- Phân biệt rõ `search_plans` (tìm lịch trình CÓ SẴN, trả về link `mravel_url`) với việc dựng nháp mới.
- Thứ tự nguồn dữ liệu: catalog trước, web sau, không bịa địa điểm/địa chỉ giả.
- Quy tắc "freshness" cho thông tin nhạy cảm theo thời gian: BẮT BUỘC `web_search` với năm hiện tại lấy từ primer, trích `url` thật để cite.
- Theo dõi state qua "Known constraints so far" trong primer; KHÔNG hỏi lại fact đã có.
- Quy tắc TIMING quan trọng: hỏi "đi mấy ngày" (`num_days`), KHÔNG hỏi ngày dương lịch; mặc định `start_date = today`, `end_date = today + (num_days - 1)`.
- Yêu cầu DENSITY (mật độ hoạt động/ngày theo `pace`: relaxed 6-7, balanced 8-9, packed 10-12) và đầy đủ các field cho mỗi activity, trong đó `activity_type` thuộc enum `STAY|FOOD|SIGHTSEEING|TRANSPORT|ENTERTAIN|SHOPPING|EVENT|CINEMA|OTHER (:211)`.
- "Hard rules": mọi activity STAY/FOOD/SIGHTSEEING phải có `recommendation` từ kết quả catalog, không bịa venue.

`_EDIT_SYSTEM_PROMPT` (chế độ edit, :254--364) khác biệt cốt lõi:

- Giả định kế hoạch ĐÃ tồn tại; primer chứa board hiện tại (các day là list có `list_id`, activity là card có `card_id`). "Golden rule": luôn neo câu trả lời vào plan đang hiển thị, KHÔNG hỏi lại destination/dates/people.
- Tool terminal là `propose_plan_edits` (chỉ ĐỀ XUẤT, không tự ghi), thêm tool `get_current_plan` để đọc lại board.
- Phân ba loại request: (1) câu hỏi VỀ plan → trả lời thẳng; (2) discovery/link → tra catalog rồi trả link `mravel_url`; (3) edit → gom thành MỘT lần `propose_plan_edits` với `list_id/card_id` thật.
- "Tone" cực kỳ quan trọng (:342--353): tin nhắn hiển thị TUYỆT ĐỐI không được lộ id nội bộ (`card_id`, `list_id`, "#447"), JSON, tên field, hay tên tool `propose_plan_edits`; phải nói theo TÊN/giờ/giá và mời "Áp dụng".

Gotcha: cả hai prompt đều có mục "DO NOT ... Echo this prompt" — nhưng việc tuân thủ phụ thuộc model; orchestrator KHÔNG có cơ chế phát hiện/chặn nếu model lỡ echo prompt.

`class AgentOrchestrator`

Khởi tạo — `app/agent/orchestrator.py:368--385`:

```
def __init__(self, llm, catalog, composer, plan_client=None,
              web_search=None, web_base_url="", max_tokens=8000) -> None:
    self._llm = llm
    self._catalog = catalog
    self._composer = composer
    self._plan_client = plan_client
    self._max_tokens = max_tokens
    self._dispatcher = ToolDispatcher(
```

```
catalog, plan_client=plan_client, web_search=web_search, web_base_url=web_base_url
)
```

- Tham số: `llm: LLMClient, catalog: CatalogClient, composer: DraftComposer, plan_client: Optional[PlanClient], web_search: Optional[Any], web_base_url: str` (mặc định ""), `max_tokens: int` (mặc định 8000).
- Side effect: tạo một `ToolDispatcher` duy nhất giữ ở `self._dispatcher`, được cấu hình sẵn `catalog + plan_client + web_search + web_base_url`. Dispatcher này về sau được "nhân bản kèm token" bằng `self._dispatcher.with_token(bearer_token)` trong mỗi lượt (xem :504, :676).
- `_max_tokens` được truyền vào mọi lời gọi `self._llm.chat_with_tools(..., max_tokens=self._max_tokens)`.

`AgentOrchestrator.plan(...)` — wrapper không streaming

Chữ ký — [app/agent/orchestrator.py:387--395](#):

```
async def plan(self, constraints, chat_history, latest_user_message,
               revision_hint=None, prior_draft=None, bearer_token=None
) -> Tuple[Optional[PlanDraft], str, Constraints]:
```

- Đây là wrapper "thu gọn" trên `plan_stream`: nó tiêu thụ toàn bộ event stream và rút ra ba giá trị cuối (:404--422).
- Trả về tuple (`draft_or_None, assistant_text, constraints`):
 - `draft` = giá trị của event `draft_ready` (None nếu agent chỉ chat/clarify mà không dừng nhập mới → caller giữ nguyên prior draft).
 - `narrative` = `text` của event `assistant_message` cuối cùng (giữ giá trị cũ nếu event sau rỗng, qua `event.get("text", "")` or `narrative`).
 - `final_constraints` = `constraints` của event `constraints_updated` cuối cùng (mặc định bằng `constraints` đầu vào). Đây là trạng thái trip "đã được agent tinh chỉnh" — agent là single source of truth.
- Không có side effect ngoài (HTTP/Mongo/LLM) trực tiếp; mọi I/O xảy ra bên trong `plan_stream`.

`AgentOrchestrator.plan_stream(...)` — endpoint streaming chế độ plan

Chữ ký — [app/agent/orchestrator.py:424--432](#); vocabulary event tài liệu hóa ở docstring :433--440:

- `{"event": "thinking", "iteration": int}`
- `{"event": "tool_call", "name": str, "arguments": dict}`
- `{"event": "tool_result", "name": str, "summary": str}`
- `{"event": "draft_ready", "draft": PlanDraft}`
- `{"event": "assistant_message", "text": str}`
- `{"event": "constraints_updated", "constraints": Constraints}` (có thực tế nhưng không liệt kê trong docstring)
- `{"event": "error", "message": str}`

Logic phân nhánh (:442--463):

- Edge case 1 — LLM không hỗ trợ tool use (`self._llm.supports_tool_use()` False, thường là stub khi thiếu `LLM_API_KEY`): đi đường `_stub_path` (deterministic), không bao giờ chạy vòng lặp agent.
- Edge case 2 — vòng lặp agent ném exception bất kỳ: bắt rộng (`BLE001`), log cảnh báo kèm traceback, phát event `error`, RỒI phát chuỗi event từ `_deterministic_fallback(constraints)`. Lưu ý fallback ở đây dùng `constraints` GỐC (đầu vào), không phải `working` đã tinh chỉnh — vì biến `working` chỉ tồn tại trong scope của `_run_agent_loop_stream` và không truyền ra được khi exception.

`AgentOrchestrator._deterministic_fallback(constraints)` — lưới an toàn

Chữ ký — [app/agent/orchestrator.py:465--467](#). Logic — :475--483:

- Triết lý: không hỏi lại mù quáng — nhìn vào những gì ĐÃ biết. Nếu trip đủ tối thiểu (`constraints.is_minimally_complete()`) thì dừng draft thật qua `DraftComposer.compose` thay vì hỏi lại; nếu chưa đủ thì chỉ hỏi đúng field còn thiếu qua `_clarification_text`.
- Lời gọi ra ngoài: `self._composer.compose(constraints)` (DraftComposer; bên trong có thể gọi `catalog` → HTTP). Có thể ném exception (catalog rỗng) → bắt và rơi xuống nhánh `clarification`.
- Giá trị yield: hoặc cặp `draft_ready + assistant_message` (narrative mặc định), hoặc một `assistant_message` clarification.

`AgentOrchestrator._stub_path(constraints)` — đường stub khi không có LLM

Chữ ký — [app/agent/orchestrator.py:485](#). Logic :486--489:

- Nếu `not constraints.is_minimally_complete()` → trả (`None, _clarification_text(constraints)`).
- Ngược lại → `draft = await self._composer.compose(constraints)`, trả (`draft, _default_narrative(draft)`).
- Trả về tuple (`Optional[PlanDraft], str`). Lưu ý: KHÔNG bọc try/except quanh `compose` ở đây (khác với `_deterministic_fallback`) — nếu `compose` lỗi, exception lan ra `plan_stream` và sẽ bị nhánh except chung ở :459 bắt lại (rồi gọi `_deterministic_fallback`, lần này CÓ try/except).

`AgentOrchestrator._run_agent_loop_stream(...)` — lõi vòng lặp chế độ plan

Chữ ký — [app/agent/orchestrator.py:491--499](#). Đây là tim của chế độ plan.

Chuẩn bị (:500--508):

```
tools = tool_definitions()
messages = _build_messages(chat_history, constraints, latest_user_message, revision_hint, prior_draft)
dispatcher = self._dispatcher.with_token(bearer_token)
narrative = ""
working = constraints # bản sao trạng thái trip; agent có thể tinh chỉnh qua set_constraints
```

Vòng lặp `for iteration in range(MAX_ITERATIONS)` (:510):

1. Yield `{"event": "thinking", "iteration": iteration + 1}` (:511).
2. Gọi LLM: `turn = await self._llm.chat_with_tools(_SYSTEM_PROMPT, messages, tools, max_tokens=self._max_tokens)` (:512). Đây là lời gọi LLM duy nhất mỗi vòng.
3. Nếu `turn.stop_reason == "error"` → `raise RuntimeError(turn.error or "LLM error")` (:516) — sẽ bị bắt ở `plan_stream` và chuyển sang deterministic fallback.
4. Nếu có `turn.raw_assistant_message` → append vào `messages` để giữ lịch sử cho vòng sau (:519).
5. Nếu `turn.stop_reason != "tool_use"` (model kết thúc lượt) (:522--535):
 - `narrative = turn.text or narrative`. Log agent end_turn after N iterations, `draft=kept-prior|none`.
 - Nếu có `narrative` → `yield assistant_message`. Nếu `narrative` RỖNG (model im lặng) → KHÔNG hỏi lại mù quáng, gọi `_deterministic_fallback(working)` (chú ý: dùng `working`, không phải `constraints`). Rồi `return`.

Xử lý tool calls trong lượt (:537--591) — duyệt `turn.tool_uses`, mỗi tu có `.name`, `.arguments`, `.id`:

- `set_constraints` (:542--556): `working = apply_set_constraints(tu.arguments, working)`; append tool result `{"ok": True, "constraints": json.loads(working.model_dump_json())}`; yield `constraints_updated` (kèm `working`) và `tool_result` với summary `constraints updated (dest=...)`. Đây là cơ chế agent ghi nhớ fact mới.
- `finalize_draft` (:558--586) — terminal cho chế độ plan: nếu `parse_finalize_draft` ném `DraftValidationError` → trả tool result LỖI (`is_error=True`) cho model và `continue` (model có cơ hội sửa ở vòng sau, KHÔNG kết thúc). Nếu hợp lệ → đồng bộ ngược `destination/start_date/end_date/travelers` từ draft về `working` (để regenerate không bị chặn vì "incomplete"), yield `constraints_updated`, append tool result `{"ok": True, "draft_id": finalized.draft_id}`, yield `tool_result` summary `draft saved (N days)`. Biến `finalized` được giữ để xử lý SAU vòng tool (không yield `draft_ready` ngay).
- Các tool catalog/web khác (:588--591): `result = await dispatcher.run(tu.name, tu.arguments)` (đi qua `ToolDispatcher` → HTTP tới catalog/plan service hoặc `web_search`), append tool result, yield `tool_result` với summary từ `_tool_summary`.

Hoàn tất draft sau vòng tool (:593--614):

- Nếu trong cùng lượt model đã viết text (`turn.text`) thì dùng làm `narrative`; nếu KHÔNG, orchestrator gọi LLM thêm MỘT lần nữa (với `tools=[]`, tức không cho gọi tool) chỉ để xin 1-2 câu tóm tắt tiếng Việt. Đây là một lời gọi LLM phụ — gotcha về chi phí/độ trễ. Nếu vẫn rỗng, fallback `_default_narrative(finalized)`.
- Sau đó yield `draft_ready + assistant_message` và `return`. Vậy `finalize_draft` luôn KẾT THÚC lượt agent.

Cạn vòng lặp (:616--621): nếu hết `MAX_ITERATIONS` mà không `finalize` → log, rồi nếu có `narrative` thì yield nó, ngược lại gọi `_deterministic_fallback(working)`.

Gotcha: nếu trong một lượt model gọi NHIỀU tool kèm cả `finalize_draft`, các tool khác vẫn được chạy trước, và chỉ `finalized` (lần `finalize_draft` cuối được parse thành công) mới được dùng — vì `finalized` bị ghi đè qua mỗi lần gặp `finalize_draft`.

`AgentOrchestrator.edit_stream(...)` — vòng lặp chế độ edit

Chữ ký — `app/agent/orchestrator.py:623--630` (lưu ý: chỉ keyword-only args, có dấu *):

```
async def edit_stream(self, *, plan_id: int, chat_history: List[ChatMessage],
                      latest_user_message: str, bearer_token: str
) -> AsyncIterator[Dict[str, Any]]:
```

Khác `plan_stream` ở chỗ `bearer_token` BẮT BUỘC (không Optional) và cần `plan_id`. Vocabulary event giống plan nhưng thêm `edit_proposal`.

Trình tự:

1. **Đọc board** (:635--644): `board = await self._plan_client.get_board(bearer_token, plan_id)` — lời gọi HTTP tới plan-service. Nếu lỗi → yield `error` + một `assistant_message` thân thiện ("Minh chưa đọc được nội dung kế hoạch này...") và `return`.
2. **Cổng quyền** (permission gate) (:646--660): nếu `myRole` không phải `OWNER/EDITOR` (ví dụ `VIEWER`), agent KHÔNG được phép dựng đề xuất sửa (vì không bao giờ apply được). Đây là defense-in-depth: quyền còn được enforce ở lúc tạo session / apply và authoritatively ở plan-service. Thông điệp tùy biến theo `VIEWER` vs các vai khác.
3. `summary = board_summary(board)` (:662) — chuyển board JSON thành text tóm tắt cho primer.
4. **Edge case không có LLM** (:664--672): nếu `not self._llm.supports_tool_use()` → trả luôn text gồm `summary` + lời nhắc cấu hình `LLM_API_KEY`, không chạy vòng lặp.
5. **Vòng lặp** (:674--754): `tools = edit_tool_definitions()`, `messages = _build_edit_messages(summary, chat_history, latest_user_message)`, `dispatcher = self._dispatcher.with_token(bearer_token)`. Trong `for iteration in range(MAX_ITERATIONS)`:
 - Yield `thinking`, gọi `turn = await self._llm.chat_with_tools(_EDIT_SYSTEM_PROMPT, messages, tools, max_tokens=self._max_tokens)` (:681) — chú ý dùng `_EDIT_SYSTEM_PROMPT`.
 - Nếu `turn.stop_reason == "error"` (:685--694): KHÁC với plan — KHÔNG raise. Thay vào đó yield `error` rồi yield `assistant_message` chứa thông điệp lỗi + `summary` board hiện tại (để user vẫn xem được plan), rồi `return`.
 - Nếu có `turn.raw_assistant_message` → append (:695).

- Nếu `turn.stop_reason != "tool_use"` (:698--704): `narrative = turn.text or narrative`; `yield assistant_message` (fallback text "Bạn muốn mình chỉnh sửa gì trong kế hoạch này?"); `return`.
- Duyệt `tool_uses` (:707--740):
 - `get_current_plan` (:711--719): đọc lại board (`get_board` → HTTP), nếu lỗi gộp text (`lỗi đọc plan: ...`); append tool result (`"board": text`); summary đã đọc plan.
 - `propose_plan_edits` (:721--735) — terminal: `parse_operations(..., board=board)` validate operations dựa trên board GỐC (không phải board fresh từ `get_current_plan`). Nếu `EditValidationError` → trả lỗi cho model, đặt `proposed = None`, `continue` (model có thể sửa ở vòng sau). Nếu ok → trả `{"ok": True, "count": N}`.
 - **Tool khác** (:737--740): `dispatcher.run` → HTTP catalog/web.
- Hoàn tất đề xuất (:742--749): nếu `proposed is not None` → `narrative = turn.text or _default_edit_narrative(proposed)`; `yield edit_proposal` với `operations = [op.model_dump() for op in proposed]`; `yield assistant_message`; `return`. (Khác plan: KHÔNG có lời gọi LLM phụ để xin narrative — nếu model không viết text thì dùng `_default_edit_narrative`.)

6. **Cạn vòng lặp** (:751--754): `yield assistant_message = narrative` hoặc "Mình chưa rõ thay đổi bạn muốn — bạn mô tả cụ thể hơn nhé?".

Gotcha: nếu `proposed` bị set `None` do `EditValidationError` ở lần `propose_plan_edits` cuối nhưng trước đó từng `propose_plan_edits` thành công, kết quả `proposed` sẽ là `None` (lần sau ghi đề lần trước) → coi như không có đề xuất ở vòng đó, vòng lặp tiếp tục.

Các hàm dựng message & primer (module-level)

`_build_edit_messages(board_text, chat_history, latest_user_message)` — :757--787

Trả về `List[Dict[str, Any]]` theo định dạng OpenAI. Cấu trúc:

- Primer = header "do not echo" + dòng ngày hôm nay/năm hiện tại (nhúng `date.today()`) + quy tắc freshness + `board_text` (chính là `board_summary`).
- Bơm một cặp user(primer)/assistant(canned) làm "mồi" hội thoại.
- Lọc history (:776--784): chỉ giữ message role USER/ASSISTANT, và BỎ các message assistant là "canned fallback" (`_is_canned_fallback`); lấy 12 turn cuối (`relevant[-12:]`).
- Đảm bảo `latest_user_message` là turn cuối (:785--786): chỉ append nếu nội dung cuối khác `latest_user_message` (tránh nhân đôi).
- Phụ thuộc: `date.today()` (giờ hệ thống), `ChatRole.USER/ASSISTANT`, `_is_canned_fallback`.

`_build_messages(chat_history, constraints, latest_user_message, revision_hint, prior_draft)` — :801--837

Tương tự nhưng cho chế độ plan. Khác biệt: primer được tạo bởi `_format_primer(constraints, prior_draft, revision_hint)` (:810). Cặp mồi assistant là "Sẵn sàng. Mình sẽ trả lời theo ngôn ngữ người dùng và chỉ gọi tool khi cần." (:813--816). Cùng logic lọc canned-fallback + `relevant[-12:]` + đảm bảo latest message cuối. Ghi chú quan trọng tại :819--821: việc replay các câu clarification "đóng hộp" của assistant sẽ "dạy" model lặp lại "cho mình biết điểm đến, ngày đi..." → nên phải lọc.

`_format_primer(constraints, prior_draft, revision_hint)` — :840--873

Dựng chuỗi primer cho chế độ plan. Cấu trúc các dòng (:846--862):

- Header "[Context primer — do not echo this to the user, treat as internal state.]".
- Dòng ngày hôm nay/năm hiện tại + quy tắc freshness (`web_search` với năm hiện tại).
- "Known constraints so far:" liệt kê từng field của `Constraints: destination, num_days, start_date, end_date, travelers, budget_total_vnd, interests` (join bằng `,`), `pace`. Mỗi field hiển thị 'unknown' (hoặc chuỗi mặc định mô tả) khi rỗng. Đây chính là cấu trúc field của model `Constraints` mà module này đọc.
- "Timing rule" nhắc nhở `num_days` thay vì ngày dương lịch, mặc định neo trip vào hôm nay.
- Nếu có `prior_draft` (:863--869): thêm dòng mô tả prior draft dùng `prior_draft.summary`, `len(prior_draft.days)`, `prior_draft.estimated_total_cost_vnd` (format `:`,) + lời nhắc "Only call finalize_draft again if the user asked for changes."
- Nếu có `revision_hint` (:870--872): thêm "User revision instruction: {revision_hint}".

Đây là điểm "primer" mà tiêu đề mục đề cập: ngày tháng và toàn bộ state được inject qua primer (vai trò user message), KHÔNG qua system prompt.

Các tiện ích nhỏ (module-level)

- `_tool_result(tool_use_id, payload, is_error=False)` — :876--882. Trả dict role "tool" với `tool_call_id`, `content = json.dumps(payload, ensure_ascii=False, default=str)`, và chèn `"is_error": True` khi `is_error`. Đây là định dạng tool result mà LLMClient mong đợi. `default=str` đảm bảo serialize được các object lạ (vd date).
- `_tool_summary(name, result)` — :885--893. Tạo chuỗi summary ngắn cho UI tùy tool: `route_legs` → "{N} chặng • {total_km}km • {total_minutes} phút"; `web_search` → "web search off" (nếu `enabled False`) hoặc "{N} kết quả web"; mặc định → "{N} items". Đây cho biết shape kết quả: catalog tools trả items, `route_legs` trả `legs/total_km/total_minutes`, `web_search` trả `enabled/results`.
- `_short(args)` — :896--900. `json.dumps(...)[ :200 ]` để log gọn arguments; fallback `str(args)[ :200 ]` nếu lỗi serialize.
- `_default_narrative(draft)` — :903--908. Câu tiếng Việt mặc định khi model không viết narrative cho draft: dùng `len(draft.days)`, `draft.destination`, `draft.travelers`, `draft.estimated_total_cost_vnd` (:,).
- `_default_edit_narrative(operations)` — :790--798. Câu mặc định cho edit proposal: "Mình đề xuất {N} thay đổi..." + tối đa 8 dòng • `{op.summary}` + "Bạn bấm **Áp dụng**..."
- `_clarification_text(constraints)` — :911--921. Sinh câu hỏi clarification tối thiểu, phụ thuộc field còn thiếu: thiếu `destination` → "Bạn muốn đi đâu nhỉ?"; có `destination` nhưng thiếu cả `num_days` lẫn (`start_date+end_date`) → "Bạn dự định đi mấy ngày?"; còn lại → câu chung.
- `_is_canned_fallback(text)` — :936--938. Trả True nếu `text` (lowercase) chứa BẤT KỲ marker nào trong `_CANNED_FALLBACK_MARKERS` (:926--933). Đây là cơ chế chống "vòng lặp lặp câu hỏi".

Cấu trúc dữ liệu / field / enum / env mà module dùng tới

- **Model Constraints**: các field `destination, num_days, start_date, end_date, travelers, budget_total_vnd, interests` (list[str]), `pace`; method `is_minimally_complete()`, `model_dump_json()`, `model_copy(update={...})`.

- **Model PlanDraft**: field `days` (list), `destination`, `travelers`, `start_date`, `end_date`, `estimated_total_cost_vnd`, `summary`, `draft_id`.
- **Model ChatMessage** / **enum ChatRole**: `ChatRole.USER`, `ChatRole.ASSISTANT`; `m.role`, `m.content`.
- **Enum activity_type** (chỉ trong prompt): `STAY|FOOD|SIGHTSEEING|TRANSPORT|ENTERTAIN|SHOPPING|EVENT|CINEMA|OTHER`.
- **Board fields** (chế độ edit): `board.get("myRole")` với giá trị `OWNER/EDITOR/VIEWER`; cấu trúc list/card có `list_id/card_id`.
- **Tool result shapes**: `catalog` → `items` (kèm `mravel_url`, `recommendation`, `no_results_for_location`, `available_on_mravel`, `city`); `route_legs` → `legs/total_km/total_minutes`; `web_search` → `enabled/results/url`.
- **Biến môi trường (env)**: file KHÔNG đọc env trực tiếp. Tham chiếu gián tiếp `LLM_API_KEY` chỉ xuất hiện trong text gửi user (:670) và phản ánh qua `self._llm.supports_tool_use().web_base_url` và `bearer_token` được truyền vào, không phải env đọc tại file này.
- **Hàng số**: `MAX_ITERATIONS = 10`; `max_tokens` mặc định `8000` (constructor).

Lỗi gọi ra ngoài (HTTP / LLM) tổng hợp

- **LLM**: `self._llm.supports_tool_use()`, `self._llm.chat_with_tools(system_prompt, messages, tools, max_tokens=...)`. Mỗi vòng lặp gọi `chat_with_tools` MỘT lần; chế độ plan có thể gọi THÊM một lần nữa để xin narrative đóng lượt khi `finalize_draft` không kèm text (:596–609).
- **Plan-service (HTTP)**: `self._plan_client.get_board(bearer_token, plan_id)` ở `edit_stream` (đọc board ban đầu :636 và khi `get_current_plan` :713).
- **Catalog / web_search (HTTP)**: gián tiếp qua `dispatcher.run(tu.name, tu.arguments)` (:588, :737) — dispatcher là `ToolDispatcher` đã `with_token(bearer_token)`.
- **DraftComposer**: `self._composer.compose(constraints) (_stub_path, _deterministic_fallback)` — bên trong có thể chạm catalog (HTTP).

Tổng hợp xử lý lỗi / fallback / edge case

- LLM stub (không tool-use): plan → `_stub_path`; edit → trả summary + nhắc `LLM_API_KEY`.
- `turn.stop_reason == "error"`: plan → `raise RuntimeError` (bị `plan_stream` bắt → deterministic fallback); edit → KHÔNG raise, trả lỗi thân thiện + summary board.
- `DraftValidationError` (finalize) / `EditValidationError` (propose): trả tool result lỗi cho model + `continue` (model có thể sửa), KHÔNG kết thúc lượt.
- Model im lặng (`stop_reason != tool_use` nhưng text rỗng): plan → `_deterministic_fallback(working)`; edit → câu hỏi "Bạn muốn mình chỉnh sửa gì...".
- Cận `MAX_ITERATIONS`: plan → `_deterministic_fallback(working)`; edit → câu "Mình chưa rõ thay đổi bạn muốn...".
- `get_board` lỗi (edit): event `error` + thông điệp thân thiện, `return`.
- Permission gate (edit): chặn nếu `myRole` không phải `OWNER/EDITOR`, không bao giờ chạy vòng lặp.
- Gotcha quan trọng: nhánh except chung trong `plan_stream` (:459) dùng `constraints` GỐC cho fallback, mất các tình hình `working`; ngược lại các fallback NỘI BỘ trong `_run_agent_loop_stream` dùng `working` (đã tinh chỉnh). `_build_messages/_build_edit_messages` chống nhân đôi `latest_user_message` và lọc `canned-fallback` để chống vòng lặp hỏi lại.

Tool definitions & ToolDispatcher (catalog/web/route + mravel_url + fallback)

Cụm module này tập trung trong `app/agent/tools.py`. Đây là "tool catalog" — tập hợp định nghĩa các công cụ (tools) mà LLM agent được phép gọi, cộng với lớp `ToolDispatcher` thực thi các tool đó phía server, và các hàm helper biến đổi dữ liệu (`constraints`, `draft`, `mravel_url`). File mở đầu bằng docstring nêu rõ triết lý: mỗi tool gồm một JSON-schema mà model nhìn thấy và một executor async chạy server-side; tool kết thúc (`terminal`) là `finalize_draft` — khi model gọi nó, orchestrator parse payload thành `PlanDraft` và dừng vòng lặp (`app/agent/tools.py:1–app/agent/tools.py:9`).

app/agent/tools.py

Trách nhiệm tổng quát của file:

1. Khai báo schema các tool cho hai chế độ: tạo plan mới (`tool_definitions`) và chỉnh sửa plan có sẵn (`edit_tool_definitions`).
2. Thực thi các tool read-only thay mặt LLM (`ToolDispatcher`), gọi ra `CatalogClient`, `PlanClient`, và web search provider.
3. Xây link tới frontend SPA (`catalog_web_url`, `plan_web_url`) và tóm tắt (summarise) các bản ghi catalog/plan thành payload gọn để tiết kiệm context của model.
4. Hợp nhất payload `set_constraints` vào state (`apply_set_constraints`) và parse payload `finalize_draft` thành `PlanDraft` (`parse_finalize_draft`).

Import & phụ thuộc (app/agent/tools.py:11–app/agent/tools.py:28)

- `ToolDefinition` là cấu trúc trung tính (`name`, `description`, `input_schema`) mà tầng LLM tự dịch sang format của provider.
- `UpstreamError` (từ `app/core/errors.py`) là exception được bắt ở nhiều chỗ trong dispatcher để fallback "mềm" thay vì làm hỏng cả vòng lặp agent.
- `PlanActivityType` là enum dùng cả trong schema (`finalize_draft`) lẫn lúc parse.

tool_definitions() -> List[ToolDefinition] (app/agent/tools.py:31)

Hàm thuần dữ liệu, không có side effect, trả về danh sách 9 `ToolDefinition` mà agent dùng khi TẠO plan mới. Các tool và những điểm đáng chú ý của schema:

- `set_constraints` (`app/agent/tools.py:33`): ghi/cập nhật "trip facts". Schema không có `required` — model chỉ điền field nào thực sự biết. Các field: `destination` (string), `num_days` (integer, minimum: 1), `start_date/end_date` (string YYYY-MM-DD, chỉ khi user nói ngày lịch cụ thể), `travelers` (integer ≥ 1), `budget_total_vnd` (integer ≥ 0), `interests` (array string), `pace` (enum `relaxed/balanced/packed`). Description nhấn mạnh: ưu tiên `num_days` hơn `start/end_date`, và đây là "single source of truth" để hệ thống không hỏi lại điều đã nắm.
- `search_hotels` (`app/agent/tools.py:76`): `location` bắt buộc (`required`); `max_results` (1–20, default 5); `max_price_vnd`; `min_star_rating` (1–5).
- `search_restaurants` (`app/agent/tools.py:97`): `location` bắt buộc; `cuisine` (free text); `max_price_per_person_vnd`; `max_results` (1–20, default 10).
- `search_places` (`app/agent/tools.py:115`): KHÔNG có field bắt buộc nhưng description yêu cầu truyền ít nhất một trong `destination` / `query`. Cảnh báo quan trọng: place search **diacritic-sensitive** — phải truyền tiếng Việt CÓ dấu (vd 'Đà Nẵng'). `max_results` (1–20, default 10).
- `route_legs` (`app/agent/tools.py:146`): `stops` bắt buộc — mảng object {`name` (`required`), `latitude` (`number`), `longitude` (`number`)}. Tính khoảng cách/thời gian/phương tiện thực giữa các điểm liên tiếp.
- `web_search` (`app/agent/tools.py:176`): `query` bắt buộc (description yêu cầu KÈM năm hiện tại để tránh số liệu cũ); `max_results` (1–8, default 5).
- `view_my_plans` (`app/agent/tools.py:203`): read-only, liệt kê plan đã duyệt của user. `max_results` (1–20, default 5), `required`: [].

- **search_plans** ([app/agent/tools.py:219](#)): tìm itinerary user có thể truy cập (của mình + public/friends/shared). Field: `query`, `destinations` (array string), `budget_min_vnd`, `budget_max_vnd`, `days_min`, `days_max`, `max_results` (1–20, default 8). Description phân biệt rõ với `search_places/search_hotels` (tìm cả lịch trình, không phải venue lẻ).
- **finalize_draft** ([app/agent/tools.py:264](#)): TERMINAL tool, gọi đúng một lần ở cuối. Schema lồng sâu:
 - Top-level required: `["destination", "start_date", "end_date", "days"]` ([app/agent/tools.py:359](#)).
 - `days[]`: mỗi day required `["day_index", "day_date", "title", "activities"]`.
 - `activities[]`: required `["title", "activity_type"]`; `activity_type` là enum sinh động từ `[t.value for t in PlanActivityType]` ([app/agent/tools.py:301](#)).
 - `recommendation` (object): `catalog_kind` enum `["PLACE", "RESTAURANT", "HOTEL"]`, `catalog_id` (mô tả: "Mongo ObjectId — copy verbatim"), `catalog_slug`, `name`, `latitude`, `longitude`, `cover_image_url`, `avg_rating`, `estimated_cost_vnd`.

Gotcha: `activity_type` enum được build runtime từ enum Python, nên thêm giá trị mới vào `PlanActivityType` sẽ tự động phản ánh vào schema mà model nhìn thấy.

strip_accents(text: str) -> str ([app/agent/tools.py:365](#))

```
text = text.replace("đ", "d").replace("Đ", "D")
decomposed = unicodedata.normalize("NFD", text)
return "".join(c for c in decomposed if unicodedata.category(c) != "Mn")
```

Bỏ dấu tiếng Việt về ASCII để 'Đà Nẵng' và 'Da Nang' cùng trùng một location keyword của catalog. Lý do xử lý đ/Đ riêng: chúng KHÔNG phải combining mark (category Mn) nên NFD không tách được — phải replace tay trước. Hàm thuần, không side effect.

edit_tool_definitions() -> List[ToolDefinition] ([app/agent/tools.py:373](#))

Tập tool dùng khi CHỈNH SỬA plan có sẵn. Cách xây dựng:

- Tái dùng 5 tool đọc-only từ `tool_definitions()` (lọc theo set `reuse = {"search_hotels", "search_restaurants", "search_places", "route_legs", "web_search"}`). Lưu ý: `set_constraints`, `view_my_plans`, `search_plans`, `finalize_draft` KHÔNG có mặt ở chế độ edit.
- Chèn đầu danh sách `get_current_plan` (input_schema rỗng `{"type": "object", "properties": {}}` để model đọc board hiện tại (lấy đúng `list_id/card_id`).
- Thêm cuối danh sách `propose_plan_edits` — đây là TERMINAL tool của chế độ edit; schema operations lấy từ `propose_tool_schema()` (import động từ [app/agent/edits.py](#) để tránh vòng import). Required: `["operations"]`.
- Side effect: import động bên trong hàm.

location_slug(text: str) -> str ([app/agent/tools.py:410](#))

`strip_accents(text).strip().lower().replace(" ", "-")` — chuẩn hóa địa điểm thành slug. Dùng cho `location` của hotel/restaurant và cho `destination` scope của place faceted search.

catalog_web_url(kind, slug, base_url="") -> Optional[str] ([app/agent/tools.py:419](#)) và hằng `_CATALOG_PATHS`

```
_CATALOG_PATHS = {"HOTEL": "/hotels", "RESTAURANT": "/restaurants", "PLACE": "/place"}
```

Build link tới trang chi tiết catalog ở frontend SPA. Gotcha quan trọng (comment [app/agent/tools.py:414–app/agent/tools.py:416](#)): `place` là số ít (/place), còn `hotels/restaurants` số NHIỀU — khớp đúng route trong `Mravel_Frontend App.jsx`.

- Trả `None` nếu `slug` rỗng (không link được) hoặc `kind` không nằm trong `_CATALOG_PATHS`.
- `kind` được upper-case trước khi tra `((kind or "").upper())`.
- Khi `base_url` rỗng, link là RELATIVE (vd `"/hotels/<slug>"`) — browser tự resolve theo origin SPA để link chạy in-app. `base_url` được `rstrip('/')` để tránh slash kép.

plan_web_url(plan_id, base_url="") -> Optional[str] ([app/agent/tools.py:433](#))

Build link tới board của plan: `"/plans/{plan_id}"`. Trả `None` nếu `plan_id` là `None` hoặc chuỗi rỗng/whitespace. Cùng quy ước relative-khi-base-url-rỗng.

Hàng số & helper biến đổi dữ liệu

- `_MAX_ITEMS_PER_TOOL = 8` ([app/agent/tools.py:442](#)): hard cap số item mỗi tool response trả về cho model (giữ context gọn). Được dùng vừa làm `size` khi gọi catalog vừa làm slice `[:_MAX_ITEMS_PER_TOOL]`.
- `_drop_none(d)` ([app/agent/tools.py:445](#)): loại bỏ key có value `None` HOẶC chuỗi rỗng `""` — dùng để làm sạch các summary dict.
- `_full_address(item)` ([app/agent/tools.py:449](#)): ghép địa chỉ người-đọc-được từ các phần `addressLine`, `wardName`, `districtName`, và `cityName`-hoặc-`provinceName`. Khử trùng lặp (giữ thứ tự) rồi join bằng `" "`; trả `None` nếu rỗng.

Các hàm `_summarise_*` đều nhận item: Dict (bản ghi thô từ catalog/plan) + `base_url`, trả dict đã `_drop_none`. Hình dạng field cụ thể:

- `_summarise_hotel` ([app/agent/tools.py:465](#)): `id`, `slug`, `name`, `mravel_url` (`kind` HOTEL), `address`, `city` (`cityName`||`provinceName`), `district`, `latitude`, `longitude`, `min_nightly_price_vnd` (`minNightlyPrice`), `reference_nightly_price_vnd` (`referenceNightlyPrice`), `avg_rating`, `star_rating`, `hotel_type`, `distance_to_center_km` (`distanceToCityCenterKm`), `facilities` (`mainFacilitiesSummary`), `cover_image_url`.

- `_summarise_restaurant` ([app/agent/tools.py:489](#)): `id`, `slug`, `name`, `mravel_url` (kind `RESTAURANT`), `address`, `city`, `district`, `lat/lng`, `min_price_per_person_vnd`, `max_price_per_person_vnd`, `price_level`, `restaurant_type`, `cuisines` (cắt còn tối đa 3 phần tử nếu `cuisineNames` là list >3), `avg_rating`, `cover_image_url`.
- `_summarise_place` ([app/agent/tools.py:515](#)): `id`, `slug`, `name`, `mravel_url` (kind `PLACE`), `address`, `lat/lng`, `kind`, `venue_type`, `price_level`, `avg_rating`, `province` (`provinceName`), `cover_image_url`.
- `_summarise_plan_search_item` ([app/agent/tools.py:535](#)): trim một `PlanFeedItem` từ `/api/plans/search`. Field: `id`, `title`, `mravel_url` (qua `plan_web_url`), `days`, `start_date` (`startDate`), `end_date` (`endDate`), `destinations` (danh sách tên — xử lý cả khi phần tử là dict {`name`} hoặc string), `budget_total_vnd` (`budgetTotal`), `budget_per_person_vnd` (`budgetPerPerson`), `author` (chỉ lấy `author.name`), `views`, `visibility`, `description`, `cover_image_url` (`thumbnail`).
- `_summarise_user_plan` ([app/agent/tools.py:565](#)): KHÔNG qua `_drop_none` (giữ nguyên cả None) và không có `mravel_url`. Field: `id`, `title`, `start_date`, `end_date`, `destinations` (list tên), `status`. Comment ghi rõ `/api/plans/me` dùng field của `PlanFeedItem`.

Gotcha: các tên field đầu vào (`minNightlyPrice`, `startDate`, `budgetTotal`...) là camelCase từ JSON Java upstream; tên field đầu ra là snake_case cho model — đây là điểm dễ lệch nếu backend đổi tên.

`class ToolDispatcher` ([app/agent/tools.py:582](#))

Lớp thực thi các tool read-only thay mặt LLM. Docstring nhấn mạnh tính per-request: truyền `bearer_token` để các tool user-scoped (vd `view_my_plans`) xác thực được.

`__init__` ([app/agent/tools.py:586](#)): nhận `catalog`: `CatalogClient` (bắt buộc), `plan_client`: `Optional[PlanClient]`, `bearer_token`: `Optional[str]`, `web_search`: `Optional[Any]`, `web_base_url`: `str` = `""`. Lưu vào `self._catalog`, `self._plan`, `self._bearer`, `self._web_search`, `self._web_base_url`.

`with_token(bearer_token)` ([app/agent/tools.py:600](#)): trả về một `ToolDispatcher` MỚI (immutable pattern) với cùng các client nhưng token khác — dùng để gắn token của request hiện tại mà không mutate instance dùng chung.

`_empty_location_fallback(kind, location)` ([app/agent/tools.py:605](#)): khi search theo location trả rỗng, broaden thành search KHÔNG location (None) để agent vẫn chào được những gì Mravel CÓ (thường là thành phố lân cận) kèm link thật, thay vì dead-end ra site ngoài. Trả về dict shape: `items: [], count: 0`, `no_results_for_location` (= location hoặc `"(unspecified)"`), `available_on_mravel` (list alt summaries), và note tiếng Việt hướng dẫn model gợi ý kèm `mravel_url.kind` chỉ là `'hotels'/'restaurants'`; bắt `UpstreamError` để alt = [] nếu fallback cũng lỗi.

`run(tool_name, arguments) -> Dict[str, Any]` ([app/agent/tools.py:631](#)): dispatcher chính, một chuỗi if `tool_name == ...`. Xử lý từng tool:

- `search_hotels` ([app/agent/tools.py:632](#)): nếu location rỗng → trả ngay `{items:[], count:0, error: "Thiếu 'location'..."}`. Nếu có: slug hóa location, clamp `max_results` về `_MAX_ITEMS_PER_TOOL`, build `filters`. Gotcha quan trọng (comment [app/agent/tools.py:643](#)): tên field filter PHẢI khớp `HotelSearchRequest` (`maxPrice / starRating`[]) nếu không Jackson sẽ âm thầm drop. `starRating` được mở rộng thành "ít nhất N sao" (range từ N tới 5). Gọi `self._catalog.search_hotels(...)` (HTTP ra catalog-service). Nếu trimmed rỗng → gọi `_empty_location_fallback("hotels", location)`.
- `search_restaurants` ([app/agent/tools.py:655](#)): tương tự, default `max_results` 10. `cuisine` là hint free-text được slug hóa thành `filters["cuisineSlugs"]` = `[location_slug(...)]`. Rỗng → `_empty_location_fallback("restaurants", location)`.
- `search_places` ([app/agent/tools.py:667](#)): dedupe theo `slug||id`. Hai bước: (1) destination-scoped POIs qua faceted endpoint (slug-based → accent-proof) nếu có `destination` (đi qua `location_slug`); (2) free-text query truyền NGUYỄN có dấu (place index diacritic-sensitive). Mỗi bước bọc `try/except UpstreamError: pass`. Trả `{items, count}`.
- `route_legs` ([app/agent/tools.py:699](#)): import động `from app.services.geo import route_legs`. Nếu `stops` không phải list hoặc < 2 phần tử → `{legs:[], note: "need at least 2 stops"}`. Tính `total_km` (round 1 chữ số) và `total_minutes` (sum). Trả `{legs, total_km, total_minutes}`. Lưu ý: `route_legs` (từ [app/services/geo.py](#)) là hàm SYNC (không await).
- `web_search` ([app/agent/tools.py:709](#)): nếu `self._web_search` None hoặc `enabled` False → trả `{enabled: False, results:[], note: "Web search chưa được cấu hình..."}`. Nếu query rỗng → `{results:[], note: "empty query"}`. Ngược lại `await self._web_search.search(query, max_results=limit)`. Lưu ý `enabled` được đọc qua `getattr(self._web_search, "enabled", False)`.
- `view_my_plans` ([app/agent/tools.py:721](#)): cần CẢ `self._plan` lẫn `self._bearer`; thiếu → `{items:[], count:0, note: "plan-service not available in this context"}`. Gọi `self._plan.list_my_plans(self._bearer, page=1, size=limit)`. Bắt `UpstreamError` → trả `error: str(exc)`. Đọc `data["items"]`, map qua `_summarise_user_plan`.
- `search_plans` ([app/agent/tools.py:732](#)): cũng cần `_plan` + `_bearer`. Gọi `self._plan.search_plans(...)`. Bắt `UpstreamError`. Đọc lồng: `data["plans"]` [`"items"`] ([app/agent/tools.py:762](#)) — chú ý plans nằm trong sub-object `plans`, KHÁC với `view_my_plans` đọc thẳng `data["items"]`. Map qua `_summarise_plan_search_item`.
- `Default` ([app/agent/tools.py:766](#)): `{error: f"Unknown tool: {tool_name}"}`.

Lưu ý chung về lỗi: dispatcher KHÔNG raise ra ngoài cho tool read-only — mọi nhánh đều trả dict (có thể chứa `error/note`) để orchestrator feed lại model. Hai tool terminal (`set_constraints`, `finalize_draft`, `propose_plan_edits`) KHÔNG được xử lý trong `run` — chúng được orchestrator chặn ở tầng trên.

`apply_set_constraints(arguments, prior) -> Constraints` ([app/agent/tools.py:769](#))

Hợp nhất payload `set_constraints` vào `Constraints` đang sống. Triết lý (docstring): agent là single source of truth — field CÓ trong `arguments` thắng, field VẮNG giữ giá trị prior, "garbage" (chuỗi rỗng, ngày không parse được, số ≤0 không hợp lệ) bị bỏ qua chứ không cho làm hỏng state.

Chi tiết sanitization từng field:

- `_date(value, fallback)` (closure): chấp nhận `date` sẵn, hoặc string ISO (cắt [:10]); parse lỗi → trả `fallback`.
- `destination` ([app/agent/tools.py:787](#)): sanity check không geocode — phải dài ≥2 và có ≥2 ký tự alphanumeric — loại "???", "---", rỗng/whitespace. Gotcha: tên 1 ký tự hoặc toàn ký hiệu bị từ chối (giữ prior).
- `num_days/travelers`: `max(1, int(...))`, bọc `try/except (TypeError, ValueError): pass`.
- `budget_total_vnd`: `max(0, int(...))`.
- `interests`: chỉ ghi đề khi là list non-empty; ép từng phần tử `str(i)`.
- `pace`: chỉ nhận khi value ∈ (`"relaxed"`, `"balanced"`, `"packed"`).
- Trả về `Constraints` mới (immutable) với `start_date/end_date` qua `_date`.

`class DraftValidationError(Exception)` ([app/agent/tools.py:839](#))

Exception riêng dùng khi parse `finalize_draft` thất bại — để orchestrator phân biệt lỗi validation draft với lỗi khác.

`parse_finalize_draft(arguments, constraints) -> PlanDraft` ([app/agent/tools.py:843](#))

Chuyển payload `finalize_draft` của LLM thành `PlanDraft`. Logic và edge case:

- `_date(value, fallback) -> date` (closure, [app/agent/tools.py:846](#)): KHÁC bản trong `apply_set_constraints` ở chỗ trả về `date` (không Optional) — nếu parse fail và `fallback is None` thì **raise** `DraftValidationError`.
- `destination` ([app/agent/tools.py:858](#)): lấy từ `arguments` hoặc `constraints.destination`; rỗng → raise.
- `start/end_date`: dùng `constraints.resolved_date_range()` làm fallback. Comment quan trọng ([app/agent/tools.py:861](#)): khi user chỉ cho `num_days`, `constraints` không có ngày lịch → anchor về hôm nay. Nếu `end_date < start_date` → raise.
- `travelers: int(arguments.get("travelers") or constraints.travelers or 2)` — mặc định 2.
- `days` rỗng → raise "must include at least one day".
- Mỗi activity: `activity_type` parse qua `PlanActivityType(...)`, lỗi → fallback `PlanActivityType.OTHER` ([app/agent/tools.py:882](#)). `recommendation` chỉ được dùng khi payload là dict CÓ `name`; `ValidationError` (pydantic) → bỏ qua (None) chứ không làm hỏng cả draft.
- `cost: int(... or 0)`; `cost_ÂM` → raise `DraftValidationError` ([app/agent/tools.py:894](#)). Cộng dồn vào `total_cost`.
- `DraftActivity` field defaults: `title` "Untitled", `description` "", `day_index` 1, `duration_minutes` 60, các field text rỗng → None.
- `DraftDay`: `day_index` fallback `len(days)+1`, `title` fallback `f"Ngày {len(days)+1}"`.
- `warnings`: chỉ giữ nếu là list.
- `estimated_total_cost_vnd` ([app/agent/tools.py:933](#)): nếu LLM cung cấp số (int/float) thì dùng số đó; ngược lại dùng `total_cost` tự tính. Gotcha: nếu model gửi tổng sai, tổng do model thắng tổng cộng dồn từng activity.
- Trả về `PlanDraft(summary, destination, start_date, end_date, travelers, estimated_total_cost_vnd, days, warnings)`; `summary` mặc định `f"Lịch trình tại {destination}"`.

Hàm này KHÔNG có lời gọi HTTP/Mongo/LLM — thuần biến đổi dữ liệu, chỉ raise `DraftValidationError`.

Tổng kết lời gọi ra ngoài & cấu hình

- **HTTP ra catalog-service** (`CatalogClient`): `search_hotels`, `search_restaurants`, `search_places`, `search_places_faceted`.
- **HTTP ra plan-service** (`PlanClient`): `list_my_plans (/api/plans/me)`, `search_plans (/api/plans/search)`.
- **Web search provider** (`self._web_search.search`): HTTP ra dịch vụ web ngoài, gated bởi flag `enabled`.
- **Local compute** (sync): `app.services.geo.route_legs`.
- **LLM**: không gọi trực tiếp trong file này — file chỉ định nghĩa tool và parse payload.
- File này không đọc trực tiếp `os.environ`. Các giá trị cấu hình được tiêm qua constructor: `web_base_url` (origin SPA để build `mtravel_url`; rỗng → link relative) và `web_search` (provider có/không `enabled`).

Edit operations (EditOperation, parse/validate, board_summary) & EditService

Cụm module này hiện thực luồng "chat sửa plan đã tồn tại" của `ai-plan-service`. Khi người dùng mở một phiên chat trên một plan có sẵn, agent đọc board hiện tại (qua `board_summary`), rồi đề xuất một danh sách thao tác chỉnh sửa thông qua terminal tool `propose_plan_edits`. Các thao tác này được validate thành các object `EditOperation` (file [app/agent/edits.py](#)), hiển thị cho người dùng phê duyệt, và CHỈ khi người dùng bấm "Áp dụng" thì `EditService` (file [app/services/edit_service.py](#)) mới thực thi chúng lên plan-service. Hai file phân chia trách nhiệm rất rạch ròi: `edits.py` là tầng MÔ HÌNH + VALIDATE + tóm tắt (không gọi mạng), còn `edit_service.py` là tầng THỰC THI (gọi `PlanClient` ra plan-service).

`app/agent/edits.py`

Trách nhiệm tổng quát: định nghĩa model `EditOperation` (một model "phẳng" duy nhất thay vì discriminated union để JSON từ LLM đơn giản — mỗi op chỉ điền field nó cần, phần còn lại để `null`), hàm validate `parse_operations`, các hàm phụ kiểm tra ràng buộc/tồn tại id, hàm sinh tóm tắt tiếng Việt cho UI phê duyệt (`operation_summary`), hàm tóm tắt board (`board_summary`), và hàm sinh JSON schema cho tool (`propose_tool_schema`). Module này KHÔNG có side effect ra ngoài (không HTTP/Mongo/LLM) — thuần data + logic. Triết lý thiết kế ([app/agent/edits.py:1–app/agent/edits.py:10](#)): "One flat model (rather than a discriminated union) keeps the LLM JSON simple".

Hàng số OP_TYPES

[app/agent/edits.py:18–app/agent/edits.py:27](#). Đây là tập hợp (set) các giá trị `op` hợp lệ — là nguồn chân lý cho cả validate (`parse_operations`) lẫn schema (`propose_tool_schema`):

```
OP_TYPES = {
    "update_card", "create_card", "delete_card", "move_card",
    "rename_list", "add_day", "delete_list", "update_plan",
}
```

8 loại thao tác: 4 thao tác trên card (`update_card`, `create_card`, `delete_card`, `move_card`), 3 thao tác trên list/ngày (`rename_list`, `add_day`, `delete_list`), và 1 thao tác trên plan tổng (`update_plan`).

`class EditOperation(BaseModel)`

[app/agent/edits.py:30–app/agent/edits.py:68](#). Pydantic `BaseModel`. Tất cả field ngoài `op` đều `Optional` với default `None`. Các nhóm field:

- Bắt buộc: `op: str`.
- Targets (id tham chiếu board): `list_id, card_id, target_list_id, target_position` (đều `Optional[int]`).
- Card/activity fields: `text` (tiêu đề card), `description, start_time/end_time` (chuỗi "HH:MM"), `duration_minutes: int, activity_type: str` (giá trị như `STAY|FOOD|SIGHTSEEING|TRANSPORT|ENTERTAIN|...`), `estimated_cost_vnd: int, unit_price_vnd: int, quantity: int, participant_count: int, address, note`.
- Rich card fields (mirror các cột Excel itinerary + đường approval): `location_name, reason, route_hint, distance_text, transport_mode, và recommendation: Optional[Dict[str, Any]]` (copy từ một kết quả tìm kiếm catalog).
- List/plan fields: `title, start_date/end_date` (chuỗi YYYY-MM-DD), `budget_total_vnd: int`.
- `summary: Optional[str]` — mô tả người-đọc-được hiển thị trên thẻ phê duyệt.

Gotcha: model KHÔNG khai báo validator riêng cho từng `op` (không phải discriminated union), nên một dict thiếu/thừa field vẫn validate được ở tầng pydantic; ràng buộc bắt buộc theo từng `op` được kiểm tra bằng tay ở `_has_required_targets` chứ không phải bằng pydantic.

`class EditValidationError(Exception)`

`app/agent/edits.py:71–app/agent/edits.py:72`. Exception trống, được `parse_operations` raise khi `operations` không phải list hoặc khi không còn `op` hợp lệ nào.

`def parse_operations(raw, board=None) -> List[EditOperation]`

`app/agent/edits.py:75–app/agent/edits.py:107`. Hàm public quan trọng nhất của file: validate payload `operations` mà LLM trả về thành list `EditOperation`.

- Tham số: `raw` (kỳ vọng là list các dict); `board` (tùy chọn — board hiện tại để kiểm tra id thật).
- Trả về: `List[EditOperation]` (đã được gán `summary` nếu LLM bỏ trống). Side effects: KHÔNG có (chỉ mutate `model.summary` của object trả về).

Logic lọc (drop-not-abort): mỗi entry không hợp lệ bị BỎ thay vì làm hỏng cả đề xuất; nhưng phải còn ÍT NHẤT một `op` hợp lệ, nếu không sẽ raise. Trình tự lọc cho mỗi entry: (1) bỏ nếu không phải dict; (2) bỏ nếu `op` không thuộc `OP_TYPES`; (3) bỏ nếu pydantic validate fail (`except Exception` nuốt mọi lỗi); (4) bỏ nếu thiếu target bắt buộc (`_has_required_targets`); (5) khi có `board`, bỏ nếu id tham chiếu KHÔNG tồn tại trên board (`_ids_exist`) — cơ chế chống LLM "hallucinate" id; (6) tự sinh `summary` nếu bỏ trống.

Gotcha quan trọng: việc kiểm tra `_ids_exist` CHỈ chạy khi `board is not None`. Nếu gọi `parse_operations(raw)` mà không truyền board thì chỉ kiểm tra `_has_required_targets`, không bắt được id ma.

`def _board_ids(board) -> tuple (helper)`

`app/agent/edits.py:110–app/agent/edits.py:122`. Gom toàn bộ `list_id` và `card_id` đang tồn tại trên board thành hai set. Trả về (`set()`, `set()`) nếu board không phải dict. `card_ids` được gom phẳng từ tất cả `cards` trong mọi list. Đây là dữ liệu nền cho `_ids_exist`.

`def _ids_exist(op, list_ids, card_ids) -> bool (helper)`

`app/agent/edits.py:125–app/agent/edits.py:135`. Trả về `True/False` tùy theo các id mà `op` tham chiếu có thật sự nằm trong các set hay không, theo từng loại `op`:

- `update_card/delete_card`: cần cả `card_id ∈ card_ids` VÀ `list_id ∈ list_ids`.
- `create_card`: chỉ cần `list_id ∈ list_ids`.
- `move_card`: cần `card_id ∈ card_ids` VÀ `target_list_id ∈ list_ids` (lưu ý: dùng `target_list_id`, không phải `list_id`).
- `rename_list/delete_list`: cần `list_id ∈ list_ids`.
- `add_day/update_plan`: luôn `True` (không tham chiếu id có sẵn).

`def _has_required_targets(op) -> bool (helper)`

`app/agent/edits.py:138–app/agent/edits.py:155`. Kiểm tra ràng buộc tối thiểu cho từng `op` (không liên quan board):

- `update_card/delete_card`: cần `list_id is not None` và `card_id is not None`.
- `create_card`: cần `list_id is not None` và `bool(op.text)` (phải có tiêu đề).
- `move_card`: cần `card_id is not None` và `target_list_id is not None`.
- `rename_list`: cần `list_id is not None` và `bool(op.title)`.
- `delete_list`: cần `list_id is not None`.
- `add_day`: luôn `True`.
- `update_plan`: cần `any([op.title, op.start_date, op.end_date, op.budget_total_vnd])` — tức ít nhất một field plan có giá trị (gotcha: `budget_total_vnd = 0` sẽ là falsy nên không tính, và một `update_plan` chỉ có `start_date` mà thiếu `end_date` vẫn pass tại đây dù `EditService` sau đó yêu cầu CẢ HAI để ghi dates).

`def operation_summary(op) -> str`

`app/agent/edits.py:158–app/agent/edits.py:197`. Sinh một dòng mô tả tiếng Việt ngắn, KHÔNG chứa id (người dùng đọc, nên mô tả theo tên/giờ/giá, không bao giờ theo `card_id/list_id` nội bộ). Trả về `str`. Các mẫu: `update_card` → "Cập nhật hoạt động: ..."; `create_card` → "Thêm hoạt động "<text>" (kèm lúc <start_time> nếu có); `delete_card` → "Xoá một hoạt động"; `move_card` → "Chuyển hoạt động sang ngày khác"; `rename_list` → "Đổi tên ngày → "<title>"; `delete_list` → "Xoá một ngày khỏi lịch trình"; `add_day` → "Thêm ngày mới"; `update_plan` → ghép tên/ngày/ngân sách. Định dạng số dùng `f"{...:,}đ"`. Fallback cuối cùng trả về chính `op.op`.

`def board_summary(board) -> str`

`app/agent/edits.py:200–app/agent/edits.py:241`. Tóm tắt board thành văn bản gọn để agent (1) trả lời câu hỏi về plan và (2) nhắm đúng `list_id/card_id` khi đề xuất sửa. Trả về `str` nhiều dòng; nếu board không phải dict trả về "(không đọc được plan hiện tại)".

Cấu trúc đầu ra:

- Dòng đầu (head): Plan #<planId> "<title>" (<startDate> → <endDate>), thêm , ngân sách <budget>đ nếu có budgetTotal. Đọc tiêu đề từ planTitle hoặc title; id từ planId hoặc id.
- Dòng Mô tả: <desc[:200]> nếu có description (cắt 200 ký tự). Board không có field "điểm đến" riêng → title/description mang thông tin thành phố để agent suy ra city cho catalog search.
- Với mỗi list có type == "DAY" (đã sort theo position, thiếu position thì coi là 9999): in một dòng • {title} (list_id={id}, {dayDate}): rồi từng card - card_id={id} | {startTime}-{endTime} {text} [{activityType}] • {cost}đ. Nếu list không có card → "(chưa có hoạt động)".

Cấu trúc board mà hàm này đọc: cấp board planTitle|title, planId|id, startDate, endDate, budgetTotal, description, lists. Mỗi list: type ("DAY"), position, title, id, dayDate, cards. Mỗi card: position, cost.estimatedCost (chỉ đọc khi cost là dict), startTime, endTime, text, activityType, id.

Gotcha: board_summary HIỂN THỊ card_id/list_id (ngược với operation_summary cố tình giấu id) — vì đây là text cho AGENT đọc để nhầm id, không phải cho người dùng. Chỉ list type == "DAY" được liệt kê; các list type khác bị bỏ qua hoàn toàn.

```
def propose_tool_schema() -> Dict[str, Any]
```

app/agent/edits.py:244–app/agent/edits.py:296. Trả về JSON schema cho mảng operations của terminal tool propose_plan_edits. Là một schema type: "array", mỗi item là object với required: ["op"]. op.enum được sinh từ sorted(OP_TYPES) (đồng bộ với hằng số), nên thêm op mới ở OP_TYPES sẽ tự động vào schema.

Đáng chú ý: schema enum cho activity_type được liệt kê tường minh và là DANH SÁCH ĐẦY ĐỦ chính tắc: ["STAY", "FOOD", "SIGHTSEEING", "TRANSPORT", "ENTERTAIN", "EVENT", "SHOPPING", "CINEMA", "OTHER"] (app/agent/edits.py:268–app/agent/edits.py:271). Schema cũng mô tả estimated_cost_vnd là "Total cost (unit × people)", unit_price_vnd là "Per-person price", và recommendation là object copy từ catalog search result.

app/services/edit_service.py

Trách nhiệm tổng quát: thực thi các EditOperation đã được người dùng phê duyệt lên plan-service. Đây là NƠI DUY NHẤT mutate một plan thật đang tồn tại (agent chỉ đề xuất). Mỗi op map tới một endpoint plan-service qua PlanClient. Lỗi được gom theo từng op (một op hỏng không abort phần còn lại) và báo lại cho người dùng. Module này CÓ side effect mạng (gọi PlanClient → HTTP tới plan-service).

Import phụ thuộc (app/services/edit_service.py:15–app/services/edit_service.py:18): EditOperation (từ app.agent.edits), PlanClient (app.clients.plan_client), PlanActivityType (enum từ app.models.session), catalog_location_fields (từ app.services.catalog_location). Logger tên "ai_plan.edit".

Hằng số idempotency

app/services/edit_service.py:22–app/services/edit_service.py:35.

- _IDEM_NAMESPACE: UUID namespace cố định cho UUIDv5.
- _idem(seed): sinh một UUIDv5 ổn định từ seed (dùng làm idempotency_key).
- _batch_seed(plan_id, operations): tạo seed ổn định cho ĐÚNG tập operations này trên plan này. Hash nội dung batch (model_dump(exclude_none=True), sort_keys=True, default=str để serialize được các kiểu lạ). Cùng một đề xuất → cùng seed (re-apply do double-click sẽ dedup tại plan-service); đề xuất khác → seed khác → apply mới. Comment ở app/services/edit_service.py:154–app/services/edit_service.py:157 nhấn mạnh: dùng index đơn thuần sẽ va chạm sai giữa các đề xuất khác nhau.

```
def _valid_type(value) -> Optional[str] (helper)
```

app/services/edit_service.py:38–app/services/edit_service.py:44. Chuẩn hóa activity_type: None/rỗng → None; nếu hợp lệ trả về PlanActivityType(value.strip().upper()).value; nếu không hợp lệ (raise ValueError) → fallback về PlanActivityType.OTHER.value. Đây là điểm phòng thủ chống activity_type lạ từ LLM.

```
def _norm_time(value) -> Optional[str] (helper)
```

app/services/edit_service.py:47–app/services/edit_service.py:56. Chuẩn hóa giờ về "HH:MM" hai chữ số. Trả về None nếu: không phải str, ít hơn 2 phần khi split ":", hoặc parse int fail. Ví dụ "9:5" → "09:05". Gotcha: phần giây (nếu có) bị bỏ qua vì chỉ lấy parts[0] và parts[1]; không kiểm tra biên 0–23/0–59.

```
def _card_body(op, *, for_create) -> Dict[str, Any] (helper)
```

app/services/edit_service.py:59–app/services/edit_service.py:142. Hàm dựng body request cho card khi create/update, ánh xạ field snake_case của EditOperation sang field camelCase mà plan-service kỳ vọng. Là hàm phức tạp nhất file, gồm các bước:

- (1) participant = op.participant_count or op.quantity — ưu tiên participant_count, fallback quantity.
- (2) Dựng activity_data (sẽ thành activityDataJson) — chỉ giữ các key có giá trị truthy (if v): reason, locationName, address, note, routeHint, distanceText, transportMode, ticketPrice (= unit_price_vnd), ticketCount (= quantity).
- (3) Nếu op.recommendation là dict không rỗng, gán reference catalog (đọc field với nhiều alias: kind|catalog_kind, id|catalog_id, slug|catalog_slug, coverImageUrl|cover_image_url, avgRating|avg_rating) rồi GỌI RA NGOÀI hàm catalog_location_fields(op.activity_type, recommendation, address=op.address) để mirror cách người dùng chọn place thủ công — sinh các field như hotelLocation/restaurantLocation/... giúp FE modal pre-fill (app/services/edit_service.py:76–app/services/edit_service.py:96).
- (4) Dựng body: text (nếu có); description được GHÉP GIỮA từ description + 📍 address + 🗨️ note + 🗺️ route_hint (distance_text) – transport_mode (app/services/edit_service.py:103–app/services/edit_service.py:118); startTime/endTime qua _norm_time; durationMinutes; activityType qua _valid_type (gotcha: khi tạo mới mà thiếu type, ép OTHER — elif for_create); participantCount; activityDataJson (JSON string, ensure_ascii=False); và cost chỉ set khi estimated_cost_vnd is not None (kể cả 0), gồm currencyCode="VND", estimatedCost, và participantCount (nếu participant truthy).

Gotcha: `cost` chỉ được set khi `estimated_cost_vnd is not None` (kể cả 0), nhưng các field trong `activity_data` lại lọc theo truthy (if v) nên giá trị 0/"" bị rớt. `currencyCode` luôn cố định "VND".

class EditService

[app/services/edit_service.py:145](#)–[app/services/edit_service.py:232](#). Khởi tạo với một `PlanClient` (`self._plan`). Hai method async.

```
async def apply(self, plan_id, operations, bearer_token) -> Dict[str, Any]
```

[app/services/edit_service.py:149](#)–[app/services/edit_service.py:166](#). Entry point: chạy lần lượt từng op, gom kết quả per-op.

- Tham số: `plan_id: int`, `operations: List[EditOperation]`, `bearer_token: str`.
- Trả về dict: {"applied": <số op thành công>, "total": len(operations), "results": [...]}. Mỗi phần tử `results` khi thành công: {"op", "ok": True, "summary", "detail"}; khi thất bại: {"op", "ok": False, "summary", "error": str(exc)}.
- `batch_seed` tính một lần cho cả batch; `_apply_one` nhận index `i + batch_seed` để dựng idempotency key duy nhất cho từng op cần idempotency. Một op lỗi không abort batch (collect, don't abort).

```
async def _apply_one(self, plan_id, op, bearer, idx, seed) -> Any
```

[app/services/edit_service.py:168](#)–[app/services/edit_service.py:231](#). Dispatch theo `op.op`, mỗi nhánh gọi đúng một method `PlanClient`. Bảng ánh xạ op → endpoint:

- `update_card` → `self._plan.update_card(bearer, plan_id, op.list_id, op.card_id, _card_body(op, for_create=False))`.
- `create_card` → `self._plan.create_card(bearer, plan_id, op.list_id, _card_body(op, for_create=True), idempotency_key=_idem(f"{seed}:{idx}:create"))`.
- `delete_card` → `self._plan.delete_card(bearer, plan_id, op.list_id, op.card_id)`.
- `move_card` → `self._plan.move_card(..., op.card_id, op.target_list_id, op.target_position, idempotency_key=_idem(f"{seed}:{idx}:move"))`, kèm guard: nếu response thiếu `entityId` → `raise RuntimeError("move_card không xác nhận được (entityId rỗng)")` (chống báo no-op là success — [app/services/edit_service.py:189](#)).
- `rename_list` → `self._plan.rename_list(bearer, plan_id, op.list_id, op.title)`.
- `delete_list` → `self._plan.delete_list(bearer, plan_id, op.list_id)`.
- `add_day` → `self._plan.create_list(bearer, plan_id, {"title": op.title or "Ngày mới"}, idempotency_key=_idem(f"{seed}:{idx}:addday"))`, rồi RE-READ board để lấy id ngày mới (vì `create_list` thực tế trả id null): trả {"new_list_id": `days[-1]["id"]` if `days` else `None`}. Nếu re-read fail → {"new_list_id": `None`} (không raise).
- `update_plan` → cập nhật từng phần độc lập (title / dates / budget), gom danh sách `done`. Gotcha quan trọng: (1) chỉ ghi dates khi có CẢ `start_date` VÀ `end_date`; (2) kiểm tra range đảo ngược (`start > end`) TRƯỚC khi ghi — nếu đảo thì raise `RuntimeError`; (3) nếu `parse date.fromisoformat` fail (`ValueError`) thì pass. Dùng slice `[:10]` để chỉ lấy phần YYYY-MM-DD.
- Mặc định (op lạ) → `raise ValueError(f"unknown op {op.op}")`.

Lưu ý chỉ `create_card`, `move_card`, `add_day` truyền `idempotency_key` (các op còn lại không). `update_card`, `delete_card`, `rename_list`, `delete_list`, và toàn bộ `update_plan` KHÔNG có idempotency key — nên dedup re-apply chỉ áp dụng cho 3 op tạo-mới/di-chuyển.

Các điểm liên hệ chéo (cross-references)

- `parse_operations` ([app/agent/edits.py:75](#)) là cổng vào duy nhất chuyển JSON LLM thành `EditOperation`; `EditService.apply` ([app/services/edit_service.py:149](#)) là cổng ra duy nhất biến `EditOperation` thành mutation thật.
- `OP_TYPES` ([app/agent/edits.py:18](#)) đồng bộ enum trong `propose_tool_schema` ([app/agent/edits.py:254](#)) và dispatch trong `_apply_one` ([app/services/edit_service.py:168](#)); thêm op mới phải sửa cả 3 chỗ (`OP_TYPES`, `_has_required_targets/_ids_exist`, và một nhánh `_apply_one`).
- Enum `activity_type` xuất hiện hai nơi: danh sách chính tắc trong schema ([app/agent/edits.py:268](#)) và `PlanActivityType` dùng để validate khi apply ([app/services/edit_service.py:38](#)–[app/services/edit_service.py:44](#), fallback `OTHER`).
- Field `recommendation` ([app/agent/edits.py:59](#)) được tiêu thụ tại `_card_body` ([app/services/edit_service.py:76](#)) và chuyển tiếp qua `catalog_location_fields` để dựng location fields cho FE.

Env / cấu hình: Hai file này KHÔNG đọc trực tiếp biến môi trường nào. Mọi cấu hình mạng (base URL plan-service, timeout, header) nằm trong `PlanClient` ([app/clients/plan_client.py](#)), được inject vào `EditService.__init__`. `bearer_token` được truyền xuyên suốt từ `apply` tới từng method `PlanClient` để xác thực với plan-service.

Lớp LLM (interface base, OpenAI-compatible client + fallback, stub offline)

Cụm module `app/llm/` định nghĩa **lớp trừu tượng** để toàn bộ phần còn lại của `ai-plan-service` gọi tới mô hình ngôn ngữ mà không cần biết nhà cung cấp cụ thể là ai. Triết lý thiết kế (ghi rõ trong docstring đầu `base.py`) là: mọi cài đặt đều nhắm tới **định dạng OpenAI Chat Completions** (`messages`, `tools`, `tool_calls`) vì hầu hết các provider lớn — OpenRouter, OpenAI, Groq, Together, Ollama, vLLM, 9Router... — đều phớt lờ API này. Đổi provider chỉ cần đổi `LLM_BASE_URL` + `LLM_MODEL`, không phải sửa code.

Cụm gồm ba file:

- `app/llm/base.py` — các dataclass + abstract base class chung (interface).
- `app/llm/openai_client.py` — cài đặt thật, gọi HTTP tới provider OpenAI-compatible, kèm cơ chế fallback nhiều model.
- `app/llm/stub.py` — cài đặt offline thuần regex (không gọi mạng), dùng khi không cấu hình LLM.

Cả ba cài đặt đều implement cùng một interface `LLMClient`, nên orchestrator có thể hoán đổi tự do. Điểm phân biệt then chốt là `supports_tool_use()`: trả `True` cho client thật → chạy vòng lặp agent với tool-calling; trả `False` cho stub → orchestrator bỏ qua vòng lặp agent và rơi về pipeline `DraftComposer` tắt định.

`app/llm/base.py` — Interface và các kiểu dữ liệu chung

Trách nhiệm: định nghĩa các **dataclass** mô tả tool, tool-call, và một "lượt" trả lời của model; cùng **abstract base class** **LLMClient** mà mọi client phải tuân theo. File không gọi mạng, không có side effect — chỉ là khai báo kiểu và chuyển đổi định dạng.

ToolDefinition (dataclass) — [app/llm/base.py:13](#)

- **Field:** `name` (str) tên tool; `description` (str) mô tả cho model; `input_schema` (Dict[str, Any]) là JSON Schema cho tham số tool.
- `to_openai_tool()` -> Dict[str, Any]: dịch sang đúng cấu trúc mảng `tools` của OpenAI Chat Completions — bọc trong `{"type": "function", "function": {...}}`, và ánh xạ `input_schema` → field `parameters`. Đây là điểm "cách ly" định dạng: phần còn lại của hệ thống định nghĩa tool theo `ToolDefinition`, chỉ tại biên gọi LLM mới chuyển sang dạng OpenAI.

ToolUse (dataclass) — [app/llm/base.py:30](#)

Đại diện cho một lời gọi tool mà model yêu cầu thực thi.

- `id` (str) — id của tool-call (do provider cấp, dùng để khớp với message `role: "tool"` trả về sau khi thực thi).
- `name` (str) — tên tool.
- `arguments` (Dict[str, Any]) — tham số đã được parse từ JSON về dict.

LLMTurn (dataclass) — [app/llm/base.py:37](#)

Đây là **giá trị trả về chuẩn hoá** của một lượt model, hình dạng cụ thể:

- `stop_reason` — enum chuỗi với đúng 4 giá trị: `"tool_use"` (model muốn gọi tool), `"end_turn"` (trả lời xong bình thường), `"max_tokens"` (bị cắt do chạm giới hạn token), `"error"` (gọi model thất bại). Orchestrator dựa vào field này để rẽ nhánh.
- `text` (str, mặc định "") — phần văn bản trợ lý trả về.
- `tool_uses` (List[ToolUse], mặc định list rỗng) — danh sách tool-call khi `stop_reason == "tool_use"`.
- `raw_assistant_message` (Optional[Dict], mặc định None) — **message assistant thô theo đúng định dạng OpenAI** (gồm cả `tool_calls` nếu có). Theo comment tại [app/llm/base.py:44](#), cài đặt client **không** điền field này để orchestrator có thể chèn nguyên vẹn message vào hội thoại cho lượt kế tiếp, **không phải tự dựng lại payload `tool_calls`**. Việc giữ message thô đảm bảo lịch sử hội thoại OpenAI nhất quán qua nhiều lượt tool-calling.
- `error` (Optional[str], mặc định None) — thông điệp lỗi thân thiện khi `stop_reason == "error"`.

LLMClient (ABC) — [app/llm/base.py:51](#)

Abstract base class với 3 phương thức trừu tượng:

- `supports_tool_use(self)` -> bool ([app/llm/base.py:52](#)): trả `False` cho stub offline. Comment ghi rõ: *"When False, callers skip the agent loop."* Đây là cờ điều phối cốt lõi.
- `async extract_constraints(self, user_message, prior_constraints)` -> Dict[str, Any] ([app/llm/base.py:57](#)): trích/gộp ràng buộc theo từng tin nhắn, trả về một JSON đơn (dict). Mục tiêu là "single-shot JSON output for cheap, per-message constraint merging" — tách riêng khỏi vòng lặp agent đắt đỏ.
- `async chat_with_tools(self, system_prompt, messages, tools, max_tokens=2048)` -> LLMTurn ([app/llm/base.py:63](#)): chạy một lượt model có bật tool-calling. Theo docstring, `messages` là **toàn bộ hội thoại đã ở định dạng OpenAI**, bao gồm mọi message `tool_call` / `tool` trước đó — tức orchestrator chịu trách nhiệm duy trì lịch sử, client chỉ thêm `system` vào đầu.

`app/llm/openai_client.py` — Client thật, OpenAI-compatible + fallback nhiều model

Trách nhiệm: cài đặt **LLMClient** gọi HTTP thực tới bất kỳ endpoint OpenAI-compatible nào. Ngoài việc gọi model, file gói rất nhiều **logic xử lý lỗi và fallback** đặc thù cho 9Router (provider gateway hay bọc lỗi upstream bên trong HTTP 400).

Module-level:

- `logger = logging.getLogger("ai_plan.llm")` ([app/llm/openai_client.py:13](#)).
- `_EXTRACTOR_SYSTEM` ([app/llm/openai_client.py:16](#)) — hằng chuỗi system prompt cho bộ trích ràng buộc. Nó mô tả **schema JSON đầu ra** mà model phải tuân theo, gồm các field: `destination` (string|null), `num_days` (integer|null), `start_date/end_date` ("YYYY-MM-DD"|null), `travelers` (integer, mặc định 2), `budget_total_vnd` (integer|null), `interests` ([string]), `pace` ("relaxed"|"balanced"|"packed"|null). Quy tắc: giữ giá trị cũ trừ khi bị mâu thuẫn; **không bịa ngày**; ưu tiên `num_days`; chỉ điền `start_date/end_date` khi user nêu ngày lịch cụ thể; chỉ xuất JSON.

`class OpenAILLMClient(LLMClient)`

`__init__` — [app/llm/openai_client.py:39](#)

- **Tham số:** `api_key`, `base_url`, `model` (model chính), `extractor_model` (model riêng cho trích ràng buộc — mặc định dùng `model`), `app_name/app_url` (đặt header OpenRouter X-Title / HTTP-Referer — vô hại với provider khác), `fallback_models` (danh sách model dự phòng).
- **Side effect / Lời gọi ra ngoài:** import lười `AsyncOpenAI` (chỉ nạp khi cần) và khởi tạo client HTTP. `base_url` bị `rstrip("/")` để tránh double-slash. `default_headers` or `None` đảm bảo không truyền dict rỗng.
- **Env liên quan:** `LLM_API_KEY`, `LLM_BASE_URL`, `LLM_MODEL`, `LLM_FALLBACK_MODELS`, và (gián tiếp) extractor model + app name/url.

`supports_tool_use(self)` -> bool — [app/llm/openai_client.py:67](#)

Luôn trả `True`.

`_chat_models(self)` -> List[str] — [app/llm/openai_client.py:70](#)

Gotcha quan trọng: model chính được thử **hai lần** trước khi rơi sang fallback (`attempts = [self._model, self._model]` rồi nối thêm các fallback ≠ model chính). Lý do (comment [app/llm/openai_client.py:71](#)): một *combo* trên 9Router ở chế độ round-robin sẽ xoay sang model nền khác mỗi lần gọi, nên lần retry thứ hai thường rơi

trúng model còn khoẻ.

`async extract_constraints(...)` — [app/llm/openai_client.py:80](#)

Trả về `Dict[str, Any]` — JSON ràng buộc đã gộp; **fallback về `prior_constraints` nguyên trạng** mỗi khi có lỗi/parse thất bại (nguyên tắc: không bao giờ để lỗi LLM giết session).

- Dựng prompt chứa `prior_constraints` (serialize bằng `json.dumps(..., ensure_ascii=False, default=str)`) + `user_message`.
- Danh sách model thử = `[extractor_model1, *fallbacks]` đã khử trùng lặp.
- Lời gọi ra ngoài:** `await self._client.chat.completions.create(...)` với `max_tokens=512`, `temperature=0.1`, và đặc biệt `response_format={"type": "json_object"}` để ép JSON mode ([app/llm/openai_client.py:97](#)).
- Xử lý lỗi:** `except Exception` (cố ý bắt rộng — # noqa: BLE001 — never let LLM errors kill the session). Khi lỗi: log ở mức ERROR kèm `_status_code(exc)` và `_provider_detail(exc)`; nếu lỗi **retryable** và còn model khác thì `continue`, ngược lại `return prior_constraints` ([app/llm/openai_client.py:108](#)).
- Parse kết quả + edge case** ([app/llm/openai_client.py:119–app/llm/openai_client.py:132](#)): có model đôi khi vẫn bọc JSON trong code-fence dù đã bật `response_format`. Code strip backtick + tiền tố `json`, thử parse lại; nếu vẫn hỏng → log WARNING (cắt 200 ký tự đầu) và trả `prior_constraints`.

`async chat_with_tools(...)` → `LLMTurn` — [app/llm/openai_client.py:134](#)

Đây là phương thức trung tâm cho vòng lặp agent.

- Dựng `full_messages = [{"role": "system", "content": system_prompt}, *messages]` ([app/llm/openai_client.py:141](#)) — client tự chèn system vào đầu.
- Chuyển `tools` sang định dạng OpenAI qua `t.to_openai_tool()` ([app/llm/openai_client.py:145](#)).
- Lấy danh sách model qua `_chat_models()` (model chính x2 + fallback).

Vòng lặp gọi model + fallback ([app/llm/openai_client.py:150–app/llm/openai_client.py:174](#)):

- Lời gọi ra ngoài:** `chat.completions.create` với `temperature=0.4`, `tool_choice="auto"` chỉ khi có tool. Lưu ý: nếu lỗi **không** retryable thì `break` ngay (không thử model tiếp theo) — khác với chỉ `continue` khi retryable.
- Log lỗi ở mức ERROR kèm `exc_info=True` (full traceback) và phân loại `retryable/fatal`.
- Nếu sau vòng lặp `response is None` → trả `LLMTurn(stop_reason="error", error=_friendly_error(last_error))` ([app/llm/openai_client.py:176](#)).

Dựng `LLMTurn` từ response ([app/llm/openai_client.py:179–app/llm/openai_client.py:217](#)):

Quy tắc rẽ nhánh `stop_reason`:

- Nếu `msg.tool_calls` có → `LLMTurn(stop_reason="tool_use", text=..., tool_uses=[...], raw_assistant_message=...)`. Mỗi `ToolUse` lấy `id`, `name`, và `arguments` đã parse qua `_safe_json(tc.function.arguments)`.
- Nếu không có `tool_calls` và `finish_reason == "length"` → `stop_reason="max_tokens"`.
- Còn lại → `stop_reason="end_turn"`.

Trong cả ba nhánh, `raw_assistant_message` luôn được điền, và `arguments` trong `raw_assistant_message` giữ **dạng chuỗi gốc** (`tc.function.arguments` chưa parse), còn `tool_uses[i].arguments` là **dạng dict đã parse** — phân biệt này có chủ ý: message thô để re-insert phải đúng wire format, còn agent dùng dict.

Các hàm helper xử lý lỗi/fallback (module-level)

Đây là phần "thông minh" nhất của file, sinh ra để khắc phục đặc thù 9Router.

- `_provider_detail(exc)`** — [app/llm/openai_client.py:220](#): lấy thông điệp lỗi giàu thông tin nhất: ưu tiên `exc.body` (đã parse) — nếu là dict thì bóc `body["error"]["message"]`; nếu không có thì dùng `exc.response.text` (cắt 1000 ký tự); cuối cùng fallback `str(exc)`. Comment ghi rõ 9Router **bọc lỗi upstream (vd 429 của model) bên trong body ngay cả khi HTTP status là 400**.
- `_RETRYABLE_CODES`** — [app/llm/openai_client.py:245](#): hằng tuple (402, 408, 409, 429, 500, 502, 503, 504).
- `_embedded_code(text)`** — [app/llm/openai_client.py:248](#): cào mã HTTP upstream từ thân lỗi (vd [429] hoặc code: 402). **Bỏ qua 400** vì đó chỉ là lớp bọc của 9Router, cần mã thật bên trong.
- `_status_code(exc)`** — [app/llm/openai_client.py:260](#): mã hiệu lực để phân loại: ưu tiên **mã nhưng** (`_embedded_code(_provider_detail(exc))`); nếu không có thì dùng `exc.status_code`; cuối cùng thử cào từ `str(exc)`.
- `_is_retryable(exc)`** — [app/llm/openai_client.py:273](#): `return _status_code(exc) in _RETRYABLE_CODES`.
- `_friendly_error(exc)`** — [app/llm/openai_client.py:278](#): sinh hướng dẫn tiếng Việt **kèm** chi tiết thật của provider (cắt 400 ký tự). Ánh xạ: 429 → "Mô hình AI hết lượt (429 — rate limit)..."; 402 → "Mô hình AI cần thêm credit (402) trên 9Router..."; 401/403 → "Khoá API AI không hợp lệ (401/403). Kiểm tra LLM_API_KEY."; >= 500 → "Máy chủ AI gặp sự cố (5xx)..."; còn lại → "Không gọi được mô hình AI (mã {code or '?'})...". Đây là chuỗi đi vào `LLMTurn.error` và (qua orchestrator) hiển thị cho người dùng cuối.
- `_safe_json(raw)`** — [app/llm/openai_client.py:295](#): parse chuỗi arguments của tool-call về dict; trả {} nếu `raw` rỗng, không phải JSON, hoặc parse ra không phải dict. Edge case: model trả arguments không hợp lệ sẽ không làm vỡ agent — chỉ thành dict rỗng.

`app/llm/stub.py` — Bộ trích ràng buộc tắt định, offline (regex)

Trách nhiệm (docstring [app/llm/stub.py:1](#)): bộ trích **rule-based** dùng khi **không cấu hình LLM ngoài**. Bắt các pattern tiếng Việt + tiếng Anh thường gặp: điểm đến, độ dài, ngày tương đối/tuyệt đối, số người, ngân sách, sở thích, nhịp độ. **Không gọi mạng, không Mongo, không LLM** — hoàn toàn cục bộ và tắt định. Điểm tinh tế ([app/llm/stub.py:10](#)): nó vẫn chạy **song song** với extractor LLM thật — baseline regex đảm bảo luôn có gì đó dùng được kể cả khi LLM ảo giác hoặc từ chối JSON mode.

Các regex và hằng số (module-level)

- `_DURATION_DAYS_RE`** ([app/llm/stub.py:20](#)): `(\d+)\s*(?:ngày|day|days|đêm|night|nights)` — bắt độ dài chuyển.
- `_PEOPLE_RE`** ([app/llm/stub.py:21](#)): `(\d+)\s*(?:người|pax|people|persons?|adults?)`.
- `_BUDGET_RE`** ([app/llm/stub.py:22](#)): bắt số + đơn vị `triệu|tr|million|m|nghìn|k|thousand`.

- `_DESTINATION_RE` (app/llm/stub.py:25) và `_DESTINATION_NOTRIGGER_RE` (app/llm/stub.py:35): hai regex bắt điểm đến — một loại có "trigger word" (đi|tới|đến|ở|tại|to|in|at|visit), một loại không trigger (du lịch|kế hoạch|lịch trình|chuyến|phượt|vi vu|khám phá).
- `_DESTINATION_STOPWORDS` (app/llm/stub.py:45): set các token **không bao giờ** là điểm đến thật (liền từ/tiểu từ). Guard chống case kiểu "31/5 đi và 3/6" bắt nhầm đi + và → điểm đến "và".
- `_VN_DESTINATIONS` (app/llm/stub.py:56): danh sách (canonical, surfaces) whitelist 24 điểm đến VN phổ biến (vd ("Đà Nẵng", ("đà nẵng", "da nang", "danang"))). Đây là **đường có độ chính xác cao nhất**.
- `_PACE_RE` (app/llm/stub.py:82), `_INTEREST_KEYWORDS` (app/llm/stub.py:84) (dict nhân → keyword: biển, núi, ẩm thực, lịch sử, mua sắm, giải trí).
- `_ISO_DATE_RE` (app/llm/stub.py:149), `_DM_RE` (app/llm/stub.py:150, dd/mm/yyyy), `_DM_VN_RE` (app/llm/stub.py:151, "ngày D tháng M [năm Y]").
- `_WEEKDAY_VN` (app/llm/stub.py:156): dict ánh xạ tên thứ (VN + EN) → index 0–6.

Các hàm parse helper

- `_parse_budget_vnd(text)` (app/llm/stub.py:94): nhân theo đơn vị — triệu/tr/million/m × 1_000_000; nghìn/k/thousand × 1_000. `replace(", ", ".")` để chấp nhận dấu phẩy thập phân kiểu VN.
- `_parse_interests(text)` (app/llm/stub.py:107): quét keyword, trả danh sách nhân khớp.
- `_clean_destination(raw)` (app/llm/stub.py:116): cắt phần đuôi sau trong|for|từ|on|với|with, strip dấu câu; **reject** nếu rỗng, nằm trong `_DESTINATION_STOPWORDS`, là số thuần, hoặc dài < 2 ký tự.
- `_parse_destination(text)` (app/llm/stub.py:130): thứ tự ưu tiên — (1) whitelist `_VN_DESTINATIONS`, (2) `_DESTINATION_NOTRIGGER_RE`, (3) `_DESTINATION_RE` (trigger word).
- **Xử lý ngày:** `_next_weekday` (app/llm/stub.py:167); `_try_parse_yyyymmdd/_try_parse_dm/_try_parse_dm_vn` (app/llm/stub.py:174, :185, :209) có **suy luận năm** (thiếu năm → năm hiện tại, nếu đã trôi qua thì +1 năm); `_try_parse_relative` (app/llm/stub.py:232) xử lý "ngày mai"/"tuần sau"/"cuối tuần"/"thứ N này/sau".
- `_parse_dates(text, today)` (app/llm/stub.py:265): hàm chiến lược nhiều bước. **Gotcha quan trọng** (app/llm/stub.py:308–app/llm/stub.py:311): nếu **chỉ có duration mà không có ngày neo** → trả (None, None), **không bịa** today+7; để trống để agent hỏi "đi từ ngày nào". Phần neo-vào-hôm-nay dời sang `Constraints.resolved_date_range()`.
- `_has_explicit_date_phrase(text)` (app/llm/stub.py:314): true nếu có ngày tuyệt đối, hoặc cụm "ngày mai/tomorrow/tuần sau/cuối tuần", hoặc tên thứ. Dùng làm điều kiện cho phép ghi đề ngày đã có.
- `_parse_pace(text)` (app/llm/stub.py:326): map về "relaxed" / "packed" / "balanced".

class StubLLMClient(LLMClient) — app/llm/stub.py:338

- `supports_tool_use(self)` → bool (app/llm/stub.py:339): trả False → caller bỏ qua agent loop, dùng pipeline `DraftComposer` tắt định.
- `async chat_with_tools(...)` → `LLMTurn` (app/llm/stub.py:344): luôn trả về `LLMTurn(stop_reason="end_turn", text="(stub LLM — no agent loop...))`. Không gọi gì cả — chỉ là placeholder để thoả interface.
- `async extract_constraints(self, user_message, prior_constraints)` → `Dict[str, Any]` (app/llm/stub.py:356): bộ trích thật của stub. Trả về **bản copy của prior_constraints** đã gộp thêm field mới. Các field được set:
 - `destination` (app/llm/stub.py:362–app/llm/stub.py:369): chỉ set nếu chưa có; kèm **sanity check không geocode**: tên hợp lệ phải có ≥ 2 ký tự alphanumeric — loại rác kiểu "???", "---". Comment ghi rõ logic này **mirror apply_set_constraints trong app/agent/tools.py**.
 - `start_date/end_date` (app/llm/stub.py:371–app/llm/stub.py:379): chỉ ghi đề khi chưa có ngày cũ, **hoặc** ngày mới khác ngày cũ và `_has_explicit_date_phrase(user_message)` đúng.
 - `num_days` (app/llm/stub.py:383): set từ `_DURATION_DAYS_RE` (tối thiểu 1), **kể cả khi không có ngày lịch** — để planner neo start vào hôm nay.
 - `travelers` (app/llm/stub.py:387): từ `_PEOPLE_RE` (tối thiểu 1).
 - `budget_total_vnd` (app/llm/stub.py:391): từ `_parse_budget_vnd`.
 - `interests` (app/llm/stub.py:395): **gộp** với sở thích cũ, khử trùng lặp giữ thứ tự bằng `list(dict.fromkeys([*prior, *new]))`.
 - `pace` (app/llm/stub.py:401): từ `_parse_pace`.

Hình dạng giá trị trả về của `extract_constraints` (stub) khớp đúng schema mô tả trong `_EXTRACTOR_SYSTEM` ở `openai_client.py`, nên hai cài đặt có thể thay thế nhau.

Tóm tắt liên kết (cross-cutting)

- **Cờ điều phối:** `supports_tool_use()` quyết định orchestrator chạy agent loop (`OpenAILLMClient` → True) hay rơi về `DraftComposer` tắt định (`StubLLMClient` → False).
- **Env quan trọng** (lấy vào `OpenAILLMClient.__init__` và xuất hiện trong `_friendly_error`): `LLM_API_KEY`, `LLM_BASE_URL`, `LLM_MODEL`, `LLM_FALLBACK_MODELS` (+ extractor model, app name/url). Đổi provider = đổi `LLM_BASE_URL` + `LLM_MODEL`.
- **Triết lý xử lý lỗi:** không bao giờ để lỗi LLM giết session — `extract_constraints` luôn fallback về `prior_constraints`, `chat_with_tools` luôn trả `LLMTurn(stop_reason="error", ...)` với thông điệp tiếng Việt + chỉ tiết provider thật.
- **Đặc thù 9Router:** provider bọc lỗi upstream (429/402) bên trong HTTP 400; vì vậy `_embedded_code/_status_code` ưu tiên cào mã thật từ body, và `_chat_models` thử model chính 2 lần để tận dụng round-robin combo.

Constraint Extractor (regex + LLM) & Draft Composer (dựng nháp tắt định)

Cụm hai module này tạo thành "đường an toàn tắt định" của `ai-plan-service`: `ConstraintExtractor` rút trích ràng buộc chuyển đi từ ngôn ngữ tự nhiên (lai giữa regex và LLM), còn `DraftComposer` dựng một lịch trình chi tiết, dày đặc và đúng giờ giấc mà KHÔNG cần LLM. Chúng đóng vai trò mạng lưới an toàn cho luồng agent: orchestrator gọi `extractor.update(...)` mỗi lượt người dùng để cập nhật ràng buộc (app/api/ai_plan.py:230, app/api/ai_plan.py:482), và gọi `composer.compose(constraints)` khi không có LLM hoặc khi vòng lặp agent thất bại (app/agent/orchestrator.py:477, app/agent/orchestrator.py:488).

app/services/constraint_extractor.py

Trách nhiệm tổng quát

Hợp nhất (merge) các "facts" mới rút ra từ tin nhắn người dùng vào đối tượng `Constraints` đã tích lũy trước đó, theo cơ chế hai bước ([app/services/constraint_extractor.py:1](#)):

- Bước 1 — regex baseline tất định:** LUÔN chạy `StubLLMClient.extract_constraints` (rẻ, miễn phí, bền với các model yếu hay phớt lờ JSON-mode). Bắt các mẫu Việt/Anh hiển nhiên: điểm đến, thời lượng, số người, ngân sách.
- Bước 2 — LLM tinh chỉnh:** CHỈ chạy khi có LLM thật được nối dây. Hữu ích cho cách diễn đạt mơ hồ hoặc lấp các field regex bỏ sót.

Trí tuệ quan trọng: nếu lời gọi LLM lỗi hoặc trả rác, baseline từ regex VẪN thắng — người dùng không bao giờ thấy tập ràng buộc trống do LLM hỏng.

Hàng số cấp module: `_STUB_EXTRACTOR = StubLLMClient()` ([app/services/constraint_extractor.py:20](#)) — một instance dùng chung, không trạng thái, để chạy bước regex.

`class ConstraintExtractor`

Khởi tạo: `__init__(self, llm: LLMClient)` — giữ tham chiếu `self._llm`. Được lắp qua DI tại `get_constraint_extractor` ([app/api/dependencies.py:58](#)). Khi không có API key, `llm` là một `StubLLMClient`.

`async def update(self, prior, user_message, conversation_text=None) -> Constraints`

Hợp nhất facts mới từ `user_message` vào `prior`.

- Tham số:** `prior: Constraints` (ràng buộc tích lũy); `user_message: str` (tin mới nhất, cửa sổ một-tin-nhắn); `conversation_text: str | None` (toàn bộ các lượt user nối lại; cho phép khôi phục điểm đến đã nêu ở lượt đầu mà cửa sổ một-tin-nhắn bỏ sót — bên gọi truyền từ [app/api/ai_plan.py:128](#)).
- Giá trị trả về:** một `Constraints` mới. Hàm là thuần — không ghi DB, không phát side effect ngoài việc gọi LLM ở bước 2.
- Lời gọi ra ngoài:** `_STUB_EXTRACTOR.extract_constraints(...)` (thuần CPU, không I/O) và — chỉ khi có LLM thật — `self._llm.extract_constraints(...)` (HTTP tới provider LLM).

Luồng cốt lõi:

- `prior.model_dump(mode="json", exclude_none=False)` ([app/services/constraint_extractor.py:40](#)): chuyển `Constraints` thành dict JSON-able (ví dụ `date` → chuỗi ISO); `exclude_none=False` GIỮ các field `None` để stub có thể đọc `result.get("destination")` mà không lỗi key.
- Bước 1** (`_normalize(stub_result, prior)`): chạy stub regex baseline.
- Bước 1b** ([app/services/constraint_extractor.py:47](#)–[app/services/constraint_extractor.py:50](#)): nếu baseline vẫn chưa có `destination` và có `conversation_text`, chạy `_parse_destination` trên toàn hội thoại để khôi phục điểm đến đã nêu ở lượt 1 — không có bước này, điểm đến chỉ nói một lần ở turn 1 sẽ "mất vĩnh viễn" khi tin nhắn đó trôi khỏi cửa sổ. Cập nhật bằng `model_copy(update=...)` (bất biến).
- Short-circuit khi là stub** ([app/services/constraint_extractor.py:53](#)–[app/services/constraint_extractor.py:54](#)): nếu `self._llm` là `StubLLMClient` thì baseline chính là kết quả cuối — KHÔNG gọi LLM lần hai.
- Bước 2** ([app/services/constraint_extractor.py:57](#)–[app/services/constraint_extractor.py:61](#)): chạy LLM refinement; `except Exception (BLE001)` nuốt MỌI lỗi LLM và trả `baseline` — bảo đảm "request không bao giờ fail vì LLM". Kết quả LLM được `_normalize(llm_result, baseline)` với `prior=baseline` (chứ không phải `prior` gốc), nên LLM tinh chỉnh TRÊN nền regex.

Gotcha: việc phân biệt stub/LLM dựa vào `isinstance(self._llm, StubLLMClient)`, không dựa vào `supports_tool_use()`. Bất kỳ client nào không phải `StubLLMClient` đều được coi là "LLM thật" và sẽ bị gọi ở bước 2.

`def _normalize(data, prior) -> Constraints`

Ép kiểu (coerce) output dạng dict của stub/LLM trở lại `Constraints` đã định kiểu, đồng thời BẢO TỒN giá trị `prior` khi field mới thiếu/sai. Đây là hàng rào kiểu dữ liệu chống lại LLM trả rác.

Hàm con `_date(value)` ([app/services/constraint_extractor.py:67](#)–[app/services/constraint_extractor.py:75](#)): chấp nhận sẵn `date`, hoặc parse chuỗi ISO bằng `date.fromisoformat(value[:10])` (cắt 10 ký tự đầu để bỏ phần giờ); `ValueError` → trả `None`.

Quy tắc ép kiểu từng field ([app/services/constraint_extractor.py:77](#)–[app/services/constraint_extractor.py:108](#)):

- `interests`: lấy `data["interests"]` hoặc `prior.interests` hoặc `[]`; nếu không phải list thì bọc thành `[str(interests)]`; cuối cùng ép từng phần tử thành `str`.
- `travelers`: `max(1, int(travelers_raw))`; nếu `TypeError/ValueError` → `prior.travelers`. Luôn ≥ 1 .
- `budget_total_vnd`: `int(budget_raw)` nếu không `None`; lỗi → `prior.budget_total_vnd`.
- `num_days`: `max(1, int(...))` nếu không `None`; lỗi → `prior.num_days`. Luôn ≥ 1 (hoặc `None`).
- `destination, start_date, end_date, pace`: dùng pattern `data.get(x)` or `prior.x` — giá trị mới chỉ thắng khi truthy, ngược lại giữ `prior`.

Edge case: việc khởi tạo `Constraints(...)` này kích hoạt validator `_validate_date_range` ([app/models/session.py:42](#)) — nếu `end_date < start_date` thì TỰ SWAP thay vì raise (để không gây giữa hội thoại). Validation cứng, hướng-người-dùng để ở `finalize_draft`.

Phụ thuộc regex: StubLLMClient & các parser ([app/llm/stub.py](#))

`ConstraintExtractor` ủy thác toàn bộ bước regex cho `StubLLMClient.extract_constraints` ([app/llm/stub.py:356](#)) và `_parse_destination` ([app/llm/stub.py:130](#)). Một số chi tiết quan trọng để hiểu output của bước 1:

- `StubLLMClient.supports_tool_use()` trả `False` ([app/llm/stub.py:339](#)) — báo cho orchestrator bỏ qua vòng lặp agent và dùng `DraftComposer`.
- `extract_constraints` bắt đầu từ dict(`prior_constraints`) rồi GHI ĐỀ có điều kiện. **Sanity-check điểm đến:** `if dest and len(dest.strip()) >= 2 and sum(c.isalnum() for c in dest) >= 2` ([app/llm/stub.py:368](#)) — chặn rác kiểu "???", "---"; mirror `apply_set_constraints` trong [app/agent/tools.py](#).
- `_parse_dates` ([app/llm/stub.py:265](#)): **gotcha quan trọng** — nếu chỉ có thời lượng mà không có ngày neo, hàm trả `(None, None)` chứ KHÔNG bịa `today+7`; phần neo-vào-hôm-nay dời sang `Constraints.resolved_date_range()`.

Biến môi trường / cấu hình liên quan

- `LLM_API_KEY` rỗng → factory tạo `StubLLMClient` (luồng regex thuần, không agent) ([app/llm/factory.py:9](#)).
- `llm_extractor_model` (env `LLM_EXTRACTOR_MODEL`) — model riêng cho extractor, fallback về `llm_model` qua `effective_extractor_model()` ([app/config.py:73](#)).
- `LLM_BASE_URL` + `LLM_MODEL` — đổi provider chỉ cần đổi hai biến này.

`app/services/draft_composer.py`

Trách nhiệm tổng quát

Bộ lập lịch lịch trình **per-person, geo-aware, tất định** — đường mạng-lưới-an-toàn dùng khi không có LLM hoặc agent fail (docstring [app/services/draft_composer.py:1](#)). Tạo ra một ngày DÀY ĐẶC, thực tế theo khuôn mẫu bảng tính Đà Nẵng tham chiếu: ăn sáng → cafe sáng → đi chơi → ăn trưa → nghỉ trưa (về KS) → đi chơi → ăn tối → đi chơi tối → về khách sạn nghỉ ngơi.

Ba thuộc tính cốt lõi: (1) **Geography** — venue được cụm theo độ gần khách sạn, giờ bắt đầu nối chuỗi qua thời gian di chuyển THẬT (haversine + mode); (2) **Per-person cost** — bữa ăn/vé scale theo số người, lưu trú tính per-room×night, giá lấy từ catalog hoặc dài VN thực tế (không bao giờ 100k phẳng); (3) **Full cards** — địa chỉ, route_hint, distance_text, transport_mode, venue thay thế trong note.

`logger = logging.getLogger("ai_plan.composer")` ([app/services/draft_composer.py:38](#)).

Hàng số & cấu trúc dữ liệu cấp module

- `DAY_START = time(6, 30)` ([app/services/draft_composer.py:40](#)) — mốc khởi đầu ngày trước khi nối chuỗi di chuyển.
- `_PACE_SLOTS: Dict[str, List[Tuple[str, str, int, Optional[time]]]]` ([app/services/draft_composer.py:49](#)) — template slot theo nhịp. Mỗi slot là (key, category, duration_min, anchor). Có 3 nhịp: "relaxed" (8 slot), "balanced" (10 slot), "packed" (12 slot). `category ∈ breakfast|cafe|sight|lunch|rest|dinner|evening|sleep`.
- `anchor` (docstring [app/services/draft_composer.py:43](#)–[app/services/draft_composer.py:48](#)) là **giờ đồng hồ SỚM NHẤT thực tế** cho slot đó. Bộ lập lịch nối chuỗi thời gian di chuyển thật nhưng KHÔNG BAO GIỜ bắt đầu slot trước anchor — nên ăn trưa rơi vào ~12:00, ăn tối ~19:00, ngày kéo dài tới ~22:00 ngay cả khi buổi sáng ít điểm dừng.

Hàm helper thuần (pure)

- `_clamp_anchor(chained, anchor)` ([app/services/draft_composer.py:89](#)): trả `anchor` nếu `chained < anchor`, ngược lại trả `chained` — giữ slot trong phần ngày thực tế.
- `_time_prefix(start)` ([app/services/draft_composer.py:96](#)): gán nhãn theo giờ đồng hồ THỰC, không theo slot key. `h >= 18` → "Tối", `h >= 13` → "Chiều", còn lại "Sáng". Ngăn nhãn "Tối:" lúc 13:11.
- `_coords(item)` ([app/services/draft_composer.py:106](#)): đọc `item["latitude"]`/`["longitude"]`, ép float; None/lỗi → None.
- `_address(item)` ([app/services/draft_composer.py:118](#)): ghép `addressLine`, `wardName`, `districtName`, `cityName`|`provinceName` (khử trùng, bỏ rỗng).
- `_add_minutes(t, minutes)` ([app/services/draft_composer.py:135](#)): cộng phút và **cap ở 23:59** thay vì cuộn sang ngày hôm sau — chặn leg cuối ngày dài làm wrap một hoạt động muộn về buổi sáng.
- `_hhmm(t)` ([app/services/draft_composer.py:143](#)): format "%H:%M".
- `_short_name(name)` ([app/services/draft_composer.py:147](#)): trả tên đã trim hoặc "điểm đến" mặc định.

class _Picker

Bộ chọn round-robin ưu tiên item CHƯA DỪNG gần nhất so với một con trỏ (cursor) đang di chuyển. Reset `_used` khi cạn pool để chuyển dài vẫn lấp đầy slot ([app/services/draft_composer.py:151](#)).

- `__init__(self, items)`: lưu `_items`, tính sẵn `_coords` cho từng item, `_used`: set rỗng.
- `next_near(origin, avoid_name=None)` ([app/services/draft_composer.py:163](#)): nếu `len(used) >= len(items)` thì `used.clear()` (vòng lại); chọn `nearest_index(origin, self._coords, self._used)`. Nếu trùng tên `avoid_name` và còn >1 item chưa dùng, đánh dấu `used` rồi chọn lại — tránh cùng-một-chỗ liên tiếp.
- `peek_alternative(origin)` ([app/services/draft_composer.py:187](#)): item chưa dùng gần thứ hai — đưa vào note như "lựa chọn khác". KHÔNG đánh dấu `used`.

class DraftComposer

Khởi tạo: `__init__(self, catalog: CatalogClient)` — giữ `self._catalog`.

`async def compose(self, constraints: Constraints) -> PlanDraft`

Hàm public chính: dựng toàn bộ lịch trình nhập.

- **Side effect / lời gọi ra ngoài**: gọi 4 lần HTTP tới `catalog-service` (qua `CatalogClient`, đều là POST JSON, không auth): `search_hotels(loc, page=0, size=8)`; `search_restaurants(loc, page=0, size=30)`; `search_restaurants(loc, page=0, size=20, cuisine="cafe")`; `search_places(destination, page=0, size=30)`. (Nhà hàng/khách sạn dùng `loc` đã slugify; places dùng `destination` thô.) Không ghi Mongo, không gọi LLM.
- **Lỗi/edge case**: nếu `not constraints.is_minimally_complete()` → `raise ValueError("Constraints must include destination, start_date, end_date")` ([app/services/draft_composer.py:201](#)). Bên gọi (`_deterministic_fallback`) bọc `compose` trong try/except để xử lý catalog rỗng.

Phần thiết lập ([app/services/draft_composer.py:203](#)–[app/services/draft_composer.py:238](#)):

- `pace` không hợp lệ → ép về "balanced". `resolved_date_range()` neo start vào hôm nay nếu chỉ có `num_days.nights = max(1, duration - 1)`.
- `location_slug` được **import trễ** trong thân hàm ([app/services/draft_composer.py:216](#)) để tránh circular import với `app/agent/tools.py`.

- Khách sạn đầu tiên (`hotels[0]`) là "trục" hình học của mọi ngày; nếu rỗng, `hotel_name = "khách sạn"`, `hotel_coords = None`.
- `cafe_picker` fallback về `restaurants` khi không có cafe riêng.
- `warnings` (`app/services/draft_composer.py:232–app/services/draft_composer.py:238`): thêm cảnh báo cho từng pool rỗng.

Vòng lặp theo ngày (`app/services/draft_composer.py:243–app/services/draft_composer.py:305`):

- Mỗi ngày reset `cursor = hotel_coords`, `cursor_name = hotel_name`, `t = DAY_START`.
- **Ngày 1 + có hotel**: chèn hoạt động thuê xe máy CHO CẢ CHUYỂN (per vehicle), `bikes = pricing.bikes_for(travelers)`, `rental = MOTORBIKE_PER_DAY * bikes * duration`, type `TRANSPORT`; cộng vào `total_cost`.
- Lặp qua `slots`, gọi `self._build_slot(...)` cho từng slot; nếu trả `act is None` thì `continue`. `cursor/cursor_name/t` được chuyển qua lại giữa các slot.
- Đóng gói thành `DraftDay(day_index, day_date, title=_day_title(...), activities=acts)`.
- Sau vòng lặp: nếu `budget_total_vnd` được đặt và `total_cost > budget` → thêm warning vượt ngân sách.

Giá trị trả về — `PlanDraft` (`app/models/session.py:139`) với các field: `draft_id` (UUID tự sinh), `summary` ("Lịch trình {duration} ngày tại {destination} cho {travelers} người (nhịp {pace})"), `destination`, `start_date`, `end_date`, `travelers`, `budget_currency="VND"`, `estimated_total_cost_vnd=total_cost`, `days: List[DraftDay]`, `warnings: List[str]`.

```
def _build_slot(self, *, ...) -> Tuple[Optional[DraftActivity], int, ..., time]
```

Dựng MỘT slot (`app/services/draft_composer.py:348`). Trả tuple (`activity|None, cost, new_cursor, new_cursor_name, new_time`). Nguyên tắc trung tâm: **giờ bắt đầu = max(Thời gian nối chuỗi di chuyển, slot anchor)** qua `_clamp_anchor(_add_minutes(t, travel), anchor)`. Pool catalog rỗng → tạo placeholder generic THAY VÌ bỏ slot (giữ độ dày & giờ giấc đúng). Tất cả tham số là keyword-only.

Ba nhánh theo `category`:

1. rest / sleep (`app/services/draft_composer.py:355–app/services/draft_composer.py:396`): di chuyển VỀ khách sạn. type `STAY`. `sleep`: "Về khách sạn nghỉ ngơi", `cost=0` (ngày cuối thêm "Trả phòng..."). `rest` ngày 1: "Nhận phòng + nghỉ trưa", `cost = pricing.lodging_total(hotel, nights, travelers)`, `quantity = pricing.rooms_for(travelers)` — ĐÂY là nơi tính tiền phòng cho cả chuyến. `rest` ngày khác: `cost=0`. Trả `new_cursor = hotel_coords`.

2. breakfast / cafe / lunch / dinner (`app/services/draft_composer.py:399–app/services/draft_composer.py:455`): chọn nhà hàng gần nhất, type `FOOD`. `label` map theo category. **Pool rỗng** (`item is None`): tạo "{label} (quán địa phương)" với giá ước tính từ dài VN, KHÔNG đổi `cursor`. Có item: tính `alt` (lựa chọn khác đưa vào note), `leg` (mặc định 12 phút nếu thiếu toạ độ), `per_person = pricing.meal_cost_per_person(item, category)`, `cost = pricing.total_for(per_person, travelers)`. `note` gồm "~{per_person:,}đ/người". Trả `new_cursor = _coords(item)`.

3. sight / evening (`app/services/draft_composer.py:458–app/services/draft_composer.py:500`): chọn địa điểm gần nhất từ `place_picker`. Pool rỗng: "{prefix}: Tự do khám phá khu vực", type `OTHER`, `cost=0`. Có item: `per_person = pricing.place_cost_per_person(item)` (có thể 0 cho điểm miễn phí); `atype = ENTERTAIN` nếu `evening`, ngược lại `SIGHTSEEING`. Tiêu đề `evening` gắn nhãn theo GIỜ THỰC qua `_time_prefix(start)`.

Nhánh không khớp category nào → return `None, 0, cursor, cursor_name, t` (`app/services/draft_composer.py:502`).

Hàm helper dựng dữ liệu

- `_day_title(day_index, duration, destination)` (`app/services/draft_composer.py:505`): ngày 1 → "Ngày 1 — Đến {destination}, nhận phòng & khám phá"; ngày cuối → "...Khám phá & trả phòng"; giữa → "...Khám phá {destination}".
- `_rec(kind, item) -> Optional[RecommendationRef]` (`app/services/draft_composer.py:513`): dựng `RecommendationRef` từ item catalog. `kind ∈ "HOTEL" | "RESTAURANT" | "PLACE"`. Map: `catalog_id=str(id)`, `catalog_slug=slug`, `name`, `lat/lng`, `cover_image_url=coverImageUrl`, `avg_rating=avgRating`, `estimated_cost_vnd = minNightlyPrice or minPricePerPerson`. **Lỗi**: bọc try/except, item dị dạng → log warning "skip malformed catalog item" và trả `None`.
- `_activity(*, ...)` (`app/services/draft_composer.py:534`): factory tạo `DraftActivity`. `end = _add_minutes(start, duration)` if `duration` else `start`. `duration_minutes = max(0, duration)`, `quantity = max(1, quantity)`, giờ format qua `_hhmm`.

Phụ thuộc định giá & địa lý

- `app/services/pricing.py` — định giá per-person, ưu tiên: (1) giá catalog cụ thể, (2) `priceLevel` map sang dài VND (`_PRICE_LEVEL_MULT`, `app/services/pricing.py:32`), (3) heuristic theo category (`_BANDS`, `app/services/pricing.py:20`). Hằng số: `MOTORBIKE_PER_DAY = 130_000` (`app/services/pricing.py:40`). `rooms_for/bikes_for = ceil(travelers/2)`. `lodging_total` mặc định 800_000đ/đêm khi catalog thiếu giá (`app/services/pricing.py:111`). `total_for(unit, travelers) = unit * max(1, travelers)`.
- `app/services/geo.py` — `leg_between(a, b)` trả `Leg(distance_km, minutes, mode, distance_text)` hoặc `None` nếu thiếu toạ độ; chọn mode theo ngưỡng km (đi bộ ≤0.8km, xe máy ≤18km, ô tô >18km); `nearest_index(origin, candidates, used)` trả index gần nhất chưa dùng, fallback về first-unused khi thiếu toạ độ. Tốc độ door-to-door bảo thủ: `_BIKE_KMH=22`, `_CAR_KMH=28`, `_INTERCITY_KMH=45`.

Cấu trúc dữ liệu / enum liên quan

- `PlanActivityType` (`app/models/session.py:9`): enum string `TRANSPORT, FOOD, STAY, ENTERTAIN, SIGHTSEEING, EVENT, SHOPPING, CINEMA, OTHER`. Composer dùng `TRANSPORT, STAY, FOOD, ENTERTAIN, SIGHTSEEING, OTHER`.
- `DraftActivity` (`app/models/session.py:95`): có validator `_validate_non_negative_cost` (`app/models/session.py:120`) — `cost` âm → **raise ValueError**. Vì vậy `_activity` không bao giờ được truyền `cost` âm.
- `RecommendationRef` (`app/models/session.py:80`): `model_config = ConfigDict(coerce_numbers_to_str=True)`.

Tổng hợp "gotcha" của DraftComposer

- Anchor che giấu thời gian di chuyển**: nếu buổi sáng ít điểm dừng, `_clamp_anchor` đẩy slot về anchor — giờ đề xuất có thể RỘNG RÃI hơn thời gian di chuyển thực tế (đúng theo thiết kế).
- _add_minutes cap 23:59**: một leg cực dài cuối ngày sẽ ép giờ về 23:59 chứ không wrap — nhưng các slot quá-muộn bị dồn vào cùng mốc 23:59.

3. **Import** `location_slug` trong `compose` để né circular import.
4. **Khách sạn = trực hình học**: chỉ dùng `hotels[0]`; chất lượng cụm geo phụ thuộc hoàn toàn vào việc catalog trả khách sạn có toạ độ.
5. **Pool rỗng KHÔNG bỏ slot**: luôn sinh placeholder generic để giữ độ dày và giờ giấc; thay vào đó phát warning ở cấp draft.
6. **Cost âm bị chặn ở model layer** (`DraftActivity` validator), không ở composer.

Approval Service & các tiện ích (geo/haversine, pricing, catalog_location)

Cụm module này gồm bốn file trong `app/services/`. `app/services/approval_service.py` là **nơi DUY NHẤT** trong `ai-plan-service` thực hiện ghi dữ liệu (write) ở luồng plan mode — nó chuyển một `PlanDraft` đã được người dùng phê duyệt thành một plan thật bên `plan-service`. Ba file còn lại là các tiện ích thuần (pure utilities): `app/services/geo.py` (tính khoảng cách/thời gian di chuyển), `app/services/pricing.py` (ước tính chi phí theo đầu người), và `app/services/catalog_location.py` (ánh xạ một mục catalog vào shape lưu trữ của modal frontend). `approval_service.py` phụ thuộc trực tiếp vào `catalog_location.py`.

`app/services/approval_service.py`

Trách nhiệm tổng quát. Convert một `PlanDraft` (đã approve) thành plan thật qua `plan-service`. Đây là điểm write duy nhất của service, luôn chạy phía sau một lời gọi `POST .../approve` tường minh. Mỗi write sang `plan-service` mang theo một **idempotency key** dẫn xuất từ `draft_id/plan_id` để retry không tạo trùng entity (docstring đầu file, `app/services/approval_service.py:1`).

Hàng số & helper idempotency.

- `_IDEM_NAMESPACE` (`app/services/approval_service.py:22`) là namespace UUID cố định cho `uuid5`. Mục đích là làm cho khóa **deterministic**.
- `_idem_key(seed)` (`app/services/approval_service.py:25`) sinh một UUID5 36 ký tự từ `seed`. Khóa 36-char vừa khít cột `operation_id` của `plan-service`. Vì deterministic, retry cùng `seed` cho cùng key → `plan-service` chống trùng.
- `fresh_idempotency_key()` (`app/services/approval_service.py:229`): trả `str(uuid4())` — một khóa NGẪU NHIÊN (không deterministic), helper public không được `approve()` dùng nội bộ.

Hàm `ordered_day_lists(board)`

Chữ ký: `ordered_day_lists(board) -> List[Dict[str, Any]]` (`app/services/approval_service.py:29`). Trích các list loại `DAY` ra khỏi một `BoardResponse` rồi sắp xếp theo thứ tự bảng/thời gian:

- Đầu vào: `board` (dict) có key `lists`. Nếu `board` không phải dict hoặc `lists` không phải list → trả `[]` (guard ở :35).
- Lọc chỉ giữ phần tử có `type == "DAY"` (loại bỏ list `TRASH` ở cuối).
- Sắp xếp ưu tiên theo `position` (None → 9999, tức đẩy xuống cuối), rồi theo `dayDate` (None → chuỗi rỗng).
- Mục đích: draft day thứ `i` ánh xạ vào day-list được seed thứ `i`.

Class `ApprovalService`

Khởi tạo: `__init__(self, plan_client: PlanClient)` (`app/services/approval_service.py:51`) — lưu `self._plan_client`. Đây là dependency duy nhất; mọi I/O ra ngoài đi qua client này.

`async def approve(self, draft, bearer_token) -> Dict[str, Any]` (`app/services/approval_service.py:54`)

Entrypoint chính. Luồng đầy đủ:

1. Tạo plan. Dựng `plan_body` rồi gọi `create_plan`:

- `title` fallback sang `"AI Plan - {destination}"` khi `summary` rỗng (:56).
- `startDate/endDate` được `.isoformat()` → chuỗi `YYYY-MM-DD`.
- `visibility` luôn `"PRIVATE"` (hardcode :61).
- `budgetPerPerson`: phép chia nguyên `estimated_total_cost_vnd // travelers`; nếu `travelers` là 0/falsy → 0 (:64).
- **Side effect / call ra ngoài**: HTTP POST tới `plan-service` (create plan). Nếu response thiếu `id` → raise `RuntimeError("plan-service did not return a plan id")` (:71).

2. Đọc board, KHÔNG tạo list mới. Gotcha quan trọng (comment :73-76): `plan-service` **tự seed sẵn MỘT list DAY cho mỗi ngày** trong `[startDate, endDate]` ngay lúc create. Nếu code này tự tạo list, sẽ phát sinh "phantom" ngày dư và trả `id` null. Do đó nó **đọc board** (`get_board` → HTTP) và **gắn card vào các DAY list đã seed**, khớp với draft day theo thứ tự thời gian. Edge case: nếu số day-list seed < số ngày của draft → chỉ log warning, các ngày dư sẽ bị **bỏ qua** (không raise).

3. Lặp qua từng ngày của draft (:88):

- `if index >= len(day_lists): break` — dừng khi hết day-list seed.
- Lấy `list_id` từ day-list thứ `index`. Nếu thiếu `id` → log error + raise `RuntimeError` (:96). Đây là trường hợp DUY NHẤT việc thiếu id của list làm fail toàn bộ approve.
- **Rename list (cosmetic, không bao giờ fail approve)**: list seed mang title generic; rename thành `day.title`. Mọi exception bị nuốt (chỉ `logger.warning`) — rename là cosmetic (:105). Ghi log `{"op": "use_list", "list_id", "day"}`.
- **Tạo card cho từng activity (:109)**: `card_idem = _idem_key(f"plan:{plan_id}:card:{day.day_index}:{position}")` (:110) — idempotency key deterministic theo (day, position). Gọi `create_card` (HTTP POST, truyền `idempotency_key`). **Xử lý lỗi**: `create_card` lỗi → log error + raise `RuntimeError` với thông điệp **tiếng Việt** kèm tên card, ngày, vị trí (:124-127). Khác với rename, lỗi tạo card LÀ lỗi nghiêm trọng.

4. Trả về. Giá trị trả về: dict `{"plan_id": <str>, "operations": <List[Dict]>}`. Mỗi phần tử `operations` là một trong hai shape: `{"op": "use_list", "list_id", "day"}` hoặc `{"op": "create_card", "idempotency_key", "day"}`.

Hàm `activity_to_card_body(activity, travelers=1)`

Chữ ký: `_activity_to_card_body(activity, travelers=1) -> Dict[str, Any]` ([app/services/approval_service.py:138](#)). Dựng body cho `CreateCardRequest`. Đây là logic biên dịch metadata AI thành shape mà `plan-service` + frontend hiểu.

Tính số lượng & đơn giá:

- `travelers` và `quantity` được clamp tối thiểu 1.
- Nếu không có `unit_price_vnd` nhưng có `estimated_cost_vnd` → suy ra đơn giá = `estimated_cost_vnd // quantity` (:150–151).

activity_data (metadata AI, sẽ JSON-hóa vào `activityDataJson`): gồm `reason`, `locationName`, `address`, `note`, `routeHint`, `distanceText`, `transportMode`, `ticketPrice` (= `unit_price`), `ticketCount` (= `quantity`).

- Gotcha lọc bằng if v:** dùng `truthiness`, nên không chỉ `None` mà cả `0/""` cũng bị loại (:165). Ví dụ `ticketPrice = 0` sẽ KHÔNG được giữ.
- Cost được gửi **cả hai chiều**: tổng (`cost.estimatedCost`) và per-unit (`ticketPrice × ticketCount`), để card hiển thị đúng "giá/lượt" và "số lượt/người tham gia".

Khối recommendation: khi activity gắn mục catalog thật (:169–178): chuẩn hóa recommendation thành dict với key

`kind/id/slug/name/latitude/longitude/coverImageUrl/avgRating`. **Lời gọi sang `catalog_location.py`:**

`catalog_location_fields(activity.activity_type.value, recommendation, address=activity.address)` thêm các key giúp modal frontend pre-select + auto-focus mục catalog. Lưu ý truyền `activity.activity_type.value` (giá trị enum, không phải enum object).

Description người-đọc-được (:189–190): gộp `description` + 📍 `address` + 💡 `note` + 🗺️ `route_hint` (`distance_text`) – `transport_mode`. Mục đích: card body hiển thị thông tin hữu ích NGAY CẢ trước khi frontend render JSON.

Body cuối: `text`, `description`, `activityType`, `activityDataJson` (chuỗi JSON, `ensure_ascii=False` để giữ tiếng Việt — :211), `durationMinutes`, `participantCount` (số khách của trip, hiển thị "👥 N người"), `startTime/endTime` (chỉ thêm khi có, qua `_normalize_time`), `cost` (chỉ thêm khi `estimated_cost_vnd` truthy; gồm `currencyCode="VND"`, `estimatedCost`, `participantCount`).

Hàm `_normalize_time(value)`

Chữ ký: `_normalize_time(value) -> str` ([app/services/approval_service.py:233](#)). `plan-service` deserialize `LocalTime` dạng `HH:mm` hoặc `HH:mm:ss`. Hàm chuẩn hóa về `HH:mm` có số 0 đầu: `"9:00" → "09:00"`. **Edge case / fallback:** nếu `value` không phải `str`, hoặc thiếu `:`, hoặc parse int thất bại → trả nguyên `value`. Giấy bị bỏ (chỉ lấy 2 phần đầu).

Logger. `logger = logging.getLogger("ai_plan.approval")` ([app/services/approval_service.py:47](#)).

`app/services/geo.py`

Trách nhiệm tổng quát. Các helper địa lý cho việc lập lịch itinerary: tính khoảng cách great-circle (haversine), chọn phương tiện theo cự ly, ước tính thời gian di chuyển, tìm điểm gần nhất (nearest-neighbour), và tính các "leg" cho cả deterministic composer lẫn LLM agent (gián tiếp qua tool `route_legs`). **Toàn bộ là pure + deterministic** ([app/services/geo.py:1–app/services/geo.py:10](#)).

Hàng số tốc độ (km/h, door-to-door): `_WALK_KMH = 4.5`, `_BIKE_KMH = 22.0` (xe máy in mixed city traffic), `_CAR_KMH = 28.0` (taxi/ô tô), `_INTERCITY_KMH = 45.0` (long highway legs) ([app/services/geo.py:20–app/services/geo.py:23](#)). Cố ý đặt thận trọng để lịch có slack.

Ngưỡng cự ly chọn phương tiện: `_WALK_MAX_KM = 0.8`, `_BIKE_MAX_KM = 18.0` ([app/services/geo.py:26–app/services/geo.py:27](#)).

@dataclass `Leg` ([app/services/geo.py:30–app/services/geo.py:35](#))

Field: `distance_km: float`, `minutes: int`, `mode: str` ("đi bộ" | "xe máy" | "taxi" | "ô tô"), `distance_text: str` ("500m", "~2km", "30km"). Lưu ý comment liệt kê "taxi" như mode hợp lệ, nhưng `pick_mode` chỉ trả "đi bộ"/"xe máy"/"ô tô" — taxi chỉ xuất hiện nếu caller truyền `mode="taxi"` thủ công.

`haversine_km(a, b)` ([app/services/geo.py:38](#))

Khoảng cách great-circle (km) giữa hai cặp (`lat`, `lng`). Bán kính Trái Đất `r = 6371.0`. **Edge case:** trả `None` nếu một trong hai điểm là falsy, hoặc bất kỳ tọa độ nào là `None`. `min(1.0, sqrt(h))` chặn lỗi domain của `asin` do sai số dấu phẩy động.

`format_distance(km)` ([app/services/geo.py:54](#))

`None` → `""`. `< 1km` → mét làm tròn bội số 50 (vd "500m"). `< 10km` → `"~X.Xkm"`. `>= 10km` → `"Xkm"` làm tròn nguyên.

`pick_mode(km)` ([app/services/geo.py:64](#))

`km <= 0.8` → "đi bộ"; `km <= 18.0` → "xe máy"; còn lại → "ô tô".

`travel_minutes(km, mode=None)` ([app/services/geo.py:72](#))

- Nếu `mode` không truyền → suy ra qua `pick_mode`.
- Gotcha tốc độ intercity:** khi `mode == "ô tô"` mà `km > 18` thì dùng `_INTERCITY_KMH=45` (highway) thay vì `_CAR_KMH=28`. Điều này áp dụng cả khi caller truyền tay `mode="taxi"` (vì "taxi" không khớp 2 nhánh đầu, rơi vào nhánh `km > _BIKE_MAX_KM`).
- Overhead cố định: 5 phút (đi bộ) hoặc 8 phút (còn lại). Tối thiểu 5 phút cho mọi hop (`max(5, ...)`). Trả về số phút nguyên (int).

`leg_between(a, b, mode=None)` ([app/services/geo.py:88](#))

Tính một `Leg` đầy đủ giữa hai tọa độ. Trả `None` nếu tọa độ chưa biết. `distance_km` được `round(km, 2)`.

`nearest_index(origin, candidates, used)` ([app/services/geo.py:106](#))

- Trả index của candidate **chưa dùng** (không nằm trong **used**) gần **origin** nhất.
- **Fallback quan trọng**: khi không tính được khoảng cách (tọa độ thiếu), trả về **first_unused** — index chưa-dùng đầu tiên — để caller **LUÔN** tiến triển được. Trả **None** chỉ khi mọi candidate đều đã dùng.

route_legs(stops) (app/services/geo.py:128)

Tính per-leg cho danh sách stops đã sắp thứ tự; trả một entry mỗi leg ($\text{len}(\text{stops}) - 1$ entry). Field mỗi leg: **from**, **to**, **distance_km**, **distance_text**, **minutes**, **transport_mode**, **route_hint**. Khi tọa độ thiếu (**leg is None**), các field số/mode đều **None** nhưng **from/to/route_hint** vẫn có. Được **tool route_legs** của LLM agent dùng để có khoảng cách thật thay vì đoán.

Lưu ý: không có call ra ngoài (HTTP/Mongo/LLM) trong **geo.py** — toàn bộ là tính toán cục bộ.

app/services/pricing.py

Trách nhiệm tổng quát. Ước tính chi phí **có nhận biết theo đầu người** (per-person aware) và ưu tiên giá thật khi có. Thứ tự ưu tiên (**app/services/pricing.py:1–app/services/pricing.py:12**): (1) giá catalog cụ thể (restaurant **minPricePerPerson**, hotel **minNightlyPrice**, place **minPrice**); (2) gợi ý **priceLevel** từ catalog map sang dải VND; (3) heuristic band theo category (KHÔNG flat 100k), phản ánh giá VN 2024-2025. Mọi chi phí ăn/vé/trải nghiệm nhân với số khách. Lưu trú tính theo PHÒNG (≈ 2 khách/phòng) $\times$ số đêm.

Hàng số dữ liệu.

- **BANDS** (**app/services/pricing.py:20–app/services/pricing.py:29**): mỗi band là tuple (**low**, **typical**, **high**) VND per-person cho **breakfast**, **cafe**, **lunch**, **dinner**, **streetfood**, **seafood**, **entrance**, **experience**.
- **_PRICE_LEVEL_MULT** (**app/services/pricing.py:32–app/services/pricing.py:37**): map **priceLevel** (enum catalog HOẶC số \$) $\rightarrow$ hệ số nhân: **budget/economy/\$** $\rightarrow 0.6$, **moderate/\$\$** $\rightarrow 1.0$, **expensive/\$\$\$** $\rightarrow 1.6$, **luxury/\$\$\$\$** $\rightarrow 2.4$.
- Hàng số transport (**app/services/pricing.py:40–app/services/pricing.py:43**): **MOTORBIKE_PER_DAY** = 130_000 (mỗi xe/ngày), **TAXI_SHORT** = 60_000, **TAXI_MEDIUM** = 150_000, **INTERCITY_BUS_PER_PERSON** = 120_000.

rooms_for(travelers) / bikes_for(travelers) (app/services/pricing.py:46–app/services/pricing.py:51)

Cùng công thức: $\text{ceil}(\text{travelers} / 2)$, tối thiểu 1. Tức ≈ 2 khách/phòng và ≈ 2 khách/xe.

_num(value) -> Optional[float] (app/services/pricing.py:54)

Ép kiểu sang float **đương**. **Gotcha**: chỉ trả giá trị nếu > 0 ; giá 0 hoặc âm $\rightarrow$ **None** (coi như "không có giá").

_band_value(category, level) -> int (app/services/pricing.py:64)

Lấy band theo **category**; nếu category lạ $\rightarrow$ **fallback band "lunch"**. Nếu có **level** và map được $\rightarrow$ trả $\text{int}(\text{mid} * \text{mult})$. Nếu **level** không khớp key nào $\rightarrow$ trả **mid** (typical). **str(level).strip().lower()** cho phép khớp cả enum chữ và ký tự \$/số.

meal_cost_per_person(item, meal_type) -> int (app/services/pricing.py:73)

Ưu tiên giá catalog: nếu có cả **lo** và **hi** $\rightarrow$ trung bình; chỉ có **lo** $\rightarrow$ **lo**; không có giá nhưng có **price_level/priceLevel** $\rightarrow$ band theo level; không gì cả $\rightarrow$ band typical của **meal_type**. **Hỗ trợ hai naming**: cả snake_case (**min_price_per_person_vnd**) lẫn camelCase (**minPricePerPerson**).

place_cost_per_person(item) -> int (app/services/pricing.py:88)

Gotcha: nếu không có giá và không có **price_level** $\rightarrow$ trả 0 (free spot: bãi biển, chùa, viewpoint). Khác với meal (luôn có band fallback), place mặc định **MIỄN PHÍ** khi catalog không cho tín hiệu có-thu-phí.

total_for(unit_per_person, travelers) -> int (app/services/pricing.py:101)

$\text{int}(\text{unit_per_person}) * \max(1, \text{travelers})$.

lodging_total(item, nights, travelers) -> int (app/services/pricing.py:105)

Giá đêm lấy từ catalog (**min_nightly_price_vnd/minNightlyPrice**); nếu thiếu $\rightarrow$ **fallback 800_000 VND** (3-star tầm trung). Tổng = **nightly** $\times$ **nights** $\times$ **rooms_for(travelers)**. Lưu trú tính per-ROOM, KHÔNG per-person. **Không có call ra ngoài** — pure utility.

app/services/catalog_location.py

Trách nhiệm tổng quát. Ánh xạ một catalog recommendation (hotel/restaurant/place) vào **shape lưu trữ của modal activity** ở frontend, để place picker có thể pre-select VÀ auto-focus mục đó — giống hệt khi người dùng chọn tay (**app/services/catalog_location.py:1–app/services/catalog_location.py:17**). Shape đích: **activityData.<locationKey>** = {**type**, **placeId**, **name**, **fullAddress**, **lat**, **lng**} cộng key tên hiển thị theo loại. Picker key các hàng theo **id** (rồi **slug**), nên **placeId** phải là catalog id (ưu tiên) hoặc slug. Thiếu các key này, modal mở với ô tên rỗng và không có selection.

Hàng số ánh xạ.

- **_ACTIVITY_CATALOG_FIELDS** (**app/services/catalog_location.py:24–app/services/catalog_location.py:33**): map **PlanActivityType.value** $\rightarrow$ tuple (**location key** modal đọc, **display-name key** modal đọc). **STAY** $\rightarrow$ (hotelLocation, hotelName), **FOOD** $\rightarrow$ (restaurantLocation, restaurantName), **SIGHTSEEING/ENTERTAIN** $\rightarrow$ (...Location, placeName), **SHOPPING** $\rightarrow$ (shoppingLocation, storeName), **EVENT** $\rightarrow$ (eventLocation, eventName), **CINEMA** $\rightarrow$ (cinemaLocation, cinemaName), **OTHER** $\rightarrow$ (otherLocation, locationText). **TRANSPORT** **có ý bị loại** (nó có cặp from/to, không phải một venue).
- **_PICKER_TYPES** = {"HOTEL", "RESTAURANT", "PLACE"}, **_DEFAULT_TYPE_BY_ACTIVITY** = {"STAY": "HOTEL", "FOOD": "RESTAURANT"}, **_ACTIVITY_BY_KIND** = {"HOTEL": "STAY", "RESTAURANT": "FOOD", "PLACE": "SIGHTSEEING"} (**app/services/catalog_location.py:36–**

[app/services/catalog_location.py:43](#)).

`catalog_location_fields(activity_type, recommendation, address=None)` ([app/services/catalog_location.py:46](#))

- **Edge case:** `recommendation` không phải dict hoặc rỗng $\rightarrow \{\}$ (:59). Lấy `kind` (uppercase), giải `resolved_activity`. Nếu không resolve được hoặc activity không có trong `_ACTIVITY_CATALOG_FIELDS` $\rightarrow \{\}$ (vd `TRANSPORT`).
- `place_id`: ưu tiên `id`, fallback `slug`. **Gotcha ép str** (:69–71): coerce sang `str` vì frontend so khớp `selectedId === item.id` dạng chuỗi; id số legacy sẽ không bao giờ match nếu để nguyên. Nếu không có cả `place_id` lẫn `name` $\rightarrow \{\}$.
- `loc_type`: nếu `kind` $\in \{\text{HOTEL, RESTAURANT, PLACE}\}$ $\rightarrow$ dùng `kind`; ngược lại fallback theo `_DEFAULT_TYPE_BY_ACTIVITY`, mặc định "PLACE".
- `location` lọc bỏ giá trị `None` và "" — lưu ý chỉ loại 2 giá trị này, KHÔNG loại `0` (khác với truthiness lọc ở `approval_service`).
- **Giá trị trả về:** dict gồm key `location_key` (vd `hotelLocation`) $\rightarrow$ object `location`{`type`, `placeId`, `name`, `fullAddress`, `lat`, `lng`} (các field rỗng đã bị loại), CỘNG key `name_key` (vd `hotelName`) $\rightarrow$ `name` khi có `name`. Nếu không khớp gì $\rightarrow \{\}$.

`_resolve_activity(activity_type, kind)` ([app/services/catalog_location.py:99](#))

- Nếu CÓ `activity_type`: nó "thắng". Type biết-và-có-mapping $\rightarrow$ trả chính nó; type biết-nhưng-không-mapping (vd `TRANSPORT`) $\rightarrow$ `None`. Type lạ $\rightarrow$ `None`.
- Nếu THIẾU `activity_type`: suy ra từ `kind` qua `_ACTIVITY_BY_KIND` (`HOTEL` $\rightarrow$ `STAY`, `RESTAURANT` $\rightarrow$ `FOOD`, `PLACE` $\rightarrow$ `SIGHTSEEING`); `kind` lạ $\rightarrow$ `None`.

Quan hệ với `approval_service`. Hàm này được `_activity_to_card_body` gọi ([app/services/approval_service.py:181](#)–[app/services/approval_service.py:187](#)) với `activity.activity_type.value` và dict `recommendation` đã chuẩn hóa; kết quả được `activity_data.update(...)` để frontend pre-fill modal. **Không có call ra ngoài** — pure utility.

Tổng kết "gotchas" toàn cục

- `approval_service.approve` KHÔNG tạo list — chỉ dùng các DAY list mà `plan-service` đã seed sẵn; ngày dư bị bỏ qua (chỉ warning), ngược lại list-thiếu-`id` thì raise.
- Idempotency: card key deterministic theo (`plan_id`, `day_index`, `position`) $\rightarrow$ retry an toàn; `fresh_idempotency_key()` ngược lại là random.
- Lỗi rename list bị nuốt (cosmetic); lỗi create card thì raise (thông điệp tiếng Việt).
- `_activity_to_card_body` lọc `activity_data` bằng truthiness $\rightarrow$ loại cả `0/""`, trong khi `catalog_location_fields` chỉ loại `None/""`.
- `pricing._num` coi giá ≤ 0 là "không có giá"; `place_cost_per_person` mặc định `0` (miễn phí) còn meal luôn có band fallback; lodging fallback `800_000`/đêm/phòng.
- `geo.travel_minutes` dùng tốc độ intercity (45 km/h) cho mọi mode "ô tô"/"taxi" khi `km > 18`; `nearest_index` luôn fallback về index chưa-dùng đầu tiên để caller không kẹt.
- `catalog_location` cố ý loại `TRANSPORT`; ép `placeId` về `str` để khớp so sánh chuỗi ở frontend.

Models phiên & Session Store (InMemory/File/Mongo + factory)

Cụm module này gồm hai file: [app/models/session.py](#) định nghĩa toàn bộ các Pydantic model mô tả một phiên hội thoại lập kế hoạch (constraints, draft itinerary, message lịch sử, trạng thái phiên), còn [app/services/session_store.py](#) cung cấp lớp lưu trữ phiên với ba backend hoán đổi được (in-memory, file JSON, MongoDB) cùng một factory chọn backend theo cấu hình. Hai file phối hợp chặt: `PlanSession` (model) là đơn vị dữ liệu mà mọi store tuần tự hóa/giải tuần tự hóa qua `model_dump_json()` và `PlanSession.model_validate()`.

[app/models/session.py](#) — Các Pydantic model của phiên

Trách nhiệm tổng quát: định nghĩa toàn bộ schema dữ liệu cho domain "AI plan session". Đây là các model thuần (không có I/O, không gọi mạng), chỉ chứa validation logic cục bộ (`model_validator`) và vài helper tính toán trên `Constraints`. Tất cả model kế thừa `pydantic.BaseModel` nên đều có `model_dump_json()` / `model_validate()` mà session store dựa vào để serialize.

Enums và hằng số

Các enum đều là `str`, `Enum` (giá trị string khớp đúng tên), để JSON tuần tự hóa thành chuỗi và khớp với phía Java/`plan-service`:

- `PlanActivityType` ([app/models/session.py:9](#)) — phản chiếu (mirror) `com.mravel.plan.model.PlanActivityType` bên Java. Các giá trị: `TRANSPORT`, `FOOD`, `STAY`, `ENTERTAIN`, `SIGHTSEEING`, `EVENT`, `SHOPPING`, `CINEMA`, `OTHER`. Đây là điểm đồng bộ liên-service: nếu enum Java đổi thì file này phải đổi theo.
- `PlanVisibility` ([app/models/session.py:23](#)) — `PUBLIC`, `PRIVATE`, `FRIENDS`. (Lưu ý: enum này được khai báo nhưng không được tham chiếu bởi model nào khác trong cùng file — nó là kiểu dùng chung cho các module khác import từ đây.)
- `ChatRole` ([app/models/session.py:152](#)) — `USER`, `ASSISTANT`.
- `SessionStatus` ([app/models/session.py:163](#)) — `DRAFTING`, `APPROVED`. Trạng thái vòng đời của một phiên.

`class Constraints` ([app/models/session.py:29](#))

Chữ ký các field: `destination: Optional[str]`, `start_date/end_date: Optional[date]`, `num_days: Optional[int]`, `travelers: int = 2`, `budget_total_vnd: Optional[int]`, `interests: List[str]`, `pace: Optional[str]` (theo quy ước chỉ nhận "relaxed" | "balanced" | "packed").

Trách nhiệm: gom các ràng buộc do người dùng cung cấp dần qua hội thoại. `num_days` là field then chốt cho phép lập kế hoạch ngay khi người dùng chỉ nói "đi 3 ngày 2 đêm" mà không nêu ngày cụ thể — nó neo ngày bắt đầu vào hôm nay.

Validation tự động — `_validate_date_range` ([app/models/session.py:42](#)), chạy `mode="after"`: Gotcha quan trọng: khoảng ngày bị đảo (`end_date < start_date`) KHÔNG raise lỗi mà tự HOÁN ĐỔI hai field. Lý do: `Constraints` được dựng tăng dần trong `apply_set_constraints` / `constraint_extractor`, nên raise giữa chừng sẽ làm vỡ luồng cập nhật constraint. Việc validate cứng, hướng người dùng, được dời sang `finalize_draft`.

Các method public:

- `is_minimally_complete(self)` $\rightarrow$ bool ([app/models/session.py:57](#)): trả `True` khi có `destination` VÀ (có cả `start_date+end_date` HOẶC có `num_days`).

- `duration_days(self) -> int` ([app/models/session.py:63](#)): ưu tiên tính từ khoảng ngày `((end_date - start_date).days + 1, kếp tối thiểu 1)`, nếu không có thì lấy `num_days` (kếp tối thiểu 1), cuối cùng fallback về 1.
- `resolved_date_range(self) -> tuple[date, date]` ([app/models/session.py:70](#)): trả `(start, end)` cụ thể. Nếu có cả hai ngày → trả nguyên. Nếu không → `start = self.start_date or date.today()` và `end = start + timedelta(days=duration_days() - 1)`. Đây là chỗ "neo vào hôm nay" cho trường hợp chỉ có `num_days`. Gotcha: phụ thuộc `date.today()` nên KHÔNG thuần (kết quả thay đổi theo ngày chạy nếu chỉ có `num_days`).

`class RecommendationRef` ([app/models/session.py:80](#))

Tham chiếu tới một mục trong catalog (place/restaurant/hotel) mà AI gợi ý gắn vào một activity. Field: `catalog_kind` ("PLACE" | "RESTAURANT" | "HOTEL"), `catalog_id`, `catalog_slug`, `name`, `latitude`, `longitude`, `cover_image_url`, `avg_rating`, `estimated_cost_vnd`.

Gotcha: `model_config = ConfigDict(coerce_numbers_to_str=True)` ([app/models/session.py:82](#)) — catalog trộn lẫn ObjectId dạng string của Mongo và id số (legacy), nên cấu hình này ép mọi số về `str` để `catalog_id/catalog_slug` luôn là chuỗi. `catalog_kind` chỉ là `str` (không enum).

`class DraftActivity` ([app/models/session.py:95](#))

Một hoạt động trong itinerary nháp. Các field chính: `title`, `description=""`, `day_index`, `start_time/end_time` (string "HH:MM"), `duration_minutes=60`, `activity_type: PlanActivityType = OTHER`, `estimated_cost_vnd=0`, `unit_price_vnd: Optional[int]=None`, `quantity=1`, `reason=""`, `recommendation: Optional[RecommendationRef]=None`. Nhóm field enrichment: `location_name`, `address`, `note`, `route_hint`, `distance_text`, `transport_mode`.

Quy ước chi phí: `estimated_cost_vnd` là tổng = `unit_price_vnd * quantity` khi cả hai được set; `quantity` là số người (suất ăn/vé) hoặc số đơn vị (phòng/xe).

Validation — `_validate_non_negative_cost` ([app/models/session.py:120](#)): Khác với `Constraints`, ở đây chi phí âm (`estimated_cost_vnd < 0` hoặc `unit_price_vnd < 0`) bị RAISE `ValueError` (không sửa lạng lẽ), vì chi phí âm luôn không hợp lệ. Đây là edge case có thể làm `PlanSession.model_validate()` ném lỗi khi load lại session đã hỏng.

`class DraftDay` ([app/models/session.py:132](#))

Một ngày trong itinerary: `day_index: int`, `day_date: date`, `title: str`, `activities: List[DraftActivity]` (mặc định rỗng).

`class PlanDraft` ([app/models/session.py:139](#))

Bản nháp kế hoạch hoàn chỉnh đang chờ duyệt. Field: `draft_id` (tự sinh `uuid4()`), `summary=""`, `destination=""`, `start_date/end_date` (BẮT BUỘC, không default — khác với `Constraints` nơi chúng `Optional`), `travelers=2`, `budget_currency="VND"`, `estimated_total_cost_vnd=0`, `days: List[DraftDay]`, `warnings: List[str]` (do tăng planning dần).

`class ChatMessage` ([app/models/session.py:157](#))

`role: ChatRole`, `content: str`, `created_at: datetime` (mặc định `datetime.now(timezone.utc)` — timezone-aware UTC).

`class PlanSession` ([app/models/session.py:168](#)) — model trung tâm

Field then chốt:

- `session_id` — UUID tự sinh; làm khóa chính trong mọi store (và `_id` trong Mongo).
- `user_id: int` — BẮT BUỘC, dùng để kiểm tra quyền sở hữu (`get()` so khớp `user_id`).
- `status` — DRAFTING mặc định, chuyển APPROVED khi kế hoạch được duyệt.
- `constraints: Constraints` (default factory), `draft: Optional[PlanDraft]` (None khi chưa có), `messages: List[ChatMessage]`.
- `approved_plan_id` — id plan đã tạo bên `plan-service` sau khi duyệt.
- `plan_id + pending_edits` — chế độ EDIT: khi chat mở trên một plan có sẵn, `plan_id` được set và agent đọc/sửa plan đó thay vì sinh draft mới; `pending_edits` (`List[Dict[str, Any]]`) giữ các thao tác agent đề xuất đang chờ "Áp dụng".
- `created_at / updated_at` — timezone-aware UTC; `updated_at` được store cập nhật mỗi lần `save()`.

Đây là đối tượng được session store tuần tự hóa nguyên khối qua `model_dump_json()` và phục hồi qua `model_validate()`.

`app/services/session_store.py` — Lớp lưu trữ phiên (3 backend + factory)

Trách nhiệm tổng quát: cung cấp một giao diện thống nhất (`create / get / save / list_by_user`) với ba cài đặt backend, cùng factory `get_session_store()` chọn backend theo cấu hình và tự fallback khi Mongo không khả dụng. Docstring đầu file ([app/services/session_store.py:1](#)) ghi rõ: phiên nhỏ (~vài KB), single-tenant per-process, chỉ truy vấn theo `session_id`; file là đủ cho scope hiện tại, đổi sang Redis/Postgres khi cần concurrency.

Module dùng `logger = logging.getLogger("ai_plan.session_store")` ([app/services/session_store.py:26](#)) và import `DomainError` từ `app.core.errors` ([app/services/session_store.py:23](#)).

Lưu ý kiến trúc: KHÔNG có lớp base/ABC tường minh. `InMemorySessionStore` đóng vai trò interface chuẩn; `FileBackedSessionStore` kế thừa nó; còn `MongoSessionStore` là lớp ĐỘC LẬP (không kế thừa) chỉ cài lại đúng 4 method công khai. Đây là duck typing — annotation trả về của factory là `InMemorySessionStore` dù thực tế có thể trả `MongoSessionStore`.

`class InMemorySessionStore` ([app/services/session_store.py:29](#))

Backend mặc định cho test. Cấu trúc: `self._lock = RLock()` và `self._sessions: Dict[str, PlanSession] = {}`. Mọi thao tác đọc/ghi dict đều bọc trong `with self._lock:` để thread-safe.

- `create(self, user_id) -> PlanSession` ([app/services/session_store.py:34](#)): tạo `PlanSession(user_id=user_id)`, lưu vào dict dưới lock, trả về session.

- `get(self, session_id, user_id) -> PlanSession (app/services/session_store.py:40)`: lấy session. Nếu không có → `raise DomainError(f"Session not found: {session_id}")`. Nếu `session.user_id != user_id` → `raise DomainError("Session does not belong to current user")`. Đây là chốt kiểm soát quyền sở hữu.
- `save(self, session) -> PlanSession (app/services/session_store.py:49)`: set `session.updated_at = datetime.now(timezone.utc)` rồi ghi đè vào dict dưới lock.
- `list_by_user(self, user_id) -> List[PlanSession] (app/services/session_store.py:55)`: lọc các session thuộc user, sort theo `updated_at` GIẢM DẦN (mới nhất trước). Dùng cho danh sách lịch sử chat.

class FileBackedSessionStore(InMemorySessionStore) (app/services/session_store.py:64)

Kế thừa toàn bộ logic in-memory, BỔ SUNG persistence ra một file JSON. Đọc file một lần lúc khởi tạo, ghi lại file mỗi lần `save()/create()`. An toàn single-process.

Constructor `__init__(self, file_path) (app/services/session_store.py:71)`: gọi `super().__init__()`, lưu `self._file_path`, `mkdir(parents=True, exist_ok=True)`, rồi `self._load()`.

`_load(self) (app/services/session_store.py:77)` — đọc file lúc startup, đầy đủ xử lý lỗi: File không tồn tại → return. Đọc lỗi `OSError` → log warning, return rỗng (KHÔNG crash). JSON hỏng (`json.JSONDecodeError`) → log warning rồi ĐỔI TÊN file sang đuôi `.bad` (rotate), bắt đầu rỗng. Với mỗi `session_dict: PlanSession.model_validate(...)`; nếu một session hỏng → log warning "skip invalid persisted session" và bỏ qua (tiếp tục), KHÔNG làm hỏng cả lần load.

`_flush(self) (app/services/session_store.py:109)` — ghi nguyên trạng thái ra file bằng GHI NGUYÊN TỬ (atomic): tạo `snapshot` dưới lock (serialize từng session qua `model_dump_json()` rồi `json.loads()` lại để áp Pydantic JSON encoder); định dạng file `{"version": 1, "sessions": [...]}`; ghi tmp file CÙNG THƯ MỤC với file đích rồi `os.replace(tmp_path, self._file_path)` (atomic rename); `ensure_ascii=False, default=str`. Gotcha: flush thất bại chỉ log warning, KHÔNG raise — dữ liệu vẫn còn trong memory nhưng không bền sau restart.

`create (app/services/session_store.py:130)` và `save (app/services/session_store.py:135)` override: gọi `super()` rồi `self._flush()`. Gotcha hiệu năng: mỗi `save()` ghi LẠI TOÀN BỘ file — không tốt khi lịch sử lớn (chính lý do `MongoSessionStore` được khuyến nghị).

class MongoSessionStore (app/services/session_store.py:141)

Backend khuyến nghị. Mỗi session là một document keyed bởi `_id = session_id`, nên `save` chỉ là một upsert (không rewrite cả file). Có index theo user để list lịch sử nhanh. Lớp ĐỘC LẬP (không kế thừa InMemory).

Constructor `__init__(self, uri, db_name, collection, migrate_path=None) (app/services/session_store.py:149)`:

- Import `pymongo` BÊN TRONG hàm (lazy import) — để dependency chỉ cần khi backend này được dùng.
- `MongoClient(uri, serverSelectionTimeoutMS=3000, tz_aware=True)` — timeout chọn server 3s, `tz_aware=True` để datetime đọc ra có timezone.
- `self._client.admin.command("ping")` — fail fast nếu Mongo không tới được, để factory bắt exception và fallback.
- `create_index([("user_id", ASCENDING), ("updated_at", DESCENDING)])` — index phục vụ `list_by_user`.
- Nếu có `migrate_path` → gọi `_migrate_from_file(migrate_path)`.

Helper tĩnh: `_to_doc(session) (app/services/session_store.py:167)`: `doc = json.loads(session.model_dump_json())` rồi gán `doc["_id"] = session.session_id`. `_from_doc(doc) (app/services/session_store.py:173)`: `doc.pop("_id", None)` rồi `PlanSession.model_validate(doc)`.

`_migrate_from_file(self, path) (app/services/session_store.py:178)` — import một lần file JSON legacy khi collection còn rỗng: nếu `estimated_document_count() > 0` HOẶC file không tồn tại → return; đọc file (lỗi → log warning, return); `insert_many(docs, ordered=False)`; cuối: đổi tên file nguồn sang `.migrated.json` để wipe collection sau này không kích hoạt re-import.

Các method public (cùng chữ ký với InMemory):

- `create (app/services/session_store.py:204)`: `insert_one(self._to_doc(session))`.
- `get (app/services/session_store.py:209)`: `find_one({"_id": session_id})`; None → `DomainError("Session not found")`; lệch user → `DomainError("Session does not belong to current user")`.
- `save (app/services/session_store.py:218)`: set `updated_at` rồi `replace_one({"_id": ...}, doc, upsert=True)`.
- `list_by_user (app/services/session_store.py:225)`: `find({"user_id": user_id}).sort("updated_at", DESCENDING)`, map qua `_from_doc`.

Gotcha Mongo: `MongoSessionStore` KHÔNG bọc lock (an toàn nhờ Mongo atomic per-document). `save` dùng `upsert=True` nên cũng dùng được cho session chưa tồn tại. Không như file backend, save là upsert một document → không có chi phí rewrite-all.

Factory `get_session_store()` (app/services/session_store.py:235) và singleton

Biến module `_store_singleton (app/services/session_store.py:232)` giữ instance duy nhất.

Logic chọn backend (lazy: chỉ chạy lần đầu, cache vào singleton):

- Đọc cấu hình qua `from app.config import get_settings` (lazy import để tránh vòng phụ thuộc).
- `backend = (settings.session_store_backend or "mongo").strip().lower()` — MẶC ĐỊNH "mongo".
- Helper nội bộ `_file_or_memory()`: nếu có path → `FileBackedSessionStore(Path(path))`; ngược lại → `InMemorySessionStore()`.
- `backend == "memory"` → in-memory. `backend == "file"` → `_file_or_memory()`.
- Mặc định (else, gồm "mongo" và mọi giá trị lạ): thử `MongoSessionStore.migrate` được truyền chỉ khi `settings.session_migrate_from_file` bật VÀ có path. Nếu khởi tạo Mongo ném BẤT KỲ exception nào (ping thất bại, pymongo lỗi) → log warning và FALLBACK về `_file_or_memory()` (`app/services/session_store.py:267`).

Cấu hình (env/settings) mà module dùng tới: `session_store_backend ("memory" | "file" | "mongo", mặc định "mongo")`; `session_store_path` (đường dẫn file JSON; cũng dùng làm nguồn migrate); `session_migrate_from_file` (bật/tắt migrate một lần); `mongo_uri, mongo_db, session_collection`.

`reset_session_store_for_tests()` (`app/services/session_store.py:273`): test helper, set `_store_singleton = None` để mỗi test khởi đầu sạch.

Gotcha tổng quát của factory:

- Singleton tiến trình: backend được "đóng băng" sau lần gọi đầu; đổi env giữa chừng không có hiệu lực trừ khi reset.
- Fallback Mongo→file/memory là "âm thầm" (chỉ log warning) — nếu Mongo chết lúc khởi động, service vẫn chạy nhưng lịch sử có thể chỉ nằm in-memory (mất khi restart) nếu không cấu hình `session_store_path`.
- Annotation trả về `InMemorySessionStore` chỉ mang tính danh nghĩa (duck typing); thực tế có thể là `MongoSessionStore` vốn không kế thừa lớp đó.

Liên hệ với phần còn lại của service

`PlanSession` và họ model là hợp đồng dữ liệu giữa session store và tầng agent/orchestrator; mọi backend đều phụ thuộc trực tiếp `PlanSession.model_dump_json()` / `PlanSession.model_validate()` để round-trip. `DomainError` (từ `app/core/errors.py`) là lỗi domain duy nhất mà store ném ra (not-found / sai chủ sở hữu), để tầng API map sang phản hồi phù hợp.

Upstream Clients REST (catalog-service, plan-service, Tavily web search)

Cụm module này gồm ba HTTP client async (dựa trên `httpx.AsyncClient`) mà `ai-plan-service` dùng để gọi ra các service khác trong hệ thống Mavel cũng như dịch vụ web search bên ngoài:

- `CatalogClient` (`app/clients/catalog_client.py`) — client **read-only**, **không cần auth**, để tra cứu địa điểm (places), nhà hàng (restaurants), khách sạn (hotels) từ `catalog-service`.
- `PlanClient` (`app/clients/plan_client.py`) — client **có auth**, **forward bearer token của caller**, để đọc/ghi plan và board (lists/cards) trên `plan-service`.
- `WebSearchClient` (`app/clients/web_search_client.py`) — fallback web search qua **Tavily**, chỉ dùng khi catalog thiếu item hoặc thiếu giá; **không bao giờ raise**.

Cả ba đều dùng chung pattern khởi tạo: nhận `settings: Settings` và một `client: httpx.AsyncClient | None`. Nếu `client` được inject thì client đó được tái sử dụng (không tự đóng); nếu `None` thì mỗi request tự tạo một `AsyncClient` tạm rồi đóng ở khối `finally` (`owns_client/owns`). Đây là một gotcha quan trọng: khi không inject client, mỗi lời gọi mở/đóng connection riêng (không tái dùng connection pool).

Cả `CatalogClient` và `PlanClient` đều dựa trên một **API envelope chung** từ phía Java backend có hình dạng `{"success": bool, "message": str, "data": ...}`. Logic bóc tách thống nhất:

- `if not payload.get("success", True) → raise UpstreamError` với `payload.get("message", ...)` (lưu ý: nếu thiếu field `success` thì mặc định coi là `True`, tức thành công).
- Trả về `payload.get("data")` or `{}` — nếu `data` là `None` hoặc rỗng/falsy thì trả về `{}` (không bao giờ trả `None`).

`UpstreamError` được import từ `app.core.errors` và là loại lỗi thống nhất khi gọi service bên ngoài thất bại (`app/clients/catalog_client.py:6`, `app/clients/plan_client.py:6`).

`app/clients/catalog_client.py` — `CatalogClient`

Trách nhiệm: client read-only cho `catalog-service`. Docstring nêu rõ "Endpoints are public, no auth needed" (`app/clients/catalog_client.py:10`) — nên class này **không gắn header Authorization** ở bất kỳ đâu.

Khởi tạo và env

`__init__(self, settings, client=None)` (`app/clients/catalog_client.py:12`): `self._base_url = settings.catalog_service_base_url.rstrip("/")` (tránh double slash khi nối path), `self._timeout = settings.http_timeout_seconds`, `self._client = client`.

`_request` — helper gọi HTTP nội bộ (`app/clients/catalog_client.py:17`)

- **Tham số:** `method` (HTTP verb), `path` (ghép vào sau `_base_url`), `**kwargs` được forward thẳng vào `client.request` (thực tế là `params=` và `json=`).
- **Giá trị trả về:** `Dict[str, Any]` — chính là phần `data` của envelope (hoặc `{}`).
- **Xử lý lỗi / edge case:**
 - Chỉ status `>= 500` mới raise `UpstreamError`. **Gotcha quan trọng:** các lỗi 4xx (400/404/422...) **không** bị chặn ở đây — code chạy thẳng xuống `response.json()`. Nếu `catalog-service` trả 4xx kèm envelope `success: false` thì sẽ bị bắt bởi nhánh `success`; nhưng nếu body 4xx không phải JSON hợp lệ thì `response.json()` raise `ValueError` mà `ValueError` **không** nằm trong `except httpx.HTTPError` → exception này sẽ thoát ra ngoài chưa được bọc thành `UpstreamError`.
 - Mọi lỗi mạng/timeout (`httpx.HTTPError`) → bọc thành `UpstreamError("catalog-service unreachable: ...")`.
- **Side effects:** một HTTP request tới `catalog-service`; tự đóng client nếu `owns_client`.

`search_places(q, page=0, size=10) -> List[Dict[str, Any]]` (`app/clients/catalog_client.py:34`)

- **Endpoint:** `POST /api/catalog/places/search`. Body chỉ có `{"q": q}`; phân trang qua query param `page/size`.
- **Trả về:** list các place dict, lấy từ `data.content` (Spring Page) hoặc fallback `data.items`, cuối cùng `[]`.
- **Gotcha:** index free-text của endpoint này giữ nguyên dấu tiếng Việt, nên truy vấn đã bỏ dấu kiểu `"Đà Nẵng"` sẽ **miss** các POI `"Đà Nẵng"`. Đó là lý do tồn tại biến thể `faceted`.

`search_places_faceted(parent_slug=None, q=None, page=0, size=10) -> List[Dict[str, Any]]` (`app/clients/catalog_client.py:43`)

- **Endpoint:** `POST /api/catalog/places/search/faceted`.
- **Mục đích:** scope theo `parentSlug` (slug của một điểm đến, ví dụ `'da-nang'`) cho kết quả **deterministic** và **accent-proof** — khắc phục hạn chế bỏ-dấu của `/search` free-text.
- **Hình dạng response khác biệt:** list nằm dưới `data.results` (KHÔNG phải `data.content`); fallback đọc `data.content` rồi cuối cùng `[]`.
- **Field gửi đi:** `parentSlug, q` (chỉ thêm vào body nếu truthy).

`search_restaurants(location, page=0, size=10, **filters) -> List[Dict[str, Any]]` ([app/clients/catalog_client.py:66](#))

- **Endpoint:** POST `/api/catalog/restaurants/search`.
- **Tham số:** `location` (nếu `truthy` → thêm field `location` vào body) và `**filters` tùy ý; chỉ những filter có giá trị **không phải None** mới được đưa vào body (giá trị falsy khác như `0`, `""`, `False` vẫn được gửi).
- **Trả về:** list restaurant dict từ `data.content` → `data.items` → `[]`.

`search_hotels(location, page=0, size=10, **filters) -> List[Dict[str, Any]]` ([app/clients/catalog_client.py:80](#))

Hoàn toàn đối xứng với `search_restaurants`, chỉ khác endpoint `POST /api/catalog/hotels/search`. Cùng quy tắc lọc `is not None`, cùng fallback `content/items/[]`.

`app/clients/plan_client.py` — `PlanClient`

Trách nhiệm: client **có xác thực** cho `plan-service`; mọi method nhận `bearer_token` và forward nó dưới dạng header `Authorization: Bearer <token>` ([app/clients/plan_client.py:10](#)). Đây là cách AI service hành động "thay mặt" user, để `plan-service` tự scope quyền truy cập theo user id trong token.

Class chia làm hai nhóm helper request:

1. `_request` ([app/clients/plan_client.py:17](#)) — helper "đời đầu", hỗ trợ tham số `idempotency_key`, hiện chỉ được `create_plan` dùng.
2. `_raw_request` ([app/clients/plan_client.py:303](#)) + các wrapper `_raw_get/_raw_post` — helper tổng quát hơn, nhận `extra_headers`; được hầu hết các edit operation dùng.

Ngoài ra `list_my_plans` và `search_plans` **không dùng helper nào** mà lặp lại toàn bộ logic request bằng tay (gotcha duplicate code).

Khác biệt xử lý lỗi so với CatalogClient

Tất cả helper của `PlanClient` có thêm một nhánh kiểm tra auth trước nhánh 5xx (ví dụ [app/clients/plan_client.py:39](#) và [app/clients/plan_client.py:323](#)):

- 401/403 → `UpstreamError("plan-service rejected token (...))` — tín hiệu token sai/hết hạn/không đủ quyền.
- 5xx → `UpstreamError`.
- Cùng gotcha như `catalog`: các 4xx khác (400/404/422...) **không** bị chặn riêng, đi thẳng xuống `response.json()`; nếu body không phải JSON thì `ValueError` thoát ra (không nằm trong `except httpx.HTTPError`).

`_request` (có `idempotency`) và `env`

`__init__(self, settings, client=None)` ([app/clients/plan_client.py:12](#)): `self._base_url = settings.plan_service_base_url.rstrip("/")`, `self._timeout = settings.http_timeout_seconds`. `_request(method, path, bearer_token, *, json_body=None, idempotency_key=None)` ([app/clients/plan_client.py:17](#)): build header `Authorization + Content-Type: application/json`; nếu có `idempotency_key` thì thêm header `Idempotency-Key`. Trả về `payload.get("data")` or `{}`.

`create_plan(bearer_token, body)` ([app/clients/plan_client.py:53](#))

POST `/api/plans`, gửi body làm `json_body`. Method duy nhất còn gọi qua `_request`. Trả về `data` của plan vừa tạo (chứa `id` plan).

`list_my_plans(bearer_token, page=1, size=10)` ([app/clients/plan_client.py:56](#))

- **Endpoint:** GET `/api/plans/me`, query `page/size`. Read-only, paginated list plan của user hiện tại.
- **Gotcha:** method tự viết lại toàn bộ block try/finally (không gọi helper). Lưu ý mặc định `page=1` (1-based) ở đây, khác với `catalog` dùng `page=0`.

`search_plans(bearer_token, *, q=None, destinations=None, budget_min=None, budget_max=None, days_min=None, days_max=None, size=8)` ([app/clients/plan_client.py:86](#))

- **Endpoint:** GET `/api/plans/search`.
- **Mục đích:** tìm các plan mà caller được phép truy cập (của chính mình + public/của bạn bè/được share). `plan-service` tự scope visibility theo user id của bearer.
- **Tham số → query param:** `size`, cố định `sortBy=RELEVANCE` và `userLimit=0`; `q`, `budgetMin`, `budgetMax`, `daysMin`, `daysMax` chỉ thêm khi `truthy` / `is not None`; `destinations` là **repeated query param** (`List<String>` phía Java) — `httpx` serialize list thành nhiều cặp `destinations=...`.
- **Lưu ý mapping:** `budget_min/max` không phân biệt `0 falsy` vì dùng `is not None`; còn `q` dùng `if q` nên chuỗi rỗng bị bỏ qua.
- **Trả về:** `PlanSearchResponse.data` có hình dạng `{query, plans: {items, ...}, users}`.

Board: đọc và sửa lists/cards

- `get_board(bearer_token, plan_id)` ([app/clients/plan_client.py:141](#)): GET `/api/plans/{plan_id}/board` (qua `_raw_get`). Trả về `BoardResponse`. **Mục đích quan trọng:** `plan-service` tự **seed một DAY list cho mỗi ngày** trong khoảng ngày của plan ngay lúc create. Nên đây là cách để khám phá `id` của các list đã có sẵn để attach card vào — thay vì tạo thêm ngày mới.
- `rename_list(bearer_token, plan_id, list_id, title)` ([app/clients/plan_client.py:147](#)): PUT `/api/plans/{plan_id}/board/lists/{list_id}/rename`, body `{"title": title}`.
- `create_list(bearer_token, plan_id, body, idempotency_key=None)` ([app/clients/plan_client.py:161](#)): POST `/api/plans/{plan_id}/board/lists`. **DEPRECATED cho approval flow:** vì `plan-service` đã auto-seed DAY list, endpoint này APPEND thêm một ngày dư (`dayDate = endDate + 1`) và response `id` trả về `null`. `ApprovalService` nay đọc các seeded list qua `get_board`. Method này chỉ giữ lại cho dùng ad-hoc (vd `add_day` ở edit flow).
- `create_card(bearer_token, plan_id, list_id, body, idempotency_key=None)` ([app/clients/plan_client.py:180](#)): POST `/api/plans/{plan_id}/board/cmd/lists/{list_id}/cards` — chú ý dùng **command endpoint** (`/cmd/`) để header `Idempotency-Key` **thực sự được tôn trọng**. Đường legacy `/board/lists/{id}/cards` bỏ qua key này → một lần approve bị retry sẽ **tạo trùng card** (gotcha quan trọng). Caller (approval/edit) **không đọc** giá trị trả về — chỉ cần write thành công.

- **update_card** ([app/clients/plan_client.py:202](#)): PUT `/api/plans/{plan_id}/board/lists/{list_id}/cards/{card_id}`.
- **delete_card** ([app/clients/plan_client.py:217](#)): DELETE `.../cards/{card_id}`.
- **move_card** (`bearer_token`, `plan_id`, `card_id`, `target_list_id`, `target_position=None`, `idempotency_key=None`) ([app/clients/plan_client.py:226](#)): PATCH `/api/plans/{plan_id}/board/cmd/cards/{card_id}/move`. Body luôn có `{"targetListId": target_list_id}`; chỉ thêm `targetPosition` khi `target_position` is not None. Cũng dùng command endpoint `/cmd/`. Lưu ý method này **không cần list_id nguồn** (chỉ cần `card_id` + list đích).
- **delete_list** ([app/clients/plan_client.py:247](#)): DELETE `/api/plans/{plan_id}/board/lists/{list_id}`.

Cập nhật metadata plan

- **update_plan_title** ([app/clients/plan_client.py:254](#)): PATCH `/api/plans/{plan_id}/title`, body `{"title": title}`.
- **update_plan_dates** ([app/clients/plan_client.py:259](#)): PATCH `/api/plans/{plan_id}/dates`, body `{"startDate", "endDate"}` (chuỗi ngày).
- **update_plan_budget** (`bearer_token`, `plan_id`, `budget_total`, `budget_per_person=None`, `currency="VND"`) ([app/clients/plan_client.py:269](#)): PUT `/api/plans/{plan_id}/budget`. **Gotcha**: nếu `budget_per_person` là None thì `budgetPerPerson` được set bằng `budget_total` ([app/clients/plan_client.py:281](#)).

Helper tổng quát: `_raw_request` / `_raw_post` / `_raw_get`

`_raw_request` ([app/clients/plan_client.py:303](#)) là trục chính cho mọi method board/edit/metadata; merge thêm `extra_headers` (ví dụ `Idempotency-Key`). `_raw_post` ([app/clients/plan_client.py:288](#)) và `_raw_get` ([app/clients/plan_client.py:300](#)) chỉ là wrapper mỏng. Cùng quy tắc trả về `payload.get("data")` or `{}` và cùng xử lý lỗi 401/403, 5xx, `httpx.HTTPError`.

Tổng kết hàng số/quy ước của PlanClient:

- Header bắt buộc: `Authorization: Bearer ...`, `Content-Type: application/json`; tùy chọn `Idempotency-Key`.
- Quy ước endpoint command vs legacy: dùng `/board/cmd/...` cho `create_card` & `move_card` để `idempotency` được honor.
- Param tìm kiếm cố định: `sortBy="RELEVANCE"`, `userLimit=0`.

app/clients/web_search_client.py — WebSearchClient

Trách nhiệm (docstring [app/clients/web_search_client.py:1](#)): web search **fallback**, chỉ dùng khi catalog thiếu item hoặc thiếu giá. Provider mặc định là **Tavily** — JSON sạch, free tier rộng, hợp để lấy giá hiện hành (ví dụ "giá thuê xe máy Đà Nẵng", "vé Bà Nà Hills 2025"). Lý do dùng API search riêng: 9Router không expose model nào có web-grounding qua OpenAI Chat Completions API.

Degrade gracefully: nếu **không cấu hình** `WEB_SEARCH_API_KEY`, thuộc tính `enabled` là `False` và caller fallback về heuristic pricing — agent được thông báo điều này để có thể trả lời trung thực.

Hàng số & khởi tạo

```
_TAVILY_URL = "https://api.tavily.com/search"; logger tên "ai_plan.web_search".
__init__(self, settings, client=None)
(app/clients/web_search_client.py:23): self._provider = (settings.web_search_provider or "tavily").strip().lower(), self._api_key =
(settings.web_search_api_key or "").strip(), self._timeout = settings.http_timeout_seconds, self._client = client.
```

enabled (property) ([app/clients/web_search_client.py:33](#))

Trả `True` chỉ khi có api key không rỗng. Lưu ý: `enabled` **không** kiểm tra provider; việc provider có hợp lệ hay không được kiểm trong `search`.

search(query, max_results=5) -> Dict[str, Any] ([app/clients/web_search_client.py:37](#))

- **Endpoint/lời gọi ra ngoài**: POST `https://api.tavily.com/search` (gửi `api_key` trong body, không qua header).
- **Tham số gửi đi**: `query`; `max_results` bị **clamp về khoảng [1, 8]** qua `max(1, min(int(max_results), 8))` (gotcha: dù caller truyền số lớn hơn, Tavily chỉ được yêu cầu tối đa 8); `search_depth="basic"`; `include_answer=True`.
- **Hình dạng giá trị trả về** — luôn là Dict có ít nhất key `enabled` và `results`:
 - Chưa cấu hình key: `{"enabled": False, "results": [], "note": "web search not configured"}`.
 - Provider không hỗ trợ: `{"enabled": False, "results": [], "note": "unsupported provider <name>"}`.
 - HTTP ≥ 400 từ Tavily: `{"enabled": True, "results": [], "note": "search HTTP <code>"}` (đồng thời `logger.warning`).
 - Thành công: `{"enabled": True, "answer": <data.answer>, "results": [...]}`. Mỗi item trong `results` có ba field: `title`, `url`, `content` (bị **cắt còn 600 ký tự** đầu).
 - Lỗi mạng/parse: `{"enabled": True, "results": [], "note": "search unavailable"}`.
- **Edge case về số lượng**: request gửi `max_results` đã clamp ≤ 8 , nhưng vòng lặp lại slice thêm `[ :max_results ]` (giá trị gốc chưa clamp) — nếu `max_results` gốc $>$ số kết quả Tavily trả thì slice không cắt gì.

Triết lý xử lý lỗi — "Never raises" ([app/clients/web_search_client.py:75](#))

Gotcha / điểm khác biệt với hai client kia: `WebSearchClient.search` **không bao giờ raise**. Khác với `CatalogClient/PlanClient` (raise `UpstreamError`), ở đây mọi lỗi mạng (`httpx.HTTPError`) và lỗi parse JSON (`ValueError`) đều bị nuốt và trả về dict rỗng với `note`. Lý do: một lần search hỏng không được phép làm hỏng cả lượt planning. Đây cũng là điểm bất `ValueError` mà hai client REST kia không có.

Tổng hợp gotchas xuyên suốt cụm module

1. **Quản lý client**: không inject `httpx.AsyncClient` → mỗi request tự mở/đóng connection (không tái dùng pool). Inject một client dùng chung sẽ hiệu quả hơn cho các flow nhiều request liên tiếp.
2. **4xx không được xử lý tường minh** ở `CatalogClient/PlanClient`: chỉ chặn 401/403 (riêng `PlanClient`) và ≥ 500 . Các 4xx khác đi qua `response.json()`; nếu body không phải JSON → `ValueError` thoát ra ngoài không bọc thành `UpstreamError`.
3. **Envelope mặc định success=True**: thiếu field `success` được coi là thành công; `data falsy` → trả `{}`.

- 4. **Idempotency**: chỉ honor đúng khi gọi qua command endpoint `/board/cmd/...` (create_card, move_card). Đường legacy bỏ qua `Idempotency-Key` → retry sẽ nhân bản card.
- 5. **Accent-proofing**: ưu tiên `search_places_faceted(parent_slug=...)` thay vì `search_places(q=...)` khi cần kết quả chính xác cho điểm đến tiếng Việt có dấu.
- 6. `create_list deprecated` cho approval flow; dùng `get_board` để lấy seeded DAY list ids.
- 7. **WebSearchClient là fail-soft tuyệt đối**, còn hai client REST là fail-hard (raise `UpstreamError`).

Cấu hình (.env)

Mọi thứ swappable nằm trong `.env` (git-ignored) — copy từ `.env.example` rồi điền. Tên field trong `app/config.py` map sang env theo quy ước viết hoa (vd field `llm_api_key` ↔ env `LLM_API_KEY`); biến lạ bị bỏ qua (`extra="ignore"`). Mọi field đều có default nên service vẫn chạy được khi thiếu `.env`.

Biến (env)	Default	Ý nghĩa & ghi chú
AI_PLAN_HOST	0.0.0.0	Host bind khi chạy <code>python -m app.main</code> .
AI_PLAN_PORT	8092	Port của service.
CATALOG_SERVICE_BASE_URL	http://localhost:8083	Base URL catalog-service (read-only, public).
PLAN_SERVICE_BASE_URL	http://localhost:8086	Base URL plan-service (writes, có auth).
JWT_SECRET	"" (rỗng)	Secret HMAC để verify JWT. Rỗng → giải mã JWT KHÔNG verify chữ ký (plan-service verify lại khi ghi); set để harden luôn session ownership của ai-plan-service.
JWT_ALGORITHM	HS256	Thuật toán JWT.
LLM_API_KEY	9router-local	API key LLM. Rỗng → chế độ STUB (regex + <code>DraftComposer</code> tắt định, KHÔNG agent loop). Có → agent mode .
LLM_BASE_URL	http://localhost:20128/v1	Endpoint OpenAI-compatible. Đổi provider chỉ cần đổi biến này + <code>LLM_MODEL</code> .
LLM_MODEL	Mrave1	Model chính (Chat Completions).
LLM_EXTRACTOR_MODEL	""	Model rẻ/nhanh riêng cho bước trích constraint; rỗng → fallback về <code>LLM_MODEL</code> .
LLM_FALLBACK_MODELS	""	Danh sách model (phân tách dấu phẩy) thử khi model chính trả 429/402/5xx.
LLM_APP_NAME	Mrave1 AI Plan Agent	Tên app gửi kèm (header <code>X-Title</code> cho OpenRouter; vô hại với provider khác).
LLM_APP_URL	""	URL app gửi kèm (header <code>HTTP-Referer</code>).
LLM_MAX_TOKENS	8000	Max output tokens/lượt agent. Quan trọng : kế hoạch dày (nhiều card trong 1 lệnh <code>finalize_draft/propose_plan_edits</code>) cần nhiều token — để 2048 sẽ bị cắt cụt JSON.
WEB_SEARCH_PROVIDER	tavily	Provider web search.
WEB_SEARCH_API_KEY	""	Tavily key. Rỗng → web search bị tắt (<code>enabled=False</code>); agent dùng heuristic giá và nói rõ "ước tính/chưa xác minh".
PUBLIC_WEB_BASE_URL	""	Origin web app để tạo link tuyệt đối. Rỗng → link catalog tương đối (<code>/hotels/<slug></code>) để browser tự resolve theo origin SPA.
CORS_ORIGINS	http://localhost:5173,http://localhost:8080	Danh sách origin CORS (phân tách dấu phẩy).
HTTP_TIMEOUT_SECONDS	30.0	Timeout cho httpx client gọi downstream.
SESSION_STORE_BACKEND	mongo	Backend lưu session: <code>mongo</code> (khuyến nghị) <code>file</code> <code>memory</code> . Giá trị lạ rơi vào nhánh <code>mongo</code> .
MONGO_URI	mongodb://localhost:27017	Connection string MongoDB. Atlas thật để trong .env (chứa credential, không hardcode).
MONGO_DB	mrave1_catalog	Tên DB (session lưu chung DB với catalog, chỉ khác collection).
SESSION_COLLECTION	ai_plan_sessions	Collection lưu session.
SESSION_MIGRATE_FROM_FILE	True	Lần đầu khởi động Mongo, import session từ file legacy nếu collection rỗng.
SESSION_STORE_PATH	.data/sessions.json	Path file backend legacy / nguồn migrate.

Chọn LLM (OpenRouter trực tiếp vs 9Router)

Client là OpenAI-compatible, chạy được với OpenRouter / OpenAI / Groq / Together / Ollama / vLLM / 9Router — đổi provider chỉ cần đổi `LLM_BASE_URL` + `LLM_MODEL`.

A. OpenRouter trực tiếp (đơn giản nhất):

```
LLM_API_KEY=sk-or-v1-<key>
LLM_BASE_URL=https://openrouter.ai/api/v1
LLM_MODEL=openai/gpt-oss-120b:free      # nói thẳng OpenRouter → bỏ tiền tố "openrouter/"
```

B. 9Router (proxy cục bộ, cổng 20128, tự fallback/round-robin):

```
LLM_API_KEY=9router-local
LLM_BASE_URL=http://localhost:20128/v1
LLM_MODEL=openrouter/openai/gpt-oss-120b:free
```

Lưu ý 9Router: provider này hay bực lỗi upstream (429 rate-limit / 402 hết credit của model nền) bên trong HTTP 400. `openai_client._embedded_code/_status_code` ưu tiên cào mã thật từ body lỗi, và `_chat_models` thử model chính **hai lần** để tận dụng round-robin combo trước khi rơi sang fallback. Muốn kế hoạch dày và chi tiết ổn định → đổi `LLM_MODEL` sang model mạnh hơn (vd `google/gemini-2.5-flash`, `deepseek/deepseek-chat`, `gpt-4o-mini`); model free `gpt-oss-120b` khá yếu, hay ra it hoạt động.

MongoDB

Dùng cluster Atlas của dự án: điền `MONGO_URI` (giống chuỗi trong `catalog-service/.../application.yml`) và **whitelist IP** trong Atlas → Network Access. Nếu Mongo không kết nối được, factory `get_session_store()` sẽ log `falling back to file` và chuyển sang file/memory (lịch sử có thể mất khi restart nếu không cấu hình `SESSION_STORE_PATH`). Dev thuần local có thể dùng `mongodb://localhost:27017` hoặc `SESSION_STORE_BACKEND=memory`. Log khởi động in store đã chọn, vd `session store: mongo mravel_catalog/ai_plan_sessions`.

Bảng tra cứu file

File	Vai trò
app/main.py	App-factory FastAPI, CORS, đăng ký exception handlers, mount router, endpoint health, khởi <code>__main__</code> chạy uvicorn (no reload).
app/config.py	Class <code>Settings</code> (pydantic-settings) — toàn bộ env (LLM, Mongo, web search, CORS, max-tokens...); helper <code>has_llm/fallback_model</code> singleton.
app/api/ai_plan.py	TẤT CẢ 9 endpoint + dò plan id (<code>_extract_plan_id</code>) + gate quyền (<code>_assert_can_edit</code>) + đóng gói SSE (<code>_consume_agent_stream/_consume_edit_stream/_one_shot_assistant_stream</code>) + persist phiên.
app/api/schemas.py	DTO request/response: <code>CreateSessionRequest</code> , <code>SessionView</code> , <code>SessionSummary</code> , <code>SendMessageRequest/Response</code> , <code>RegenerateFeedbackRequest/Response</code> , <code>ApplyEditsResult</code> .
app/api/dependencies.py	DI wiring: singleton hạ tầng (<code>_catalog_client/_plan_client/_web_search_client/_llm_client</code> , <code>@lru_cache</code>) + factory service (<code>get_constraint_extractor/get_draft_composer/get_agent_orchestrator/get_approval_service/get_edit_service/get_geo_service</code>).
app/agent/orchestrator.py	<code>AgentOrchestrator</code> : vòng lặp agent <code>plan_stream/edit_stream</code> (≤ <code>MAX_ITERATIONS=10</code>), hai SYSTEM PROMPT (plan/edit), <code>_format_primer/_build_messages/_build_edit_messages</code> , các fallback (<code>_stub_path/_deterministic_fallback</code>), tiện ích <code>plan_summary</code> .
app/agent/tools.py	<code>tool_definitions/edit_tool_definitions</code> (schema tool LLM thấy); <code>ToolDispatcher</code> thực thi tool read-only; <code>catalog_web_url_summarise_*</code> ; <code>apply_set_constraints</code> ; <code>parse_finalize_draft</code> ; <code>strip_accents/location_slug</code> .
app/agent/edits.py	Model <code>EditOperation</code> (flat), <code>OP_TYPES</code> , <code>parse_operations</code> (validate + drop id ảo qua board), <code>_has_required_targets/_ids_exist</code> , <code>propose_tool_schema</code> .
app/llm/base.py	Interface <code>LLMClient</code> (ABC) + dataclass <code>ToolDefinition/ToolUse/LLMTurn.to_openai_tool()</code> dịch sang format OpenAI.
app/llm/openai_client.py	<code>OpenAILLMClient</code> — gọi HTTP OpenAI-compatible, <code>chat_with_tools/extract_constraints</code> , fallback nhiều model (<code>_chat_models/_embedded_code/_status_code/_is_retryable/_friendly_error</code>).
app/llm/stub.py	<code>StubLLMClient</code> — trích constraint thuần regex (offline, <code>supports_tool_use()=False</code>); whitelist <code>_VN_DESTINATIONS</code> , parse ngày tháng.
app/llm/factory.py	<code>build_llm_client</code> — chọn <code>StubLLMClient</code> (key rỗng) hay <code>OpenAILLMClient</code> (import lazy).
app/core/response.py	Envelope <code>ApiResponse[T]</code> (<code>success/message/data/timestamp</code>), constructor <code>ok/err</code> ; khớp <code>ApiResponse<T></code> Java.
app/core/errors.py	Exception miền (<code>DomainError</code> →400, <code>UnauthorizedError</code> →401, <code>ForbiddenError</code> →403, <code>UpstreamError</code> →502) + <code>register_exception</code> (<code>RequestValidationError/StarletteHTTPException</code>).
app/core/security.py	<code>CurrentUser</code> (dataclass frozen: <code>id/raw_token</code>), <code>_decode_subject</code> (verify/không-verify JWT), dependency <code>get_current_user</code> .
app/services/session_store.py	<code>InMemorySessionStore</code> / <code>FileBackedSessionStore</code> / <code>MongoSessionStore</code> + factory <code>get_session_store()</code> (chọn backend, fallback file).
app/services/constraint_extractor.py	<code>ConstraintExtractor.update</code> — lai regex (stub luôn chạy) + LLM refine; <code>_normalize</code> ép kiểu giữ prior; bước 1b khôi phục destir từ <code>plan_summary</code> .
app/services/draft_composer.py	<code>DraftComposer.compose</code> — dựng nháp tất định, geo-aware, per-person (đường stub/fallback); <code>_PACE_SLOTS</code> , <code>_Picker</code> , <code>_build_summary</code> .
app/services/approval_service.py	<code>ApprovalService.approve</code> — draft → plan thật trên plan-service (create_plan + cards vào DAY list seed sẵn); <code>_activity_to_cards</code> .
app/services/edit_service.py	<code>EditService.apply</code> — <code>EditOperation</code> đã duyệt → board mutations plan-service; <code>_card_body</code> , dispatch <code>_apply_one</code> , batch iderr.
app/services/geo.py	<code>Haversine</code> , <code>pick_mode/travel_minutes/leg_between/nearest_index/route_legs</code> ; pure + deterministic.
app/services/pricing.py	Ước tính chi phí per-person: <code>_BANDS/_PRICE_LEVEL_MULT</code> , <code>meal_cost_per_person/place_cost_per_person/lodging_total/tax</code> .

File	Vai trò
app/services/catalog_location.py	catalog_location_fields — map catalog recommendation → shape modal frontend (hotelLocation/restaurantLocation/...); loại
app/clients/catalog_client.py	CatalogClient REST (read-only, no auth): search_hotels/search_restaurants/search_places/search_places_faceted.
app/clients/plan_client.py	PlanClient REST (auth, forward bearer): create_plan/get_board/create_card/update_card/delete_card/move_card/rename_list/delete_list/create_list/upd
app/clients/web_search_client.py	WebSearchClient (Tavily) — fail-soft, never raises; enabled theo WEB_SEARCH_API_KEY.
app/models/session.py	Domain model: PlanSession, Constraints, PlanDraft, DraftDay, DraftActivity, RecommendationRef, ChatMessage + enum PlanActivityType/PlanVisibility/ChatRole/SessionStatus.

Thuật ngữ & Gotchas

Glossary ngắn

- **MAI** — Mravel AI, trợ lý du lịch; mặt ngoài của ai-plan-service.
- **Draft-first** — AI chỉ đề xuất; không ghi plan thật cho đến khi user bấm Duyệt (/approve) / Áp dụng (/apply-edits).
- **Plan mode** (plan_id == None) — dựng PlanDraft mới rồi Duyệt thành plan. **Edit mode** (plan_id != None) — chat sửa plan có sẵn qua propose_plan_edits rồi Áp dụng.
- **Constraints** — ràng buộc chuyến đi (destination/num_days/dates/travelers/budget/interests/pace) tích lũy dần; agent là single source of truth qua set_constraints.
- **PlanDraft** — bản nháp itinerary nhiều ngày (days[].activities[]) chờ duyệt.
- **EditOperation / pending_edits** — thao tác sửa đề xuất (8 loại) chờ user Áp dụng.
- **Stub mode** — không có LLM_API_KEY: regex extractor + DraftComposer tắt định, không agent loop.
- **Agent loop** — vòng chat_with_tools ≤ MAX_ITERATIONS=10; terminal tool là finalize_draft (plan) / propose_plan_edits (edit).
- **Primer** — user-message "mời" tiềm hôm-nay + năm hiện tại + known constraints (plan) / board summary với list_id/card_id (edit); KHÔNG nằm trong system prompt.
- **mravel_url** — link tới SPA frontend (/hotels/<slug>, /place/<slug>, /plans/<id>) để chat có link bấm được.
- **Seeded DAY list** — plan-service tự tạo 1 list/ngày lúc create plan; approve gắn card vào các list này, KHÔNG tạo list mới.
- **ApiResponse envelope** — {success, message, data, timestamp} chung với backend Java.

Gotchas quan trọng

- **Plan id vs duration/đếm** — _extract_plan_id chỉ bind khi gặp /plans/43 hoặc "kế hoạch/plan [số] 43"; negative-lookahead _PLAN_UNIT loại "kế hoạch 3 ngày" / "plan 2 người" để không nhầm con số đếm thành plan id.
- **Model không được in card_id/list_id/JSON** trong chat — đó là plumbing nội bộ; UI hiển thị approval cards. Quy tắc enforce ở _EDIT_SYSTEM_PROMPT (tone rules), nhưng KHÔNG có cơ chế chặn cứng nếu model lỡ lộ — phụ thuộc model.
- **Small-model caveat** — model free gpt-oss-120b khá yếu; prompt viết phòng thủ/tường mình và có nhiều tầng deterministic fallback. Đổi LLM_MODEL sang model mạnh hơn cho itinerary dày/ổn định hơn.
- **add_day cần 2 lượt** — create_list thực tế trả id null, nên EditService._apply_one phải RE-READ board để lấy new_list_id của ngày vừa thêm; do đó dựng plan từ 1 ngày thường cần 2 lượt (thêm ngày → rồi điền hoạt động vào ngày mới).
- **LLM_MAX_TOKENS=8000** — kế hoạch dày (nhiều card trong một lệnh) cần nhiều output tokens; để mặc định OpenAI 2048 sẽ bị cắt cụt JSON của finalize_draft/propose_plan_edits.
- **Chỉ có num_days, không có ngày lịch** — extractor/composer KHÔNG bịa today+7; ngày được neo về hôm nay chỉ tại Constraints.resolved_date_range() (phụ thuộc date.today(), không deterministic theo thời gian).
- **Idempotency** — chỉ honor khi gọi qua endpoint command /board/cmd/... (create_card, move_card); đường legacy bỏ qua Idempotency-Key → retry nhận bản card. Approve dùng key deterministic theo (plan_id, day, position); apply-edits dùng _batch_seed (hash nội dung tập ops).
- **Date-range đảo ngược** — Constraints._validate_date_range tự HOÁN ĐỔI (không raise) để không gây giữa hội thoại; DraftActivity cost âm thì RAISE. Validate cứng hướng-người-dùng dời sang finalize_draft/EditService (start>end → raise).
- **4xx không bọc tướng mình** ở CatalogClient/PlanClient — chỉ chặn 401/403 (PlanClient) và ≥500; body 4xx không phải JSON → ValueError thoát ra ngoài (không thành UpstreamError).
- **9Router bọc lỗi trong HTTP 400** — openai_client cáo mã thật (429/402...) từ body, bỏ qua 400 wrapper; thử model chính 2 lần để tận dụng round-robin.
- **JWT không verify khi JWT_SECRET rỗng** — chấp nhận được cho service nội bộ sau gateway (plan-service verify lại khi ghi), nhưng KHÔNG expose trực tiếp ra internet mà thiếu secret.
- **Truthiness filter** — _activity_to_card_body / _card_body lọc activity_data bằng if v (loại cả 0/""), trong khi catalog_location_fields chỉ loại None/"" (giữ 0).
- **Fallback Mongo âm thầm** — Mongo chết lúc khởi động → log warning rồi dùng file/memory; lịch sử có thể mất khi restart nếu không cấu hình SESSION_STORE_PATH.
- **.claude/... & .data/** — file ~/ .claude/... là ghi chú của Claude Code (không phải app); .data/sessions.json là backend file legacy (git-ignored).